

```

      value={background}
      onChange={chooseBackground}
      options={BACKGROUNDS.map((b) => ({
        id: b.id,
        title: b.hint,
        label: (
          <>
            <Switch kind={b.id} />
            {b.label}
          </>
        ),
      })))
    />
    <p className="-mt-1.5 px-0.5 text-[11px] leading-snug text-muted-foreground">
      {BACKGROUNDS.find((b) => b.id === background)?.hint}
    </p>
    <Segmented
      label="Sharpness"
      value={scale}
      onChange={(v) => setScale(v)}
      options={CAPTURE_SCALES.map((s) => ({
        id: s,
        label: `${s}x`,
        title: `${Math.round(region.width * s)} × ${Math.round(region.height * s)} px`,
      })))
    />
    {request.selectionIds && request.selectionIds.length > 0 && (
      <Segmented
        label="Include"
        value={onlySelection ? "sel" : "all"}
        onChange={(v) => setOnlySelection(v === "sel")}
        options={[
          { id: "all", label: "Whole area" },
          { id: "sel", label: "Selection only" },
        ]}
      />
    )}
    {missing && (
      <p className="rounded-lg bg-amber-500/10 px-2.5 py-1.5 text-[11px] leading-snug text-amber-700 dark:text-amber-300">
        {missing}
      </p>
    )}
  </section>

  <section>
    <div className="mb-1.5 text-[10px] font-semibold uppercase tracking-wider text-muted-foreground">
      Send
    </div>
    <div className="grid grid-cols-3 gap-2">
      <SendTile icon={Copy} label={copied ? "Copied" : "Copy"} onClick={copy} busy={busy === "copy"} done={copied} disabled={!ready} />
      <SendTile
        icon={Share2}
        label="Share..."
        onClick={share}
        busy={busy === "share"}
        primary
        disabled={!ready}
      />
      <SendTile
        icon={native ? GalleryIcon : Download}
        label={native ? "Gallery" : "Save"}
        onClick={save}
        busy={busy === "save"}
        disabled={!ready}
      />
    </div>
    {!native && !canChooseApp() && (
      <p className="mt-1.5 px-0.5 text-[10.5px] leading-snug text-muted-foreground">
        This browser has no app chooser, so Share saves the file instead.
      </p>
    )}
  </section>

  <section>
    <div className="mb-1 text-[10px] font-semibold uppercase tracking-wider text-muted-foreground"

```

```

d">
  Add to notes
</div>
<div className="-mx-1 space-y-0.5">
  <AddRow
    icon={ImagePlus}
    title={notebook ? "On this page" : "On this board"}
    hint="Placed in view, ready to move"
    onClick={addHere}
    disabled={!ready || !!busy}
  />
  {notebook && (
    <AddRow
      icon={FilePlus2}
      title="On a new page"
      hint={livePage ? `Inserted after page ${livePage}` : "Inserted after this page"}
      onClick={() => addTo(board.id, true)}
      disabled={!ready || !!busy}
    />
  )}
  <AddRow
    icon={BookOpen}
    title="In another notebook..."
    hint="On a new page at its end"
    onClick={() => setView("pick")}
    disabled={!ready || !!busy}
    chevron
  />
</div>
</section>
</>
) : (
<section className="flex min-h-0 flex-1 flex-col">
  <div className="mb-2 flex items-center gap-1.5">
    <button
      type="button"
      onClick={() => setView("main")}
      aria-label="Back"
      className="flex h-8 w-8 items-center justify-center rounded-full text-muted-foreground transition hover:bg-muted hover:text-foreground"
    >
      <ArrowLeft size={17} />
    </button>
    <span className="text-[13px] font-semibold">Where should it go?</span>
  </div>
  <label className="mb-2 flex h-9 items-center gap-2 rounded-xl border border-border bg-muted/40 px-2.5 focus-within:border-indigo-400 dark:bg-zinc-800/60">
    <Search size={14} className="shrink-0 text-muted-foreground" />
    <input
      autoFocus
      value={query}
      onChange={(e) => setQuery(e.target.value)}
      placeholder="Search notebooks and boards"
      className="min-w-0 flex-1 bg-transparent text-[13px] outline-none placeholder:text-muted-foreground"
    />
  </label>
  <div className="-mx-1 max-h-[46vh] min-h-[120px] flex-1 space-y-0.5 overflow-y-auto @[40rem]:max-h-none">
    {others.length === 0 && (
      <p className="px-2 py-6 text-center text-[12px] text-muted-foreground">
        {query ? "Nothing matches." : "There are no other notebooks yet."}
      </p>
    )}
    {others.map((b) => {
      const nb = isNotebook(b);
      const count = nb ? getPages(b).length : 0;
      return (
        <AddRow
          key={b.id}
          icon={nb ? NotebookPen : LayoutGrid}
          title={b.name || "Untitled"}
          hint={nb ? `Notebook · ${count} page${count === 1 ? "" : "s"} · new page ${count + 1}` : "Board · placed beside its content"}
          onClick={() => addTo(b.id, false)}
          disabled={!busy}
        />
      )
    })}
  </div>
);

```

```

        }}}
      </div>
    </section>
  )}
</div>
</div>
</div>
</DialogContent>
);
}

=====
// === END FILE: src/components/whiteboard/CapturePanel.tsx ===
=====

=====
// === START FILE: src/components/whiteboard/ColorSwatches.tsx ===
// File 46 of 194 | Lines: 147 | Size: 4142 bytes
=====

"use client";

/**
 * Colour controls shared by the contextual property bar and the tool panel.
 *
 * Includes a real custom-colour control (the rainbow disc), because a fixed
 * palette is the single most common complaint about note apps' pen tools –
 * users want *their* colour, not the nearest preset.
 */

import React from "react";
import { useWhiteboard } from "@store/whiteboard";
import { cn } from "@lib/utils";

/** One round colour chip. */
export function Swatch({
  color,
  active,
  onSelect,
  size = 22,
  title,
}: {
  color: string;
  active: boolean;
  onSelect: () => void;
  size?: number;
  title?: string;
}) {
  return (
    <button
      onClick={onSelect}
      title={title ?? color}
      aria-label={title ?? color}
      className={cn(
        "shrink-0 rounded-full border transition-transform",
        active
          ? "scale-110 border-foreground ring-2 ring-primary ring-offset-1 ring-offset-background"
          : "border-black/15 hover:scale-105 dark:border-white/20"
      )}
      style={{ backgroundColor: color, width: size, height: size }}
    />
  );
}

/**
 * The rainbow disc: opens the OS colour picker. Rendered as a label wrapping a
 * visually-hidden <input type="color"> – the native control is the only way to
 * get a full picker (including eyedropper on desktop) without shipping one.
 */
export function CustomColorButton({
  value,
  onChange,
  size = 22,
}: {
  value: string;
  onChange: (color: string) => void;
  size?: number;

```

```

    }) {
      return (
        <label
          title="Custom colour..."
          className="relative shrink-0 cursor-pointer rounded-full border border-black/15 transition-transform hov
            er:scale-105 dark:border-white/20"
          style={{
            width: size,
            height: size,
            background:
              "conic-gradient(#ef4444,#f97316,#eab308,#22c55e,#06b6d4,#3b82f6,#8b5cf6,#ec4899,#ef4444)",
          }}
        >
          <input
            type="color"
            value={value}
            onChange={(e) => onChange(e.target.value)}
            className="absolute inset-0 h-full w-full cursor-pointer opacity-0"
          />
          { /* Hole in the middle so the disc reads as a picker, not a colour. */ }
          <span
            className="pointer-events-none absolute rounded-full bg-background"
            style={{ inset: size * 0.3 }}
          />
        </label>
      );
    }
  }

  /**
   * Palette + recents + custom, as used inside the tool panel.
   * `compact` renders the single row used by the contextual bar.
   */
  export function ColorRow({
    colors,
    value,
    onChange,
    compact = false,
    swatchSize = 22,
  }): {
    colors: string[];
    value: string;
    onChange: (c: string) => void;
    compact?: boolean;
    swatchSize?: number;
  } {
    const recentColors = useWhiteboard((s) => s.recentColors);
    const addRecentColor = useWhiteboard((s) => s.addRecentColor);

    const pick = (c: string) => {
      onChange(c);
      addRecentColor(c);
    };

    // In the compact row, recents come first: the colour you just mixed is the
    // one you are most likely to want again, and it would otherwise be lost.
    const shown = compact
      ? [...recentColors.filter((c) => !colors.includes(c)).slice(0, 2), ...colors].slice(0, 7)
      : colors;

    return (
      <div className={cn("flex items-center gap-1.5", !compact && "flex-wrap")}>
        {shown.map((c) => (
          <Swatch
            key={c}
            color={c}
            size={swatchSize}
            active={value.toLowerCase() === c.toLowerCase()}
            onSelect={() => pick(c)}
          />
        ))}
        <CustomColorButton value={value} onChange={pick} size={swatchSize} />

        { !compact && recentColors.length > 0 && (
          <div className="mt-2 w-full">
            <div className="mb-1 text-[11px] text-muted-foreground">Recent</div>
            <div className="flex flex-wrap items-center gap-1.5">
              {recentColors.map((c) => (
                <Swatch

```

```

        key={`recent-${c}`}
        color={c}
        size={swatchSize}
        active={value.toLowerCase() === c.toLowerCase()}
        onSelect={() => pick(c)}
      />
    ))}
  </div>
</div>
)}
</div>
);
}

=====
// === END FILE: src/components/whiteboard/ColorSwatches.tsx ===
=====

=====
// === START FILE: src/components/whiteboard/DockControls.tsx ===
// File 47 of 194 | Lines: 122 | Size: 4535 bytes
=====

"use client";

/**
 * Movable tool dock
 * =====
 * The dock can attach to any edge. Pinning it to the bottom forever wastes the
 * long side of a landscape tablet and parks the tools right under your writing
 * hand – moving it is the fix, and it is a setting every serious note app has.
 *
 * The grip is a deliberate 4-way picker rather than a free drag: on a touch
 * screen a drag that starts on the toolbar is indistinguishable from a
 * mis-aimed tap on a tool, and a dock only has four useful places to be.
 */

import React from "react";
import { GripVertical, PanelBottom, PanelLeft, PanelRight, PanelTop } from "lucide-react";
import { useWhiteboard } from "@store/whiteboard";
import { DOCK_POSITIONS, isVerticalDock, type DockPosition } from "@lib/whiteboard/types";
import { Popover, PopoverContent, PopoverTrigger } from "@components/ui/popover";
import { cn } from "@lib/utils";

const DOCK_ICONS: Record<DockPosition, React.ComponentType<{ size?: number }>> = {
  top: PanelTop,
  bottom: PanelBottom,
  left: PanelLeft,
  right: PanelRight,
};

/**
 * Where the whole dock stack sits, and which way it runs.
 *
 * The properties bar is always placed on the CANVAS side of the tools – above
 * a bottom dock, below a top one, and to the inside of a side dock – so the
 * thing you are configuring never sits between you and the edge.
 */
export function dockStackClasses(pos: DockPosition): string {
  switch (pos) {
    case "top":
      // BELOW the top bars, not level with them: at top-4 the dock sat on the
      // sidebar button (top-left) and the settings bar (top-right) and was
      // cut by them. Those bars end at ~60px.
      return "top-[72px] left-1/2 -translate-x-1/2 flex-col-reverse items-center";
    case "left":
      return "left-4 top-1/2 -translate-y-1/2 flex-row-reverse items-center";
    case "right":
      return "right-4 top-1/2 -translate-y-1/2 flex-row items-center";
    case "bottom":
      return "bottom-6 left-1/2 -translate-x-1/2 flex-col items-center";
    default:
      return "bottom-6 left-1/2 -translate-x-1/2 flex-col items-center";
  }
}

/** Caps the dock so it can never grow past the window on its long axis. */
export function dockSizeClasses(pos: DockPosition): string {

```

```

    return isVerticalDock(pos) ? "max-h-[calc(100%-2rem)]" : "max-w-[calc(100%-2rem)]";
  }

  /** Which side a flyout should open towards so it lands over the canvas. */
  export function flyoutSide(pos: DockPosition): "top" | "bottom" | "left" | "right" {
    switch (pos) {
      case "top":
        return "bottom";
      case "left":
        return "right";
      case "right":
        return "left";
      case "bottom":
        return "top";
      default:
        return "top";
    }
  }

  export function DockGrip({ className }: { className?: string }) {
    const position = useWhiteboard((s) => s.toolbarPosition);
    const setPosition = useWhiteboard((s) => s.setToolbarPosition);
    const [open, setOpen] = React.useState(false);
    const vertical = isVerticalDock(position);

    return (
      <Popover open={open} onOpenChange={setOpen}>
        <PopoverTrigger asChild>
          <button
            title="Move the toolbar to another edge"
            aria-label="Move toolbar"
            className={cn(
              "flex shrink-0 items-center justify-center rounded-lg text-muted-foreground transition-colors hover:
                r:bg-muted hover:text-foreground",
              vertical ? "h-6 w-8" : "h-8 w-6",
              className
            )}
          >
            <GripVertical size={14} className={vertical ? "rotate-90" : undefined} />
          </button>
        </PopoverTrigger>
        <PopoverContent side={flyoutSide(position)} className="w-auto p-1.5 align="center">
          <div className="mb-1 px-1 text-[11px] font-medium text-muted-foreground">Toolbar edge</div>
          <div className="grid grid-cols-2 gap-1">
            {DOCK_POSITIONS.map((d) => {
              const Icon = DOCK_ICONS[d.id];
              return (
                <button
                  key={d.id}
                  onClick={() => {
                    setPosition(d.id);
                    setOpen(false);
                  }}
                  className={cn(
                    "flex items-center gap-1.5 rounded-lg border px-2 py-1.5 text-[11px] transition-colors",
                    position === d.id
                      ? "border-primary bg-primary/10 text-primary"
                      : "border-border text-foreground hover:bg-muted"
                  )}
                >
                  <Icon size={13} />
                  {d.label}
                </button>
              );
            })}
          </div>
        </PopoverContent>
      </Popover>
    );
  }
}

```

```

=====
// === END FILE: src/components/whiteboard/DockControls.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/DuplicateNotebookModal.tsx ===
// File 48 of 194 | Lines: 532 | Size: 19640 bytes

```

```

=====

"use client";

/**
 * Duplicate a notebook with page range + destination.
 *
 * Replaces the one-click duplicate for notebooks.
 *
 * 1. WHICH PAGES - tick checkboxes on lazy-rendered thumbnails or type a range
 *    like "2-5, 8", kept bidirectionally in sync.
 * 2. WHERE IT GOES - drill down through folders and stop somewhere:
 *    - Stop on a folder (or root): creates a NEW notebook with chosen pages.
 *    - Asks for its name (defaulting to "X (Copy)").
 *    - Stop on a notebook: MERGES the chosen pages into that notebook.
 * 3. PUT THEM - only revealed when a notebook was chosen in step 2:
 *    pick where the pages land (at start / after page N / at end).
 */

import React, { useCallback, useEffect, useMemo, useRef, useState } from "react";
import {
  AlertTriangle,
  Check,
  Copy,
  NotebookText,
} from "lucide-react";
import { useWhiteboard } from "@store/whiteboard";
import {
  formatPageSelection,
  getPages,
  pageSizeOf,
  parsePageSelection,
} from "@lib/whiteboard/notebook";
import { buildExportSvg } from "@lib/whiteboard/export";
import type { Board, NotebookPage } from "@lib/whiteboard/types";
import {
  DestinationBrowser,
  type DestinationTarget,
} from "../MoveToDialog";
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogHeader,
  DialogTitle,
} from "@components/ui/dialog";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { cn } from "@lib/utils";

interface DuplicateNotebookModalProps {
  open: boolean;
  onClose: () => void;
  sourceBoard: Board;
}

type Placement = "start" | "after" | "end";

const PLACEMENTS: Array<{ id: Placement; label: string }> = [
  { id: "start", label: "At the beginning" },
  { id: "after", label: "After page..." },
  { id: "end", label: "At the end" },
];

/**
 * Lazy page thumbnail renderer, reused from ExportModal.
 */
function DuplicatePageThumb({
  page,
  index,
  board,
  checked,
  onToggle,
}: {
  page: NotebookPage;
  index: number;
  board: Board;
  checked: boolean;

```

```

onToggle: () => void;
}) {
  const ref = useRef<HTMLButtonElement | null>(null);
  const [svg, setSvg] = useState<string | null>(null);

  useEffect(() => {
    const node = ref.current;
    if (!node || !svg) return;
    const draw = () => {
      try {
        const single = { ...board, pages: [page], activePageId: page.id };
        setSvg(buildExportSvg(single, { pages: "current", background: "white" }).svg);
      } catch {
        setSvg("");
      }
    };
    if (typeof IntersectionObserver === "undefined") {
      draw();
      return;
    }
    const io = new IntersectionObserver(
      (entries) => {
        if (entries.some((e) => e.isIntersecting)) {
          draw();
          io.disconnect();
        }
      },
      { rootMargin: "200px" }
    );
    io.observe(node);
    return () => io.disconnect();
  }, [board, page, svg]);

  const { width, height } = pageSizeOf(board);

  return (
    <button
      ref={ref}
      type="button"
      onClick={onToggle}
      aria-pressed={checked}
      aria-label={`Page ${index + 1}${checked ? ", selected" : ""}`}
      className="group relative shrink-0 focus:outline-none"
    >
      <div
        className={cn(
          "relative w-[64px] rounded-md border-2 overflow-hidden bg-white transition-all",
          checked ? "border-primary shadow-sm" : "border-border/70 opacity-70 group-hover:opacity-100"
        )}
        style={{ aspectRatio: `${width} / ${height}` }}
      >
        {svg ? (
          <div
            className="absolute inset-0 [&svg]:w-full [&svg]:h-full"
            dangerouslySetInnerHTML={{ __html: svg }}
          />
        ) : (
          <div className="absolute inset-0 bg-muted/40" />
        )}
        {page.pdf && (
          <span className="absolute left-0.5 top-0.5 rounded bg-black/60 px-1 text-[8px] font-semibold text-white">
            PDF
          </span>
        )}
        <span
          className={cn(
            "absolute bottom-0.5 right-0.5 flex h-4 w-4 items-center justify-center rounded-full border text-[9px]",
            checked ? "border-primary bg-primary text-primary-foreground" : "border-border bg-white/90 text-transparent"
          )}
        >
          <Check size={10} strokeWidth={3} />
        </span>
      </div>
      <div className="mt-0.5 text-center text-[10px] tabular-nums text-muted-foreground">{index + 1}</div>
    </button>
  )

```

```

    });
  }

export function DuplicateNotebookModal({
  open,
  onClose,
  sourceBoard,
}: DuplicateNotebookModalProps) {
  const folders = useWhiteboard((s) => s.folders);
  const boards = useWhiteboard((s) => s.boards);
  const duplicateNotebook = useWhiteboard((s) => s.duplicateNotebook);
  const insertPagesFromBoard = useWhiteboard((s) => s.insertPagesFromBoard);

  const pages = useMemo(() => getPages(sourceBoard), [sourceBoard]);
  const pageCount = pages.length;

  // Step 1: Pages
  const [selected, setSelected] = useState<number[]>([]);
  const [rangeText, setRangeText] = useState<string>("");

  // Step 2: Destination
  const [target, setTarget] = useState<DestinationTarget>({
    kind: "folder",
    folderId: sourceBoard.folderId ?? null,
    name: "Top level",
  });
  const [newNotebookName, setNewNotebookName] = useState("");

  // Step 3: Placement (only when target is a notebook)
  const [placement, setPlacement] = useState<Placement>("end");
  const [afterPageNum, setAfterPageNum] = useState("1");

  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);

  // Initialize or reset when opened
  useEffect(() => {
    if (!open) return;
    const all = Array.from({ length: pageCount }, (_, i) => i + 1);
    setSelected(all);
    setRangeText(all.length ? `1-${all.length}` : "");
    const parentFolder = sourceBoard.folderId
      ? folders.find((f) => f.id === sourceBoard.folderId)
      : null;
    setTarget({
      kind: "folder",
      folderId: sourceBoard.folderId ?? null,
      name: parentFolder ? parentFolder.name : "Top level",
    });
    setNewNotebookName(`${sourceBoard.name} (Copy)`);
    setPlacement("end");
    setAfterPageNum("1");
    setError(null);
    setBusy(false);
  }, [open, sourceBoard, pageCount, folders]);

  const allSelected = selected.length === pageCount && pageCount > 0;

  const applySelection = useCallback((next: number[]) => {
    setSelected(next);
    setRangeText(formatPageSelection(next));
  }, []);

  const togglePage = (pageNumber: number) => {
    const next = selected.includes(pageNumber)
      ? selected.filter((n) => n !== pageNumber)
      : [...selected, pageNumber].sort((a, b) => a - b);
    applySelection(next);
  };

  const onRangeTextChange = (text: string) => {
    setRangeText(text);
    setSelected(parsePageSelection(text, pageCount));
  };

  // Find target notebook board if target is a notebook
  const targetNotebookBoard = useMemo(() => {
    if (target.kind !== "notebook") return null;

```

```

    return boards.find((b) => b.id === target.boardId) ?? null;
  }, [target, boards]);

const targetPageCount = targetNotebookBoard ? getPages(targetNotebookBoard).length : 0;

// Compute 0-based insert index for notebook merge
const insertIndex = useMemo(() => {
  if (placement === "start") return 0;
  if (placement === "end") return targetPageCount;
  const parsed = parseInt(afterPageNum, 10);
  const clampedPage = isNaN(parsed) ? 1 : Math.min(Math.max(1, parsed), targetPageCount);
  return clampedPage; // after page N means index N
}, [placement, afterPageNum, targetPageCount]);

// Size mismatch warning when merging
const sizeWarning = useMemo(() => {
  if (target.kind !== "notebook" || !targetNotebookBoard) return null;
  const a = pageSizeOf(targetNotebookBoard);
  const b = pageSizeOf(sourceBoard);
  const ratio = (s: { width: number; height: number }) => s.width / s.height;
  if (Math.abs(ratio(a) - ratio(b)) < 0.02) return null;
  return ` "${target.name}" uses a differently shaped page (${Math.round(a.width)}x${Math.round(
    a.height
  )}) against this notebook's (${Math.round(b.width)}x${Math.round(b.height)}). Inserted pages keep their cont
    ents but sit on the target's sheet.`;
}, [target, targetNotebookBoard, sourceBoard]);

// Live plain-words summary
const summary = useMemo(() => {
  const count = selected.length;
  if (count === 0) return "No pages selected.";
  const formattedRange = formatPageSelection(selected);
  const pageDesc = `${count} page${count === 1 ? "" : "s"} (${formattedRange})`;

  if (target.kind === "folder") {
    const name = newNotebookName.trim() || `${sourceBoard.name} (Copy)`;
    const folderDesc = target.folderId ? `in '${target.name}'` : "at top level";
    return `${pageDesc} → new notebook '${name}' ${folderDesc}`;
  }

  // Merge into notebook
  let posDesc = "at the end";
  if (placement === "start") posDesc = "at the start";
  else if (placement === "after") {
    const p = Math.min(Math.max(1, parseInt(afterPageNum, 10) || 1), targetPageCount);
    posDesc = `after page ${p} of ${targetPageCount}`;
  }
  return `${pageDesc} → merge into '${target.name}', ${posDesc}`;
}, [selected, target, newNotebookName, sourceBoard.name, placement, afterPageNum, targetPageCount]);

const disabledReason = useMemo(() => {
  if (selected.length === 0) return "Select at least one page";
  if (target.kind === "folder" && !newNotebookName.trim()) return "Enter a notebook name";
  return null;
}, [selected.length, target.kind, newNotebookName]);

const handleConfirm = () => {
  if (disabledReason || busy) return;
  setBusy(true);
  setError(null);
  try {
    const pageIndices = selected.map((n) => n - 1); // 0-based
    if (target.kind === "folder") {
      const id = duplicateNotebook(sourceBoard.id, {
        name: newNotebookName.trim(),
        folderId: target.folderId,
        pageIndices,
      });
      if (!id) {
        setError("Failed to create the duplicated notebook.");
        return;
      }
    } else {
      const count = insertPagesFromBoard(
        sourceBoard.id,
        { indices: pageIndices },
        insertIndex,
        target.boardId
      );
    }
  } catch {
    // Error handling
  }
};

```

```

    );
    if (count === 0) {
      setError("Nothing was merged. Check that the pages exist.");
      return;
    }
  }
  onClose();
} catch (err) {
  setError((err as Error).message);
} finally {
  setBusy(false);
}
};

return (
  <Dialog open={open} onOpenChange={(o) => !o && !busy && onClose()}>
    <DialogContent className="max-w-xl p-0 gap-0 overflow-hidden rounded-2xl border border-border/80 bg-white/98 shadow-2xl backdrop-blur-xl dark:bg-zinc-900/98">
      <DialogHeader className="space-y-0.5 border-b border-border/60 bg-muted/20 px-5 py-3.5">
        <div className="flex items-center gap-2.5">
          <div className="flex h-8 w-8 items-center justify-center rounded-lg bg-primary/10 text-primary">
            <Copy size={17} />
          </div>
          <div className="min-w-0">
            <DialogTitle className="text-base font-semibold">Duplicate notebook</DialogTitle>
            <DialogDescription className="truncate text-xs text-muted-foreground">
              Copy pages from &ldquo;{sourceBoard.name}&rdquo; ({pageCount} pages)
            </DialogDescription>
          </div>
        </div>
      </DialogHeader>

      <div className="max-h-[70vh] space-y-4 overflow-y-auto px-5 py-4">
        {/* ----- 1. Which pages ----- */}
        <section className="space-y-2">
          <div className="flex items-center justify-between">
            <h3 className="text-xs font-semibold uppercase tracking-wider text-muted-foreground">
              1. Which pages
            </h3>
            <button
              type="button"
              onClick={() =>
                applySelection(allSelected ? [] : Array.from({ length: pageCount }, (_, i) => i + 1))
              }
              className="text-xs font-medium text-primary hover:underline"
            >
              {allSelected ? "Deselect all" : "Select all"}
            </button>
          </div>

          <div className="flex gap-2 overflow-x-auto pb-1">
            {pages.map((p, i) => (
              <DuplicatePageThumb
                key={p.id}
                page={p}
                index={i}
                board={sourceBoard}
                checked={selected.includes(i + 1)}
                onToggle={() => togglePage(i + 1)}
              />
            ))}
          </div>

          <div className="space-y-1">
            <input
              value={rangeText}
              onChange={(e) => onRangeTextChange(e.target.value)}
              placeholder="e.g. 2-5, 8"
              aria-label="Page range to duplicate"
              className="w-full h-8 rounded-lg border border-border bg-background px-3 text-xs focus:border-primary focus:outline-none"
            />
            <p className="text-[11px] text-muted-foreground">
              {selected.length > 0
                ? `${selected.length} of ${pageCount} pages selected (${formatPageSelection(selected)})`
                : "No pages selected."}
            </p>
          </div>
        </section>
      </div>
    </DialogContent>
  </Dialog>
);

```

```

</section>

{/* ----- 2. Where it goes ----- */}
<section className="space-y-2">
  <h3 className="text-xs font-semibold uppercase tracking-wider text-muted-foreground">
    2. Where it goes
  </h3>
  <p className="text-[11px] text-muted-foreground">
    Navigate into a folder to create a new notebook, or tap an existing notebook to merge into it.
  </p>

  <DestinationBrowser
    selectedTarget={target}
    onSelectTarget={setTarget}
    allowNotebookTarget={true}
    excludeBoardId={sourceBoard.id}
  />

  {target.kind === "folder" ? (
    <div className="pt-1 space-y-1">
      <label className="text-xs font-medium text-foreground block">
        New notebook name
      </label>
      <Input
        value={newNotebookName}
        onChange={(e) => setNewNotebookName(e.target.value)}
        placeholder="Notebook name..."
        className="h-8 text-xs"
      />
    </div>
  ) : (
    <div className="flex items-center gap-2 rounded-lg border border-primary/40 bg-primary/5 px-3 py-2">
      <NotebookText size={16} className="text-primary shrink-0" />
      <div className="min-w-0 flex-1">
        <div className="truncate text-xs font-medium text-primary">
          Merge into &ldquo;{target.name}&rdquo;
        </div>
        <div className="text-[10px] text-muted-foreground">
          {target.pageCount} existing page{target.pageCount === 1 ? "" : "s"}
        </div>
      </div>
      <button
        type="button"
        onClick={() =>
          setTarget({
            kind: "folder",
            folderId: sourceBoard.folderId ?? null,
            name: "Top level",
          })
        }
        className="text-[11px] text-muted-foreground hover:text-foreground font-medium underline"
      >
        Change
      </button>
    </div>
  )}
</section>

{/* ----- 3. Put them (only for notebook target) ----- */}
{target.kind === "notebook" && (
  <section className="space-y-2 pt-1 border-t border-border/60">
    <h3 className="text-xs font-semibold uppercase tracking-wider text-muted-foreground">
      3. After which page
    </h3>
    <div className="grid grid-cols-3 gap-1.5">
      {PLACEMENTS.map((p) => (
        <button
          key={p.id}
          type="button"
          onClick={() => setPlacement(p.id)}
          className={cn(
            "py-1.5 px-2 rounded-lg border text-xs font-medium transition-all",
            placement === p.id
              ? "border-primary bg-primary/10 text-primary font-semibold"
              : "border-border hover:bg-muted text-foreground"
          )}
        >

```

```

        {p.label}
      </button>
    )}
  </div>

  {placement === "after" && (
    <div className="flex items-center gap-2 pt-1 text-xs">
      <span className="text-muted-foreground">Insert after page:</span>
      <input
        type="number"
        inputMode="numeric"
        min={1}
        max={Math.max(1, targetPageCount)}
        value={afterPageNum}
        onChange={(e) => setAfterPageNum(e.target.value)}
        className="w-16 h-8 rounded-lg border border-border bg-background px-2 text-center text-xs
        tabular-nums focus:border-primary focus:outline-none"
      />
      <span className="text-muted-foreground">of {targetPageCount}</span>
    </div>
  )}

  <p className="text-[11px] text-muted-foreground">
    Target has {targetPageCount} page{targetPageCount === 1 ? "" : "s"}. The copied pages will be
    come pages{" "}
    {insertIndex + 1}
    {selected.length > 1 ? `-${insertIndex + selected.length}` : ""} of{" "}
    {targetPageCount + selected.length}.
  </p>

  {sizeWarning && (
    <div className="flex gap-2 rounded-lg border border-amber-500/40 bg-amber-500/10 px-3 py-2 tex
    t-[11px] leading-snug text-amber-700 dark:text-amber-400">
      <AlertTriangle size={14} className="shrink-0 mt-0.5" />
      <span>{sizeWarning}</span>
    </div>
  )}
</section>
)}

{error && (
  <div
    role="alert"
    className="rounded-lg border border-red-500/40 bg-red-500/10 px-3 py-2 text-[11px] text-red-600
    dark:text-red-400"
  >
    {error}
  </div>
)}

{/* Live Summary Bar */}
<div className="rounded-lg bg-muted/40 p-2.5 text-xs text-foreground font-medium border border-borde
r/60">
  {summary}
</div>

<div className="flex gap-2 pt-1">
  <Button variant="outline" className="flex-1" onClick={onClose} disabled={busy}>
    Cancel
  </Button>
  <Button
    className="flex-1 font-semibold"
    disabled={Boolean(disabledReason) || busy}
    onClick={handleConfirm}
  >
    {disabledReason ??
      (busy
        ? "Working..."
        : target.kind === "folder"
        ? `Duplicate ${selected.length} page${selected.length === 1 ? "" : "s"}`
        : `Merge ${selected.length} page${selected.length === 1 ? "" : "s"}`)}
  </Button>
</div>
</DialogContent>
</Dialog>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/DuplicateNotebookModal.tsx ===
=====

=====
// === START FILE: src/components/whiteboard/ElementRenderer.tsx ===
// File 49 of 194 | Lines: 1133 | Size: 35757 bytes
=====

"use client";

/**
 * Element Renderer (v3 – Obsidian Canvas & Vector Grade)
 * =====
 * High-performance SVG renderer for all whiteboard elements:
 * - Freehand strokes (via perfect-freehand with realistic pen styles)
 * - Shapes: Rectangles, Rounded Rects, Ellipses, Diamonds, Triangles, Stars, Clouds
 * - Lines & Directional Arrows
 * - Rich Markdown Text & Sticky Notes with clickable [[WikiLinks]] and #tags
 * - Smart Directed Connections with 3 routing modes (Curved, Orthogonal, Straight),
 *   arrowheads, and interactive relationship badges.
 */

import React, { useMemo, useState, useRef } from "react";
import { useWhiteboard } from "@store/whiteboard";
import { assetIdOf, isAssetRef, loadImage } from "@lib/whiteboard/imagestore";
import { imageLayout, isUnedited } from "@lib/whiteboard/imageEdit";
import { allElements } from "@lib/whiteboard/notebook";
import {
  STICKY_COLORS,
  PEN_TYPE_PRESETS,
  getLinkTypeColor,
  getLinkTypeLabel,
  type ArrowElement,
  type ConnectionElement,
  type DiamondElement,
  type EllipseElement,
  type ImageElement,
  type LineElement,
  type RectangleElement,
  type StarElement,
  type CloudElement,
  type StickyElement,
  type StrokeElement,
  type TextElement,
  type TriangleElement,
  type WhiteboardElement,
} from "@lib/whiteboard/types";
import { buildStrokePath } from "@lib/whiteboard/stroke";
import {
  getElementBounds,
  getElementCenter,
  getOptimalConnectionAnchors,
  getConnectionPath,
  getArrowheadPath,
  getResolvedAnchorPoint,
} from "@lib/whiteboard/geometry";
import {
  parseMarkdownBlocks,
  resolveLineDir,
  toggleChecklistAtLine,
  type InlineToken,
  type BlockToken,
} from "@lib/whiteboard/textParser";

interface ElementRendererProps {
  element: WhiteboardElement;
  isSelected: boolean;
  isDraft?: boolean;
}

export const ElementRenderer = React.memo(function ElementRenderer({
  element,
  isSelected,
  isDraft,
}: ElementRendererProps) {

```

```

// Subscribe to a BOOLEAN (is THIS element being edited), not the raw id –
// otherwise every element re-renders whenever any element starts/stops editing.
const isBeingEdited = useWhiteboard((s) => s.editingId === element.id);

// If being edited inline, hide SVG representation (InlineTextEditor overlays it)
if (isBeingEdited) {
  return null;
}

const opacity = element.opacity ?? 1;
const center = getElementCenter(element);

const transform =
  element.rotation && element.rotation !== 0
    ? `translate(${element.x}, ${element.y}) rotate(${(element.rotation * 180) / Math.PI}, ${center.x - element.x}, ${center.y - element.y})`
    : `translate(${element.x}, ${element.y})`;

return (
  <g
    transform={element.type === "connection" ? undefined : transform}
    opacity={opacity}
    data-element-id={element.id}
    style={{ cursor: element.locked ? "not-allowed" : undefined }}
  >
    {renderElementContent(element, isDraft ?? false)}
  </g>
);
});

function renderElementContent(el: WhiteboardElement, isDraft: boolean): React.ReactNode {
  switch (el.type) {
    case "stroke":
      return <StrokeShape element={el} isDraft={isDraft} />;
    case "rectangle":
      return <RectangleShape element={el} />;
    case "ellipse":
      return <EllipseShape element={el} />;
    case "diamond":
      return <DiamondShape element={el} />;
    case "triangle":
      return <TriangleShape element={el} />;
    case "star":
      return <StarShape element={el} />;
    case "cloud":
      return <CloudShape element={el} />;
    case "line":
      return <LineShape element={el} />;
    case "arrow":
      return <ArrowShape element={el} />;
    case "text":
      return <TextShape element={el} />;
    case "sticky":
      return <StickyShape element={el} />;
    case "image":
      return <ImageShape element={el} />;
    case "connection":
      return <ConnectionShape element={el} />;
    default:
      return null;
  }
}

// =====
// Freehand Stroke
// =====

function StrokeShape({ element, isDraft }: { element: StrokeElement; isDraft: boolean }) {
  // Always derive the path from the points – never reuse a cached string.
  // Caching froze each stroke's geometry at the moment it was drawn, so any
  // later improvement to stroke rendering (taper, corner handling) could never
  // apply to strokes that already existed. It also cost ~143% of the points'
  // own size per stroke, duplicated into every deep-cloned history snapshot.
  // useMemo keyed on the element keeps this to one build per change.
  const path = useMemo(() => buildStrokePath(element, isDraft), [element, isDraft]);

  if (!path) return null;
  const preset = PEN_TYPE_PRESETS[element.strokeStyle] || PEN_TYPE_PRESETS.gel;

```

```

return (
  <path
    d={path}
    fill={element.color}
    stroke="none"
    // Graphite tooth. A pencil that is merely a thinner pen is the main
    // reason the old presets felt interchangeable; the grain is what makes
    // it read as pencil at any size.
    filter={preset.grain ? "url(#pencil-grain)" : undefined}
    style={{
      opacity: preset.opacity,
      mixBlendMode: preset.mixBlendMode,
      pointerEvents: "auto",
    }}
  />
);
}

/**
 * Filters used by stroke rendering. Defined once per canvas rather than per
 * stroke – an SVG filter is referenced by id, so duplicating it per element
 * would cost a node each without changing the result.
 */
export function StrokeFilterDefs() {
  return (
    <defs>
      <filter id="pencil-grain" x="-20%" y="-20%" width="140%" height="140%">
        <feTurbulence type="fractalNoise" baseFrequency="0.9" numOctaves="3" seed="7" result="noise" />
        /* Push the noise through the alpha channel so the grain eats into the
           stroke's edges instead of tinting its colour. */
        <feDisplacementMap in="SourceGraphic" in2="noise" scale="1.6" xChannelSelector="R" yChannelSelector="G"
          result="rough" />
        <feComponentTransfer in="rough">
          <feFuncA type="discrete" tableValues="0 .55 .75 .9 1" />
        </feComponentTransfer>
      </filter>
    </defs>
  );
}

// =====
// Geometric Shapes (Rectangle, Ellipse, Diamond, Triangle, Star, Cloud)
// =====

function getFillAndDash(fillStyle: string, dashStyle: string, strokeColor: string, fillColor: string) {
  let fill = "transparent";
  if (fillStyle === "solid") fill = fillColor === "transparent" ? strokeColor : fillColor;
  else if (fillStyle === "semi") fill = fillColor === "transparent" ? `${strokeColor}33` : `${fillColor}40`;

  let strokeDasharray: string | undefined = undefined;
  if (dashStyle === "dashed") strokeDasharray = "8,6";
  else if (dashStyle === "dotted") strokeDasharray = "3,4";

  return { fill, strokeDasharray };
}

function RectangleShape({ element }: { element: RectangleElement }) {
  const { fill, strokeDasharray } = getFillAndDash(
    element.fillStyle || "transparent",
    element.dashStyle || "solid",
    element.strokeColor,
    element.fillColor
  );

  return (
    <rect
      x={0}
      y={0}
      width={Math.max(element.width, 2)}
      height={Math.max(element.height, 2)}
      rx={element.cornerRadius || 6}
      ry={element.cornerRadius || 6}
      fill={fill}
      stroke={element.strokeColor}
      strokeWidth={element.strokeWidth}
      strokeDasharray={strokeDasharray}
      style={{ pointerEvents: "auto" }}
    />
  );
}

```

```
    />
  });
}

function EllipseShape({ element }: { element: EllipseElement }) {
  const { fill, strokeDasharray } = getFillAndDash(
    element.fillStyle || "transparent",
    element.dashStyle || "solid",
    element.strokeColor,
    element.fillColor
  );

  const rx = Math.max(element.width / 2, 1);
  const ry = Math.max(element.height / 2, 1);

  return (
    <ellipse
      cx={rx}
      cy={ry}
      rx={rx}
      ry={ry}
      fill={fill}
      stroke={element.strokeColor}
      strokeWidth={element.strokeWidth}
      strokeDasharray={strokeDasharray}
      style={{ pointerEvents: "auto" }}
    />
  );
}

function DiamondShape({ element }: { element: DiamondElement }) {
  const { fill, strokeDasharray } = getFillAndDash(
    element.fillStyle || "transparent",
    element.dashStyle || "solid",
    element.strokeColor,
    element.fillColor
  );

  const w = element.width;
  const h = element.height;
  const points = `${w / 2},0 ${w},${h / 2} ${w / 2},${h} 0,${h / 2}`;

  return (
    <polygon
      points={points}
      fill={fill}
      stroke={element.strokeColor}
      strokeWidth={element.strokeWidth}
      strokeDasharray={strokeDasharray}
      style={{ pointerEvents: "auto" }}
    />
  );
}

function TriangleShape({ element }: { element: TriangleElement }) {
  const { fill, strokeDasharray } = getFillAndDash(
    element.fillStyle || "transparent",
    element.dashStyle || "solid",
    element.strokeColor,
    element.fillColor
  );

  const w = element.width;
  const h = element.height;
  const points = `${w / 2},0 ${w},${h} 0,${h}`;

  return (
    <polygon
      points={points}
      fill={fill}
      stroke={element.strokeColor}
      strokeWidth={element.strokeWidth}
      strokeDasharray={strokeDasharray}
      style={{ pointerEvents: "auto" }}
    />
  );
}
```

```

function StarShape({ element }: { element: StarElement }) {
  const { fill, strokeDasharray } = getFillAndDash(
    element.fillStyle || "transparent",
    element.dashStyle || "solid",
    element.strokeColor,
    element.fillColor
  );

  const w = element.width;
  const h = element.height;
  const cx = w / 2;
  const cy = h / 2;
  const spikes = 5;
  const outerR = Math.min(w, h) / 2;
  const innerR = outerR * 0.45;

  let pts = "";
  for (let i = 0; i < spikes * 2; i++) {
    const r = i % 2 === 0 ? outerR : innerR;
    const angle = (i * Math.PI) / spikes - Math.PI / 2;
    const x = cx + Math.cos(angle) * r;
    const y = cy + Math.sin(angle) * r;
    pts += `${x.toFixed(1)},${y.toFixed(1)} `;
  }

  return (
    <polygon
      points={pts.trim()}
      fill={fill}
      stroke={element.strokeColor}
      strokeWidth={element.strokeWidth}
      strokeDasharray={strokeDasharray}
      style={{ pointerEvents: "auto" }}
    />
  );
}

function CloudShape({ element }: { element: CloudElement }) {
  const { fill, strokeDasharray } = getFillAndDash(
    element.fillStyle || "transparent",
    element.dashStyle || "solid",
    element.strokeColor,
    element.fillColor
  );

  const w = element.width;
  const h = element.height;
  const r = Math.min(w, h) * 0.2;

  const d = `
    M ${r * 1.5},${h - r}
    A ${r},${r} 0 0,1 ${r},${h * 0.5}
    A ${r * 1.2},${r * 1.2} 0 0,1 ${w * 0.35},${r}
    A ${r * 1.5},${r * 1.5} 0 0,1 ${w * 0.7},${r * 1.2}
    A ${r},${r} 0 0,1 ${w - r * 0.5},${h * 0.55}
    A ${r * 0.9},${r * 0.9} 0 0,1 ${w - r},${h - r}
    Z
  `;

  return (
    <path
      d={d}
      fill={fill}
      stroke={element.strokeColor}
      strokeWidth={element.strokeWidth}
      strokeDasharray={strokeDasharray}
      style={{ pointerEvents: "auto" }}
    />
  );
}

// =====
// Lines & Arrows
// =====

function LineShape({ element }: { element: LineElement }) {
  let strokeDasharray: string | undefined = undefined;
  if (element.dashStyle === "dashed") strokeDasharray = "8,6";

```

```

else if (element.dashStyle === "dotted") strokeDasharray = "3,4";

return (
  <line
    x1={0}
    y1={0}
    x2={element.x2}
    y2={element.y2}
    stroke={element.strokeColor}
    strokeWidth={element.strokeWidth}
    strokeDasharray={strokeDasharray}
    strokeLinecap="round"
    style={{ pointerEvents: "auto" }}
  />
);
}

function ArrowShape({ element }: { element: ArrowElement }) {
  let strokeDasharray: string | undefined = undefined;
  if (element.dashStyle === "dashed") strokeDasharray = "8,6";
  else if (element.dashStyle === "dotted") strokeDasharray = "3,4";

  const head = getArrowheadPath(
    element.x2,
    element.y2,
    0,
    0,
    element.endArrowhead || "arrow",
    element.arrowheadSize || 12
  );

  return (
    <g style={{ pointerEvents: "auto", color: element.strokeColor }}>
      <line
        x1={0}
        y1={0}
        x2={element.x2}
        y2={element.y2}
        stroke={element.strokeColor}
        strokeWidth={element.strokeWidth}
        strokeDasharray={strokeDasharray}
        strokeLinecap="round"
      />
      {head} && (
        <path
          d={head.path}
          fill={element.strokeColor}
          stroke={head.stroke}
        />
      )
    </g>
  );
}

// =====
// Text & Markdown Cards (Obsidian [[WikiLink]] & #tag support)
// =====

function renderInlineTokens(
  tokens: InlineToken[],
  handleClick: (e: React.MouseEvent, target: string) => void
): React.ReactNode {
  return tokens.map((tok, idx) => {
    switch (tok.type) {
      case "bold":
        return <strong key={idx} style={{ fontWeight: 700 }}>{tok.text}</strong>;
      case "italic":
        return <em key={idx} style={{ fontStyle: "italic" }}>{tok.text}</em>;
      case "underline":
        return <u key={idx}>{tok.text}</u>;
      case "strike":
        return <s key={idx} style={{ textDecoration: "line-through", opacity: 0.75 }}>{tok.text}</s>;
      case "code":
        return (
          <code
            key={idx}
            style={{
              backgroundColor: "rgba(128,128,128,0.15)",

```

```

        padding: "1px 4px",
        borderRadius: "4px",
        fontFamily: "monospace",
        fontSize: "0.9em",
      }}
    >
      {tok.text}
    </code>
  );
  case "highlight":
    return (
      <mark
        key={idx}
        style={{
          backgroundColor: "#fef08a",
          color: "inherit",
          padding: "0 3px",
          borderRadius: "3px",
        }}
      >
        {tok.text}
      </mark>
    );
  case "math":
    return (
      <span
        key={idx}
        style={{
          fontFamily: "'KaTeX_Main', 'Times New Roman', serif",
          fontStyle: "italic",
          padding: "0 2px",
          letterSpacing: "0.02em",
        }}
      >
        {tok.text}
      </span>
    );
  case "link":
    return (
      <span
        key={idx}
        onClick={(e) => handleLinkClick(e, tok.value!)}
        style={{
          color: "#6366f1",
          textDecoration: "underline",
          cursor: "pointer",
          fontWeight: 600,
        }}
        title={`Jump to [`${tok.value}`]}
      >
        {tok.text}
      </span>
    );
  case "tag":
    return (
      <span
        key={idx}
        style={{
          color: "#059669",
          backgroundColor: "rgba(5,150,105,0.12)",
          padding: "1px 5px",
          borderRadius: "4px",
          fontWeight: 600,
          fontSize: "0.9em",
        }}
      >
        {tok.text}
      </span>
    );
  case "url":
    return (
      <a
        key={idx}
        href={tok.value}
        target="_blank"
        rel="noopener noreferrer"
        onClick={(e) => e.stopPropagation()}
        style={{ color: "#3b82f6", textDecoration: "underline", cursor: "pointer" }}
      >

```

```

    >
      {tok.text}
    </a>
  );
  case "text":
  default:
    return <span key={idx}>{tok.text}</span>;
  }
});
}

function renderMarkdownBlocks(
  blocks: BlockToken[],
  handleClick: (e: React.MouseEvent, target: string) => void,
  onChecklistToggle?: (lineIndex: number) => void,
  lineDirs?: Array<"ltr" | "rtl" | "auto">
): React.ReactNode {
  return blocks.map((b, i) => {
    // Per-line writing direction: an explicit override wins, otherwise
    // dir="auto" lets Hebrew and English lines share one box correctly.
    const blockDir = resolveLineDir(lineDirs, b.lineIndex);
    switch (b.type) {
      case "h1":
        return (
          <div key={i} dir={blockDir} style={{ fontSize: "1.45em", fontWeight: 700, margin: "4px 0 2px 0", lineHeight: 1.25 }}>
            {renderInlineTokens(b.tokens, handleClick)}
          </div>
        );
      case "h2":
        return (
          <div key={i} dir={blockDir} style={{ fontSize: "1.25em", fontWeight: 600, margin: "3px 0 1px 0", lineHeight: 1.3 }}>
            {renderInlineTokens(b.tokens, handleClick)}
          </div>
        );
      case "h3":
        return (
          <div key={i} dir={blockDir} style={{ fontSize: "1.12em", fontWeight: 600, margin: "2px 0 1px 0", lineHeight: 1.35 }}>
            {renderInlineTokens(b.tokens, handleClick)}
          </div>
        );
      case "checklist":
        return (
          <div
            key={i}
            dir={blockDir}
            style={{
              display: "flex",
              alignItems: "flex-start",
              gap: 6,
              margin: "2px 0",
              cursor: onChecklistToggle ? "pointer" : "default",
            }}
            onClick={
              onChecklistToggle
                ? (e) => {
                    e.stopPropagation();
                    onChecklistToggle(b.lineIndex);
                  }
                : undefined
            }
          >
            <span
              style={{
                display: "inline-flex",
                alignItems: "center",
                justifyContent: "center",
                width: 16,
                height: 16,
                marginTop: 2,
                flexShrink: 0,
                borderRadius: 3,
                border: b.checked ? "1.5px solid #6366f1" : "1.5px solid currentColor",
                backgroundColor: b.checked ? "#6366f1" : "transparent",
                color: "#ffffff",
                transition: "all 0.15s ease",
              }}
            />

```

```

    }}
  >
  {b.checked && (
    <svg
      width="10"
      height="10"
      viewBox="0 0 24 24"
      fill="none"
      stroke="currentColor"
      strokeWidth="3"
      strokeLinecap="round"
      strokeLinejoin="round"
    >
      <polyline points="20 6 9 17 4 12" />
    </svg>
  )}
</span>
<div
  style={{
    textDecoration: b.checked ? "line-through" : "none",
    opacity: b.checked ? 0.6 : 1,
    flex: 1,
  }}
>
  {renderInlineTokens(b.tokens, handleLinkClick)}
</div>
</div>
);
case "bullet":
  return (
    <div key={i} dir={blockDir} style={{ display: "flex", alignItems: "flex-start", gap: 6, margin: "1px 0" }}>
      <span style={{ opacity: 0.6, userSelect: "none" }}>•</span>
      <div style={{ flex: 1 }}>{renderInlineTokens(b.tokens, handleLinkClick)}</div>
    </div>
  );
case "numbered":
  return (
    <div key={i} dir={blockDir} style={{ display: "flex", alignItems: "flex-start", gap: 6, margin: "1px 0" }}>
      <span style={{ opacity: 0.7, fontWeight: 500, minWidth: "1.2em", userSelect: "none" }}>{b.number}.</span>
      <div style={{ flex: 1 }}>{renderInlineTokens(b.tokens, handleLinkClick)}</div>
    </div>
  );
case "quote":
  return (
    <div
      key={i}
      dir={blockDir}
      style={{
        borderLeft: "3px solid #6366f1",
        backgroundColor: "rgba(99,102,241,0.06)",
        padding: "3px 8px",
        margin: "2px 0",
        borderRadius: "0 4px 4px 0",
        fontStyle: "italic",
      }}
    >
      {renderInlineTokens(b.tokens, handleLinkClick)}
    </div>
  );
case "codeblock":
  return (
    <pre
      key={i}
      dir={blockDir}
      style={{
        backgroundColor: "rgba(128,128,128,0.12)",
        padding: "6px 8px",
        borderRadius: 4,
        fontFamily: "monospace",
        fontSize: "0.88em",
        margin: "3px 0",
        overflowX: "auto",
        whiteSpace: "pre-wrap",
      }}
    >

```

```

    >
      {b.rawText}
    </pre>
  );
  case "paragraph":
  default:
    return (
      <div key={i} dir={blockDir} style={{ minHeight: "1.2em" }}>
        {b.tokens.length > 0 ? renderInlineTokens(b.tokens, handleLinkClick) : "\u00A0"}
      </div>
    );
  }
});
}

function TextShape({ element }: { element: TextElement }) {
  const setActiveBoard = useWhiteboard((s) => s.setActiveBoard);
  const findOrCreateBoardByName = useWhiteboard((s) => s.findOrCreateBoardByName);
  const setEditingId = useWhiteboard((s) => s.setEditingId);
  const updateElement = useWhiteboard((s) => s.updateElement);
  const commitHistory = useWhiteboard((s) => s.commitHistory);

  const handleLinkClick = (e: React.MouseEvent, target: string) => {
    e.stopPropagation();
    const boardId = findOrCreateBoardByName(target);
    setActiveBoard(boardId);
  };

  const handleChecklistToggle = (lineIndex: number) => {
    commitHistory();
    const newText = toggleChecklistAtLine(element.text || "", lineIndex);
    updateElement(element.id, { text: newText }, false);
  };

  const blocks = useMemo(() => parseMarkdownBlocks(element.text || ""), [element.text]);

  const isCard = !(
    element.cardMode ||
    (element.backgroundColor && element.backgroundColor !== "transparent")
  );
  const bgColor = isCard ? element.backgroundColor || "#ffffff" : "transparent";
  const borderColor = isCard ? element.borderColor || "rgba(0,0,0,0.12)" : "transparent";
  const borderRadius = isCard ? `${element.borderRadius || 8}px` : "0px";
  const padding = isCard ? `${element.padding || 12}px` : "4px 8px";
  const boxShadow = isCard ? "0 2px 10px rgba(0,0,0,0.06)" : undefined;

  return (
    <foreignObject
      x={0}
      y={0}
      width={Math.max(element.width, 20)}
      height={Math.max(element.height, 20)}
      onDoubleClick={() => setEditingId(element.id)}
    >
      <div
        style={{
          width: "100%",
          height: "100%",
          backgroundColor: bgColor,
          border: isCard ? `1.5px solid ${borderColor}` : undefined,
          borderRadius,
          boxShadow,
          color: element.color || "#1e293b",
          fontSize: `${element.fontSize || 18}px`,
          fontFamily: element.fontFamily || "Inter, system-ui, sans-serif",
          fontWeight: element.fontWeight || 400,
          textAlign: element.align || "left",
          lineHeight: element.lineHeight || 1.45,
          wordBreak: "break-word",
          padding,
          userSelect: "none",
          cursor: "default",
          boxSizing: "border-box",
        }}
      >
        {element.text ? (
          renderMarkdownBlocks(blocks, handleLinkClick, handleChecklistToggle, element.lineDirs)
        ) : (

```

```

        <span style={{ opacity: 0.4, fontStyle: "italic" }}>Type thoughts...</span>
      )}
    </div>
  </foreignObject>
);
}

// =====
// Sticky Notes
// =====

function StickyShape({ element }: { element: StickyElement }) {
  const colors = STICKY_COLORS[element.color] || STICKY_COLORS.yellow;
  const setActiveBoard = useWhiteboard((s) => s.setActiveBoard);
  const findOrCreateBoardByName = useWhiteboard((s) => s.findOrCreateBoardByName);
  const setEditingId = useWhiteboard((s) => s.setEditingId);
  const updateElement = useWhiteboard((s) => s.updateElement);
  const commitHistory = useWhiteboard((s) => s.commitHistory);

  const handleLinkClick = (e: React.MouseEvent, target: string) => {
    e.stopPropagation();
    const boardId = findOrCreateBoardByName(target);
    setActiveBoard(boardId);
  };

  const handleChecklistToggle = (lineIndex: number) => {
    commitHistory();
    const newText = toggleChecklistAtLine(element.text || "", lineIndex);
    updateElement(element.id, { text: newText }, false);
  };

  const blocks = useMemo(() => parseMarkdownBlocks(element.text || ""), [element.text]);

  return (
    <foreignObject
      x={0}
      y={0}
      width={Math.max(element.width, 40)}
      height={Math.max(element.height, 40)}
      onDoubleClick={() => setEditingId(element.id)}
    >
      <div
        style={{
          width: "100%",
          height: "100%",
          backgroundColor: colors.bg,
          border: `1.5px solid ${colors.border}`,
          borderRadius: "8px",
          color: colors.text,
          fontSize: `${element.fontSize || 16}px`,
          fontFamily: "Inter, system-ui, sans-serif",
          textAlign: element.align || "left",
          padding: "14px",
          overflow: "hidden",
          wordBreak: "break-word",
          boxShadow: "0 4px 14px rgba(0,0,0,0.08)",
          lineHeight: 1.45,
          userSelect: "none",
          boxSizing: "border-box",
        }}
      >
        {element.text ? (
          renderMarkdownBlocks(blocks, handleLinkClick, handleChecklistToggle, element.lineDirs)
        ) : (
          <span style={{ opacity: 0.45, fontStyle: "italic" }}>Write note...</span>
        )}
      </div>
    </foreignObject>
  );
}

// =====
// Image
// =====

/**
 * Resolves an image src for rendering.
 */

```

```

* `asset:<id>` means the pixels live in IndexedDB (see lib/whiteboard/
* imagestore.ts) rather than in the workspace, because a single image was
* measured at 908 KB against a ~5 MB localStorage budget. An ordinary data or
* http URL is returned unchanged, so every image saved before that change
* still renders.
*
* Resolved URLs are cached across components: the same photo used on ten pages
* must not mean ten IndexedDB reads per frame.
*/
const assetUrlCache = new Map<string, string>();

export function useResolvedImageSrc(src: string): string {
  const isRef = isAssetRef(src);
  const [resolved, setResolved] = React.useState<string>(() =>
    isRef ? assetUrlCache.get(src) ?? "" : src
  );

  React.useEffect(() => {
    if (!isRef) {
      setResolved(src);
      return;
    }
    const cached = assetUrlCache.get(src);
    if (cached) {
      setResolved(cached);
      return;
    }
    let alive = true;
    loadImage(assetIdOf(src)).then((url) => {
      if (!alive || !url) return;
      assetUrlCache.set(src, url);
      setResolved(url);
    });
    return () => {
      alive = false;
    };
  }, [src, isRef]);

  return resolved;
}

function ImageShape({ element }: { element: ImageElement }) {
  const href = useResolvedImageSrc(element.src);

  // An image whose pixels are still loading, or whose payload was dropped by a
  // full-storage save, gets a placeholder rather than an invisible gap – the
  // element is still there and still selectable.
  if (!href) {
    return (
      <rect
        x={0}
        y={0}
        width={element.width}
        height={element.height}
        rx={6}
        className="fill-zinc-200 stroke-zinc-300 dark:fill-zinc-800 dark:stroke-zinc-700"
        strokeWidth={1}
        style={{ pointerEvents: "auto" }}
      />
    );
  }

  // An untouched picture is drawn exactly as before edits existed.
  if (isUnedited(element)) {
    return (
      <image
        href={href}
        x={0}
        y={0}
        width={element.width}
        height={element.height}
        preserveAspectRatio="xMidYMid meet"
        style={{
          borderRadius: "6px",
          pointerEvents: "auto",
        }}
      />
    );
  }
}

```

```

}

// Crop = the inner svg's viewBox; flip = a transform on the picture;
// frame = a clip. The pixels themselves are never touched (imageEdit.ts).
const L = imageLayout(element);
const w = element.width;
const h = element.height;
const clipId = `imgclip-${element.id}`;
const bw = element.borderWidth && element.borderWidth > 0 ? element.borderWidth : 0;
const shape = (extra: React.SVGProps<SVGElement>) =>
  L.frame === "circle" ? (
    <ellipse cx={w / 2} cy={h / 2} rx={w / 2} ry={h / 2} {...(extra as React.SVGProps<SVGEllipseElement>)} /
    >
  ) : (
    <rect x={0} y={0} width={w} height={h} rx={L.radius} {...(extra as React.SVGProps<SVGRectElement>)} />
  );
return (
  <g style={{ pointerEvents: "auto" }}>
    <clipPath id={clipId}>{shape({})}</clipPath>
    <g clipPath={`url(#${clipId})`}>
      <svg
        x={0}
        y={0}
        width={w}
        height={h}
        viewBox={` ${L.viewBox.x} ${L.viewBox.y} ${L.viewBox.w} ${L.viewBox.h}` }
        preserveAspectRatio="xMidYMid meet"
      >
        <image
          href={href}
          x={0}
          y={0}
          width={L.nw}
          height={L.nh}
          preserveAspectRatio="none"
          transform={L.flip || undefined}
          style={L.filter ? { filter: L.filter } : undefined}
        />
      </svg>
    </g>
    {bw > 0 && shape({ fill: "none", stroke: element.borderColor || "#18181b", strokeWidth: bw })}
  </g>
);
}

// =====
// Smart Directed Connections (Obsidian Canvas Grade)
// =====

function ConnectionShape({ element }: { element: ConnectionElement }) {
  const boards = useWhiteboard((s) => s.boards);
  const customLinkTypes = useWhiteboard((s) => s.customLinkTypes);
  const updateElement = useWhiteboard((s) => s.updateElement);
  const commitHistory = useWhiteboard((s) => s.commitHistory);

  const [isEditingLabel, setIsEditingLabel] = useState(false);
  const [draftLabel, setDraftLabel] = useState(element.label || "");

  // Find source & target elements.
  //
  // A connection can point at an element on any board, and that board may be a
  // notebook – whose elements live on its pages, not on `elements`. Searching
  // the canvas field alone made every connection into a notebook resolve to
  // nothing, so the whole connection silently vanished.
  const findAcrossBoard = (boardId: string, elementId: string) => {
    const b = boards.find((x) => x.id === boardId);
    return b ? allElements(b).find((e) => e.id === elementId) : undefined;
  };
  const sourceEl = findAcrossBoard(element.source.boardId, element.source.elementId);
  const targetEl = findAcrossBoard(element.target.boardId, element.target.elementId);

  if (!sourceEl || !targetEl) return null;

  // Resolve anchor points
  const { source, target } = getOptimalConnectionAnchors(
    sourceEl,
    targetEl,
    element.source.anchor,

```

```

    element.target.anchor
  );

const pathD = getConnectionPath(source, target, element.routing || "curved");
const color = getLinkTypeColor(element.linkType, customLinkTypes);
const defaultLabel = getLinkTypeLabel(element.linkType, customLinkTypes);
const displayLabel = element.label !== undefined ? element.label : defaultLabel;

// Arrowheads
const endHead = getArrowheadPath(
  target.x,
  target.y,
  target.x + target.nx * 20,
  target.y + target.ny * 20,
  element.endArrowhead || "arrow",
  12
);

const startHead = element.startArrowhead && element.startArrowhead !== "none"
  ? getArrowheadPath(
    source.x,
    source.y,
    source.x + source.nx * 20,
    source.y + source.ny * 20,
    element.startArrowhead,
    12
  )
  : null;

// Calculate midpoint for the badge
const midX = (source.x + target.x) / 2;
const midY = (source.y + target.y) / 2;

let strokeDasharray: string | undefined = undefined;
if (element.dashStyle === "dashed") strokeDasharray = "6,4";
else if (element.dashStyle === "dotted") strokeDasharray = "2,4";

return (
  <g style={{ pointerEvents: "auto", color }}>
    {/* Invisible thicker hit-path */}
    <path
      d={pathD}
      fill="none"
      stroke="transparent"
      strokeWidth={16}
      style={{ cursor: "pointer" }}
    />

    {/* Main Connection Line */}
    <path
      d={pathD}
      fill="none"
      stroke={color}
      strokeWidth={element.strokeWidth || 2.5}
      strokeDasharray={strokeDasharray}
      strokeLinecap="round"
    />

    {/* Start Arrowhead */}
    {startHead && (
      <path d={startHead.path} fill={color} stroke="none" />
    )}

    {/* End Arrowhead */}
    {endHead && (
      <path d={endHead.path} fill={color} stroke="none" />
    )}

    {/* Interactive Semantic Relationship Badge */}
    {displayLabel && (
      <g
        transform={`translate(${midX}, ${midY})`}
        style={{ cursor: "pointer" }}
        onClick={(e) => {
          e.stopPropagation();
          setIsEditingLabel(true);
          setDraftLabel(displayLabel);
        }}
      >

```

```

>
{isEditingLabel ? (
  <foreignObject x={-60} y={-14} width={120} height={28}>
    <input
      type="text"
      autoFocus
      value={draftLabel}
      onChange={(e) => setDraftLabel(e.target.value)}
      onBlur={() => {
        commitHistory();
        updateElement(element.id, { label: draftLabel.trim() }, false);
        setIsEditingLabel(false);
      }}
      onKeyDown={(e) => {
        if (e.key === "Enter") {
          commitHistory();
          updateElement(element.id, { label: draftLabel.trim() }, false);
          setIsEditingLabel(false);
        }
        if (e.key === "Escape") {
          setIsEditingLabel(false);
        }
      }}
      className="w-full h-full px-2 text-[11px] font-semibold text-center rounded-full border border-
        -primary bg-background shadow-lg outline-none"
    />
  </foreignObject>
) : (
  <>
    {/* Badge Background */}
    <rect
      x={-(displayLabel.length * 3.8 + 12)}
      y={-11}
      width={displayLabel.length * 7.6 + 24}
      height={22}
      rx={11}
      fill="white"
      className="dark:fill-zinc-900"
      stroke={color}
      strokeWidth={1.5}
      style={{ filter: "drop-shadow(0 2px 4px rgba(0,0,0,0.1))" }}
    />
    {/* Badge Text */}
    <text
      x={0}
      y={3.5}
      fill={color}
      textAnchor="middle"
      fontSize={11}
      fontWeight={600}
      fontFamily="Inter, system-ui, sans-serif"
    >
      {displayLabel}
    </text>
  </>
)>
</g>
</g>
);
}

```

```
// === END FILE: src/components/whiteboard/ElementRenderer.tsx ===
```

```
// === START FILE: src/components/whiteboard/ExportModal.tsx ===
// File 50 of 194 | Lines: 630 | Size: 24413 bytes
```

```
"use client";

/**
 * Export / share.
 *
 * Rebuilt around the question people actually arrive with: "send THESE pages,
```

```

* as THIS kind of file, to THAT app." The three answers are on one screen,
* in that order, the way GoodNotes, Notability and Notewise all arrange it:
*
* 1. WHICH PAGES - tick them on thumbnails, or type "2-5, 8". The two stay
*    in sync, because picking 30 of 69 pages by tapping is miserable and
*    typing a range you cannot see is guesswork.
* 2. WHAT FILE - PDF, PNG, JPEG, WEBP or SVG, each with the one line that
*    says when to choose it.
* 3. WHERE - "Share" opens the system app chooser; "Save" writes the
*    file. On Android saving IS the share sheet, which offers Save to Files.
*
* Two things this must never do again, both reported from the tablet:
* - go quiet. Rasterising 40 sheets takes real time, and a button that just
*   sits there reads as a crash. Every sheet reports into a progress bar.
* - claim success it did not have. A cancelled share, a partial export and a
*   failure are three different messages, and none of them close the dialog
*   pretending the file went out.
*
* Thumbnails render lazily. A 69-page notebook must not build 69 previews to
* show the first four.
*/

import React, { useCallback, useEffect, useMemo, useRef, useState } from "react";
import {
  Check,
  Copy,
  Download,
  FileCode,
  FileText,
  Image as ImageIcon,
  Loader2,
  Share2,
  X,
} from "lucide-react";
import { useWhiteboard, useActiveBoard } from "@store/whiteboard";
import {
  allElements,
  formatPageSelection,
  getActivePageIndex,
  getPages,
  isNotebook,
  pageSizeOf,
  parsePageSelection,
} from "@lib/whiteboard/notebook";
import {
  buildExportSvg,
  canChooseApp,
  copyPngToClipboard,
  exportBoard,
  isNativeApp,
  resolveExportImages,
  resolvePageBackdrops,
  type ExportFormat,
  type ExportOptions,
  type ExportProgress,
  type ExportReport,
} from "@lib/whiteboard/export";
import type { NotebookPage } from "@lib/whiteboard/types";
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogHeader,
  DialogTitle,
} from "@components/ui/dialog";
import { Button } from "@components/ui/button";
import { cn } from "@lib/utils";

interface ExportModalProps {
  open: boolean;
  onClose: () => void;
}

/**
 * Every format, with the line that tells you when to pick it. "All the
 * formats" only helps if you can tell them apart: PNG and JPEG both make an
 * image, and the difference - one keeps transparency, one makes a file a mail
 * client will accept - is the whole basis for choosing.

```

```

*/
const FORMATS: Array<{ id: ExportFormat; label: string; hint: string; icon: typeof FileText }> = [
  { id: "pdf", label: "PDF", hint: "One page per sheet", icon: FileText },
  { id: "png", label: "PNG", hint: "Sharp, transparency", icon: ImageIcon },
  { id: "jpeg", label: "JPEG", hint: "Small file", icon: ImageIcon },
  { id: "webp", label: "WEBP", hint: "Smallest", icon: ImageIcon },
  { id: "svg", label: "SVG", hint: "Vector", icon: FileCode },
];

/** A sentence describing what an export had to leave out, or null. */
function describeReport(r: ExportReport): string | null {
  const parts: string[] = [];
  if (r.missingImages > 0) {
    parts.push(
      `${r.missingImages} picture${r.missingImages === 1 ? " was" : "s were"} missing from this device's storage and ${r.missingImages === 1 ? "is" : "are"} shown as a grey box`
    );
  }
  if (r.missingPdfPages > 0) {
    parts.push(
      `${r.missingPdfPages} PDF page${r.missingPdfPages === 1 ? "" : "s"} could not be loaded and ${r.missingPdfPages === 1 ? "was" : "were"} exported as plain paper`
    );
  }
  return parts.length ? `Exported, but ${parts.join("; ")}` : null;
}

/**
 * One page thumbnail, drawn only once it scrolls into view.
 *
 * PDF backdrops are deliberately skipped here: rasterising a PDF page per thumbnail would make opening this dialog on a 69-page import cost more than the export itself. A badge says the page has one.
 */
function PageThumb({
  page,
  index,
  board,
  checked,
  onToggle,
}: {
  page: NotebookPage;
  index: number;
  board: NonNullable<ReturnType<typeof useActiveBoard>>;
  checked: boolean;
  onToggle: () => void;
}) {
  const ref = useRef<HTMLButtonElement | null>(null);
  const [svg, setSvg] = useState<string | null>(null);

  useEffect(() => {
    const node = ref.current;
    if (!node || !svg) return;
    const draw = () => {
      try {
        const single = { ...board, pages: [page], activePageId: page.id };
        setSvg(buildExportSvg(single, { pages: "current", background: "white" }).svg);
      } catch {
        setSvg(""); // a thumbnail is never worth breaking the dialog over
      }
    };
    if (typeof IntersectionObserver === "undefined") {
      draw();
      return;
    }
    const io = new IntersectionObserver(
      (entries) => {
        if (entries.some((e) => e.isIntersecting)) {
          draw();
          io.disconnect();
        }
      },
      { rootMargin: "200px" }
    );
    io.observe(node);
  }, [page, board, index]);
}

```

```

    return () => io.disconnect();
  }, [board, page, svg]);

const { width, height } = pageSizeOf(board);

return (
  <button
    ref={ref}
    type="button"
    onClick={onToggle}
    aria-pressed={checked}
    aria-label={`Page ${index + 1}${checked ? ", selected" : ""}`}
    className="group relative shrink-0 focus:outline-none"
  >
    <div
      className={cn(
        "relative w-[68px] rounded-md border-2 overflow-hidden bg-white transition-all",
        checked ? "border-primary shadow-sm" : "border-border/70 opacity-70 group-hover:opacity-100"
      )}
      style={{ aspectRatio: `${width} / ${height}` }}
    >
      {svg ? (
        <div
          className="absolute inset-0 [&svg]:w-full [&svg]:h-full"
          dangerouslySetInnerHTML={{ __html: svg }}
        />
      ) : (
        <div className="absolute inset-0 bg-muted/40" />
      )}
      {page.pdf && (
        <span className="absolute left-0.5 top-0.5 rounded bg-black/60 px-1 text-[8px] font-semibold text-white">
          PDF
        </span>
      )}
      <span
        className={cn(
          "absolute bottom-0.5 right-0.5 flex h-4 w-4 items-center justify-center rounded-full border text-[9px]",
          checked ? "border-primary bg-primary text-primary-foreground" : "border-border bg-white/90 text-transparent"
        )}
      >
        <Check size={10} strokeWidth={3} />
      </span>
    </div>
    <div className="mt-0.5 text-center text-[10px] tabular-nums text-muted-foreground">{index + 1}</div>
  </button>
);
}

export function ExportModal({ open, onClose }: ExportModalProps) {
  const activeBoard = useActiveBoard();
  const customLinkTypes = useWhiteboard((s) => s.customLinkTypes);

  const notebook = isNotebook(activeBoard);
  const pages = useMemo(() => (activeBoard ? getPages(activeBoard) : []), [activeBoard]);
  const pageCount = pages.length;

  const [format, setFormat] = useState<ExportFormat>("pdf");
  const [background, setBackground] = useState<"white" | "dark" | "transparent">("white");
  const [scale, setScale] = useState<number>(2);
  const [includeLocked, setIncludeLocked] = useState<boolean>(true);
  const [selected, setSelected] = useState<number[]>([]);
  const [rangeText, setRangeText] = useState<string>("");
  const [busy, setBusy] = useState<false | "save" | "share">(false);
  const [progress, setProgress] = useState<ExportProgress | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [notice, setNotice] = useState<string | null>(null);
  const [copied, setCopied] = useState(false);
  const [native, setNative] = useState(false);
  const [canShare, setCanShare] = useState(false);
  const copiedTimer = useRef<ReturnType<typeof setTimeout> | null>(null);
  const [preview, setPreview] = useState<string>("");

  useEffect(() => () => { if (copiedTimer.current) clearTimeout(copiedTimer.current); }, []);

  // Opening resets to sensible defaults for what is on screen: a notebook is

```

```

// almost always wanted whole, as a PDF; a whiteboard as an image. Probed on
// open rather than at mount, because this dialog is mounted with the app.
useEffect(() => {
  if (!open || !activeBoard) return;
  const all = Array.from({ length: pageCount }, (_, i) => i + 1);
  setFormat(notebook ? "pdf" : "png");
  setSelected(all);
  setRangeText(all.length ? `1-${all.length}` : "");
  setError(null);
  setNotice(null);
  setProgress(null);
  setBusy(false);
  setNative(isNativeApp());
  setCanShare(canChooseApp());
}, [open, activeBoard?.id, pageCount, notebook]); // eslint-disable-line react-hooks/exhaustive-deps

/** Ticking a thumbnail rewrites the range box, and vice versa. */
const applySelection = useCallback((next: number[]) => {
  setSelected(next);
  setRangeText(formatPageSelection(next));
}, []);

const toggle = (pageNumber: number) => {
  const next = selected.includes(pageNumber)
    ? selected.filter((n) => n !== pageNumber)
    : [...selected, pageNumber].sort((a, b) => a - b);
  applySelection(next);
};

const onRangeChange = (text: string) => {
  setRangeText(text);
  setSelected(parsePageSelection(text, pageCount));
};

const allSelected = notebook && selected.length === pageCount && pageCount > 0;

const exportOptions: ExportOptions = useMemo(
  () => ({
    background,
    scale,
    includeLocked,
    padding: 40,
    pages: notebook ? selected : "all",
  }),
  [background, scale, includeLocked, notebook, selected]
);

const chosenPages = useMemo(
  () => (notebook ? selected.map((n) => pages[n - 1]).filter(Boolean) : []),
  [notebook, selected, pages]
);

const elementCount = !activeBoard
  ? 0
  : notebook
  ? chosenPages.reduce((n, p) => n + (p.elements?.length ?? 0), 0)
  : allElements(activeBoard).length;

// Live preview of exactly what will be written, capped so a big selection
// does not cost more to preview than to export.
useEffect(() => {
  if (!open || !activeBoard) return;
  let cancelled = false;
  (async () => {
    try {
      const images = await resolveExportImages(activeBoard);
      if (cancelled) return;
      let board = activeBoard;
      let backdrops: Map<string, string> | undefined;
      if (notebook) {
        const shown = chosenPages.slice(0, 4);
        if (shown.length === 0) { setPreview(""); return; }
        board = { ...activeBoard, pages: shown, activePageId: shown[0].id };
        backdrops = await resolvePageBackdrops(shown, pageSizeOf(activeBoard).width);
        if (cancelled) return;
      }
      setPreview(buildExportSvg(board, { ...exportOptions, pages: "all" }, customLinkTypes, images, backdrop
        s).svg);
    } catch {
  }
}

```

```

        if (!cancelled) setPreview("");
    }
    })();
    return () => { cancelled = true; };
}, [open, activeBoard, notebook, chosenPages, exportOptions, customLinkTypes]);

if (!activeBoard) return null;

const nothingChosen = notebook && selected.length === 0;

const run = async (destination: "save" | "share") => {
    if (busy || nothingChosen) return;
    setBusy(destination);
    setError(null);
    setNotice(null);
    setProgress({ done: 0, total: notebook ? Math.max(1, selected.length) : 1, label: "Preparing" });
    try {
        const { report, outcome } = await exportBoard(
            activeBoard,
            format,
            { ...exportOptions, onProgress: (p) => setProgress(p) },
            customLinkTypes,
            destination
        );
        const warning = describeReport(report);
        if (warning) { setNotice(warning); return; }
        if (outcome === "cancelled") {
            setNotice("Sharing was cancelled – the file was not sent anywhere.");
            return;
        }
        if (destination === "share" && outcome === "downloaded") {
            setNotice("No app chooser was available, so the file was saved instead.");
            return;
        }
        onClose();
    } catch (err) {
        console.error("Export failed:", err);
        // Stay open on failure. Closing looks exactly like success, which is how
        // a broken export went unnoticed for so long.
        setError(`Export failed: ${err as Error}.message`);
    } finally {
        setBusy(false);
        setProgress(null);
    }
};

const copy = async () => {
    if (busy) return;
    setBusy("save");
    setError(null);
    try {
        const ok = await copyPngToClipboard(activeBoard, exportOptions, customLinkTypes);
        if (ok) {
            setCopied(true);
            if (copiedTimer.current) clearTimeout(copiedTimer.current);
            copiedTimer.current = setTimeout(() => setCopied(false), 2000);
        } else {
            setError(
                native
                ? "This device can't copy images to the clipboard. Use Share instead."
                : "Your browser blocked clipboard access. Use Save instead."
            );
        }
    } catch (err) {
        setError(`Copy failed: ${err as Error}.message`);
    } finally {
        setBusy(false);
    }
};

const pct = progress && progress.total > 0 ? Math.round((progress.done / progress.total) * 100) : 0;

return (
    <Dialog open={open} onOpenChange={o => !o && !busy && onClose()}>
        <DialogContent className="max-w-lg p-0 gap-0 overflow-hidden rounded-2xl border border-border/80 bg-white/98 shadow-2xl backdrop-blur-xl dark:bg-zinc-900/98">
            <DialogHeader className="space-y-0.5 border-b border-border/60 bg-muted/20 px-5 py-3.5">
                <div className="flex items-center gap-2.5">

```

```

<div className="flex h-8 w-8 items-center justify-center rounded-lg bg-primary/10 text-primary">
  <Share2 size={17} />
</div>
<div className="min-w-0">
  <DialogTitle className="text-base font-semibold">Export & share</DialogTitle>
  <DialogDescription className="truncate text-xs text-muted-foreground">
    {activeBoard.name}
    {notebook ? ` · ${selected.length} of ${pageCount} pages` : ""} · {elementCount} items
  </DialogDescription>
</div>
</div>
</DialogHeader>

<div className="max-h-[60vh] space-y-4 overflow-y-auto px-5 py-4">
  {/* ----- 1. Which pages ----- */}
  {notebook && (
    <section className="space-y-2">
      <div className="flex items-center justify-between">
        <h3 className="text-xs font-semibold uppercase tracking-wider text-muted-foreground">
          Select pages
        </h3>
        <button
          type="button"
          onClick={() =>
            applySelection(allSelected ? [] : Array.from({ length: pageCount }, (_, i) => i + 1))
          }
          className="text-xs font-medium text-primary hover:underline"
        >
          {allSelected ? "Deselect all" : "Select all"}
        </button>
      </div>
    </section>

    <div className="flex gap-2 overflow-x-auto pb-1">
      {pages.map((p, i) => (
        <PageThumb
          key={p.id}
          page={p}
          index={i}
          board={activeBoard}
          checked={selected.includes(i + 1)}
          onToggle={() => toggle(i + 1)}
        />
      ))}
    </div>

    <input
      value={rangeText}
      onChange={(e) => onRangeChange(e.target.value)}
      placeholder="Enter page range to export, e.g. 2-5, 8"
      aria-label="Page range to export"
      className="h-9 w-full rounded-lg border border-border bg-background px-3 text-sm focus:border-primary focus:outline-none"
    />
    {nothingChosen && (
      <p className="text-[11px] text-amber-600 dark:text-amber-400">
        No pages selected – tick a page or type a range.
      </p>
    )}
  </section>
)}

  {/* ----- 2. What file ----- */}
  <section className="space-y-2">
    <h3 className="text-xs font-semibold uppercase tracking-wider text-muted-foreground">Export as</h3>
    >
    <div className="grid grid-cols-5 gap-1.5">
      {FORMATS.map((f) => {
        const Icon = f.icon;
        const on = format === f.id;
        return (
          <button
            key={f.id}
            type="button"
            onClick={() => setFormat(f.id)}
            title={f.hint}
            className={cn(
              "flex flex-col items-center gap-1 rounded-xl border px-1 py-2 transition-all",
              on
            )}
          >
            {Icon}
            {f.name}
          </button>
        )
      })}
    </div>
  </section>

```

```

        ? "border-primary bg-primary/10 text-primary"
        : "border-border text-muted-foreground hover:bg-muted hover:text-foreground"
    )}
  >
  <Icon size={17} />
  <span className="text-[11px] font-semibold">{f.label}</span>
  <span className="text-[8px] leading-tight opacity-70">{f.hint}</span>
</button>
);
}}
</div>
{notebook && format !== "pdf" && selected.length > 1 && (
  <p className="text-[11px] leading-snug text-muted-foreground">
    {format.toUpperCase()} is one image, so the pages come out stacked top to bottom.
    Choose PDF for one page per sheet.
  </p>
)}
{format === "jpeg" && background === "transparent" && (
  <p className="text-[11px] leading-snug text-muted-foreground">
    JPEG has no transparency, so this export will use a white background.
  </p>
)}
</section>

{ /* ----- Options ----- */ }
<section className="grid grid-cols-2 gap-3">
  <div className="space-y-1.5">
    <h3 className="text-xs font-semibold uppercase tracking-wider text-muted-foreground">Background</h3>
    <div className="grid grid-cols-3 gap-1">
      {[
        { id: "white", label: "White", bg: "#ffffff" },
        { id: "dark", label: "Dark", bg: "#18181b" },
        { id: "transparent", label: "None", bg: "transparent" },
      ] as const}.map((b) => (
        <button
          key={b.id}
          type="button"
          onClick={() => setBackground(b.id)}
          className={cn(
            "flex items-center justify-center gap-1 rounded-lg border py-1 text-[11px] font-medium transition-all",
            background === b.id ? "border-primary bg-primary/10 text-primary" : "border-border hover:bg-muted"
          )}
        >
          <span className="h-2.5 w-2.5 shrink-0 rounded-full border border-border" style={{ background: b.bg }} />
          {b.label}
        </button>
      ))}
    </div>
  </div>
  <div>
    {format !== "svg" && (
      <div className="space-y-1.5">
        <h3 className="text-xs font-semibold uppercase tracking-wider text-muted-foreground">Quality</h3>
        <div className="grid grid-cols-3 gap-1">
          {[1, 2, 3].map((s) => (
            <button
              key={s}
              type="button"
              onClick={() => setScale(s)}
              className={cn(
                "rounded-lg border py-1 text-[11px] font-medium transition-all",
                scale === s ? "border-primary bg-primary text-primary-foreground" : "border-border hover:bg-muted"
              )}
            >
              {s}x
            </button>
          ))}
        </div>
      </div>
    )}
  </div>
</section>

```

```

<label className="flex cursor-pointer select-none items-center gap-2 text-xs text-foreground">
  <input
    type="checkbox"
    checked={includeLocked}
    onChange={(e) => setIncludeLocked(e.target.checked)}
    className="h-4 w-4 rounded border-border text-primary focus:ring-primary"
  />
  Include locked items
</label>

{/* ----- Preview ----- */}
<section className="space-y-1.5">
  <h3 className="text-xs font-semibold uppercase tracking-wider text-muted-foreground">Preview</h3>
  <div
    className={cn(
      "flex h-36 items-center justify-center overflow-hidden rounded-xl border border-border/80 p-2",
      background === "dark" ? "bg-zinc-950" : "bg-white"
    )}
  >
    {preview ? (
      <div
        className="flex h-full w-full items-center justify-center [>svg]:max-h-full [>svg]:max-w-f
        ull [>svg]:w-auto [>svg]:h-auto"
        dangerouslySetInnerHTML={{ __html: preview }}
      />
    ) : (
      <span className="text-xs text-muted-foreground">
        {nothingChosen ? "Nothing selected" : "Nothing to preview"}
      </span>
    )}
  </div>
</section>
</div>

{/* ----- 3. Where ----- */}
<div className="space-y-2 border-t border-border/60 bg-muted/10 px-5 py-3.5">
  {progress && (
    <div className="space-y-1" role="status" aria-live="polite">
      <div className="flex items-center justify-between text-[11px] text-muted-foreground">
        <span className="flex items-center gap-1.5">
          <Loader2 size={12} className="animate-spin" />
          {progress.label}
        </span>
        <span className="tabular-nums">{pct}%</span>
      </div>
      <div className="h-1.5 w-full overflow-hidden rounded-full bg-border/60">
        <div className="h-full rounded-full bg-primary transition-[width] duration-200" style={{ width
          : `${pct}%` }} />
      </div>
    </div>
  )}

  {notice && (
    <div role="status" className="rounded-lg border border-amber-500/40 bg-amber-500/10 px-3 py-2 text
      -[11px] leading-snug text-amber-700 dark:text-amber-400">
      {notice}
    </div>
  )}

  {error && (
    <div role="alert" className="rounded-lg border border-red-500/40 bg-red-500/10 px-3 py-2 text-[11p
      x] leading-snug text-red-600 dark:text-red-400">
      {error}
    </div>
  )}

  <div className="flex gap-2">
    <Button variant="ghost" className="px-3" onClick={onClose} disabled={!busy}>
      <X size={15} />
    </Button>
    {format === "png" && !native && (
      <Button variant="outline" className="gap-1.5 px-3" onClick={copy} disabled={!busy || nothingCho
        sen}>
        {copied ? <Check size={14} className="text-emerald-500" /> : <Copy size={14} />}
        <span className="text-xs">{copied ? "Copied" : "Copy"}</span>
      </Button>
    )}
  </div>

```

```

        variant={canShare ? "outline" : "default"}
        className="flex-1 gap-1.5"
        onClick={() => run("save")}
        disabled={!busy || nothingChosen}
      >
      <Download size={15} />
      <span>{busy === "save" ? "Working..." : native ? "Save" : "Download"}</span>
    </Button>
    {canShare && (
      <Button className="flex-1 gap-1.5 font-semibold" onClick={() => run("share")} disabled={!busy |
        | nothingChosen}>
        <Share2 size={15} />
        <span>{busy === "share" ? "Working..." : "Share"}</span>
      </Button>
    )}
  </div>
</div>
</DialogContent>
</Dialog>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/ExportModal.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/FavoritePensBar.tsx ===
// File 51 of 194 | Lines: 772 | Size: 30539 bytes
=====

```

```

"use client";

/**
 * Favorite Pens Bar
 * =====
 * A floating, movable bar of pinned pens. Each pin is a full (pen type, color,
 * size) triple, so the SAME pen type can be pinned several times in different
 * colors – that is the point of the bar.
 *
 * Interactions:
 * - Click a pin → that pen becomes the active tool, with its color & size.
 * - Double-click / double-tap a pin → open its editor (size, color, type, move, delete).
 * - Drag a pin → reorder within the bar.
 * - Drag the grip handle → move the whole bar anywhere on screen (position persists).
 * - "+" → add a pin (starts from the pen you are currently using).
 * - Right-click (or long-press on touch) a pin → edit its type / color / size, or delete it.
 *
 * Everything here is pointer-event based (not HTML5 drag-and-drop) so it works
 * with a finger and a stylus, not just a mouse.
 */

```

```

import React, { useCallback, useEffect, useLayoutEffect, useRef, useState } from "react";
import { useWhiteboard } from "@store/whiteboard";
import {
  PEN_COLORS,
  PEN_TYPE_PRESETS,
  getToolColor,
  getToolSize,
  type PinnedPen,
  type ToolType,
  type StrokeStyle,
} from "@lib/whiteboard/types";
import {
  Brush,
  ChevronDown,
  ChevronUp,
  Feather,
  GripVertical,
  Highlighter,
  Paintbrush,
  Pen,
  PenLine,
  Pencil,
  Plus,
  RotateCw,
  Star,

```

```

    Trash2,
    X,
  } from "lucide-react";
import {
  defaultPensBarPosition,
  PENS_BAR_HEIGHT_ESTIMATE,
} from "@lib/whiteboard/placement";
import { Popover, PopoverContent, PopoverTrigger } from "@components/ui/popover";
import { Button } from "@components/ui/button";
import { Slider } from "@components/ui/slider";
import { StepButton } from "../StepperControl";
import { cn } from "@lib/utils";

const PEN_ICONS: Record<StrokeStyle, React.ComponentType<{ size?: number; className?: string }>> = {
  ballpoint: Pen,
  gel: Pen,
  fineliner: PenLine,
  fountain: Feather,
  brush: Brush,
  marker: Paintbrush,
  highlighter: Highlighter,
  pencil: Pencil,
  // legacy ids
  pen: Pen,
  calligraphy: Feather,
};

/** Thickness across the bar: its width when vertical, its height when horizontal. */
const BAR_THICKNESS = 56;
const PIN_SIZE = 40;
/** How far a pointer must travel before a press turns into a drag, not a click. */
const DRAG_THRESHOLD_PX = 5;

/**
 * Capture the pointer so a gesture that leaves the element still delivers
 * pointermove/up. Never let a refused capture abort the gesture – the drag
 * state is already set up by the time we get here.
 */
function capturePointer(e: React.PointerEvent) {
  try {
    (e.currentTarget as HTMLElement).setPointerCapture?.(e.pointerId);
  } catch {
    /* no active pointer (synthetic event, or pointer already released) */
  }
}

/**
 * The region the bar is allowed to live in: the canvas pane, NOT the window.
 *
 * The bar is `fixed`, so nothing kept it off the sidebar – and a default x of
 * 16 put it exactly there, covering the board list and the Link Files button.
 * The pane is MEASURED rather than derived from the sidebar's width, so the
 * backlinks panel (a second left panel) and a split view, which both move this
 * edge, are handled by the same code and cannot drift out of sync with it.
 */
function canvasArea(): { left: number; top: number; right: number; bottom: number } {
  const whole = {
    left: 0,
    top: 0,
    right: typeof window === "undefined" ? 0 : window.innerWidth,
    bottom: typeof window === "undefined" ? 0 : window.innerHeight,
  };
  if (typeof document === "undefined") return whole;
  const pane = document.querySelector("[data-wb-canvas-area]");
  if (!pane) return whole;
  const r = pane.getBoundingClientRect();
  // Before first layout the box is 0x0; clamping into that would pin the bar
  // to the top-left corner and it would never come back.
  if (r.width < 120 || r.height < 120) return whole;
  return { left: r.left, top: r.top, right: r.right, bottom: r.bottom };
}

function defaultPosition(): { x: number; y: number } {
  // Left edge of the canvas, vertically centred – one home shared by every
  // board. A saved drag always wins over this (the adopt-position effect
  // below); it only applies while there is nothing persisted yet, so existing
  // placements are left untouched. The height is a guess before first layout;
  // the mount effect re-centres exactly once the bar can be measured.

```

```

const a = canvasArea();
return defaultPensBarPosition(a, PENS_BAR_HEIGHT_ESTIMATE);
}

/**
 * Keep the bar inside the canvas pane. Measured in both axes, because the bar
 * can be a tall narrow rail or a wide short strip and a fixed width would
 * strand the horizontal one off the right edge.
 *
 * This also rescues a position persisted from a session with no sidebar, or
 * from a bigger window, or a tablet in the other orientation.
 *
 * Toolbar rects (tools row and, when detached, the zoom pill) are exclusion
 * zones, not just boundaries: the bar floats ABOVE them, so parking on top
 * would swallow their taps with no visible reason. Overlapping positions are
 * pushed out along the smallest penetration.
 */
function clampToCanvasArea(
  pos: { x: number; y: number },
  width: number,
  height: number
) {
  if (typeof window === "undefined") return pos;
  const a = canvasArea();
  const minX = a.left + 8;
  const minY = a.top + 8;
  const clamped = {
    x: Math.min(Math.max(minX, pos.x), Math.max(minX, a.right - width - 8)),
    y: Math.min(Math.max(minY, pos.y), Math.max(minY, a.bottom - height - 8)),
  };
  let out = clamped;
  for (const r of toolbarRects()) {
    out = pushOutOfRect(out, width, height, r);
  }
  // A push may have left the pane (toolbar hugging an edge); the pane wins,
  // because a bar half off screen is worse than a near miss.
  return {
    x: Math.min(Math.max(minX, out.x), Math.max(minX, a.right - width - 8)),
    y: Math.min(Math.max(minY, out.y), Math.max(minY, a.bottom - height - 8)),
  };
}

interface Rect {
  left: number;
  top: number;
  right: number;
  bottom: number;
}

/** Every toolbar rect: the tools row plus the detached zoom pill, if shown. */
function toolbarRects(): Rect[] {
  if (typeof document === "undefined") return [];
  const out: Rect[] = [];
  for (const el of document.querySelectorAll("[data-wb-toolbar]")) {
    const r = (el as HTMLElement).getBoundingClientRect();
    if (r.width < 8 || r.height < 8) continue;
    out.push({ left: r.left, top: r.top, right: r.right, bottom: r.bottom });
  }
  return out;
}

export function pushOutOfRect(
  pos: { x: number; y: number },
  width: number,
  height: number,
  r: Rect,
  area: Rect = canvasArea()
): { x: number; y: number } {
  const overlapX = Math.min(pos.x + width, r.right) - Math.max(pos.x, r.left);
  const overlapY = Math.min(pos.y + height, r.bottom) - Math.max(pos.y, r.top);
  if (overlapX <= 0 || overlapY <= 0) return pos;
  const M = 8;
  // How far each way the bar must travel to CLEAR the rect. (It used to move
  // by the overlap's width - enough only when the bar straddles an edge; a
  // bar narrower than the toolbar and inside it moved its own width and was
  // still on top, covering the page controls.)
  const moves = [
    { x: r.left - M - width, y: pos.y },

```

```

    { x: r.right + M, y: pos.y },
    { x: pos.x, y: r.top - M - height },
    { x: pos.x, y: r.bottom + M },
  ].filter(
    (m) => m.x >= area.left + 8 && m.x + width <= area.right - 8 && m.y >= area.top + 8 && m.y + height <= area.bottom - 8
  );
  if (!moves.length) return pos;
  moves.sort((p, q) => Math.hypot(p.x - pos.x, p.y - pos.y) - Math.hypot(q.x - pos.x, q.y - pos.y));
  return moves[0];
}

export function FavoritePensBar() {
  const pinnedPens = useWhiteboard((s) => s.pinnedPens);
  const pinnedBarOpen = useWhiteboard((s) => s.pinnedBarOpen);
  const storedPos = useWhiteboard((s) => s.pinnedBarPos);
  const tool = useWhiteboard((s) => s.tool);
  const style = useWhiteboard((s) => s.style);
  const applyPinnedPen = useWhiteboard((s) => s.applyPinnedPen);
  const pinCurrentPen = useWhiteboard((s) => s.pinCurrentPen);
  const updatePinnedPen = useWhiteboard((s) => s.updatePinnedPen);
  const removePinnedPen = useWhiteboard((s) => s.removePinnedPen);
  const reorderPinnedPens = useWhiteboard((s) => s.reorderPinnedPens);
  const setPinnedBarPos = useWhiteboard((s) => s.setPinnedBarPos);
  const togglePinnedBar = useWhiteboard((s) => s.togglePinnedBar);
  const vertical = useWhiteboard((s) => s.pinnedBarVertical);
  const toggleOrientation = useWhiteboard((s) => s.togglePinnedBarOrientation);
  // Subscribe so a dock-edge change re-renders: moving the toolbar does not
  // resize the canvas pane (so the observer below never fires), but it moves
  // the exclusion zone, and the render clamp must push the bar out of it.
  // The value itself is unused – the subscription is the point.
  useWhiteboard((s) => s.toolbarPosition);

  const barRef = useRef<HTMLDivElement>(null);
  const [pos, setPos] = useState<{ x: number; y: number }>(() => storedPos ?? defaultPosition());
  // Mirror of `pos` so drag-end can persist the final position without reading
  // it from inside a state updater (that would be a setState during render).
  const posRef = useRef(pos);
  posRef.current = pos;
  // Snapshot of whether a position persisted when we mounted, so
  // the mount-once effect below can tell "fresh workspace" from "late storage
  // load" without taking dependencies (a saved drag must always win).
  const hadStoredPosRef = useRef(storedPos != null);
  const [editingPin, setEditingPin] = useState<string | null>(null);

  // Bar drag (move the whole bar)
  const barDragRef = useRef<{ dx: number; dy: number; moved: boolean } | null>(null);
  const [movingBar, setMovingBar] = useState(false);

  // Pin drag (reorder)
  const pinDragRef = useRef<{
    id: string;
    fromIndex: number;
    startX: number;
    startY: number;
    dragging: boolean;
  } | null>(null);
  const [dragPinId, setDragPinId] = useState<string | null>(null);
  const [dropIndex, setDropIndex] = useState<number | null>(null);
  // Last tap per pin, for double-click / double-tap to open the editor.
  // Timed on pointer-up (not the dblclick event) because a mobile WebView
  // with zoom disabled never fires dblclick.
  const lastTapRef = useRef<Record<string, number>>({});
  /** Double-tap window. Long enough for a finger, short enough to feel instant. */
  const DOUBLE_TAP_MS = 350;

  // Adopt a position saved by an earlier session once it arrives from storage.
  useEffect(() => {
    if (storedPos) setPos(storedPos);
  }, [storedPos]);

  const splitSide = useWhiteboard((s) => s.splitSide);
  const splitRatio = useWhiteboard((s) => s.splitRatio);
  const splitBoardId = useWhiteboard((s) => s.splitBoardId);
  /** Where the canvas pane was last seen, so a move can carry the bar along. */
  const paneOriginRef = useRef<{ left: number; top: number } | null>(null);
  const followPane = () => {
    const area = canvasArea();

```

```

const prev = paneOriginRef.current;
paneOriginRef.current = { left: area.left, top: area.top };
setPos((p) => {
  const carried = prev ? { x: p.x + (area.left - prev.left), y: p.y + (area.top - prev.top) } : p;
  return clampToCanvasArea(
    carried,
    barRef.current?.offsetWidth ?? BAR_THICKNESS,
    barRef.current?.offsetHeight ?? 200
  );
});
});
};

// Re-clamp whenever the canvas pane changes size so the bar can never be
// stranded. A plain resize listener is not enough: the window can be resized
// while the app is backgrounded, Electron/Android restore a size after mount,
// and – the case this bar kept getting wrong – opening the sidebar or the
// backlinks panel narrows the pane without any window resize at all. So we
// observe the PANE, which changes in every one of those cases.
useEffect(() => {
  // Fresh workspace (nothing dragged + persisted yet): centre exactly with
  // the measured height – the useState initialiser had to guess it before
  // first layout. Runs once; any drag after this owns `pos` from then on,
  // and a position arriving late from storage still wins via the effect above.
  if (!hadStoredPosRef.current && barRef.current?.offsetHeight > 0) {
    setPos(defaultPensBarPosition(canvasArea(), barRef.current.offsetHeight));
  }
  const reclamp = () => followPane();
  reclamp();
  window.addEventListener("resize", reclamp);
  const ro = new ResizeObserver(reclamp);
  ro.observe(document.querySelector("[data-wb-canvas-area]") ?? document.documentElement);
  // The toolbar too: it grows (tools wrapping onto another row, the colour
  // row unfolding) without the pane changing size, and the bar was left
  // sitting on top of the page controls.
  for (const el of document.querySelectorAll("[data-wb-toolbar]")) ro.observe(el);
  return () => {
    window.removeEventListener("resize", reclamp);
    ro.disconnect();
  };
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, []);

// In split view the live pane can MOVE without changing size - swapping
// sides slides it from one half to the other at the same width - and a
// ResizeObserver never fires for a move. The bar was left behind, floating
// over the other document. So the pane's position is watched too: when the
// live half moves, the bar moves with it and keeps its spot inside it.
//
// A layout effect, not requestAnimationFrame. The DOM is already committed
// here, so the pane's new position can be read at once; waiting a frame
// added nothing but a dependency on RAF, which NEVER fires while the page is
// hidden - measured: the bar was left behind every time.
useLayoutEffect(() => {
  followPane();
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [splitSide, splitRatio, splitBoardId]);

// ---- moving the bar -----
const onGripPointerDown = (e: React.PointerEvent) => {
  e.preventDefault();
  e.stopPropagation();
  barDragRef.current = { dx: e.clientX - pos.x, dy: e.clientY - pos.y, moved: false };
  setMovingBar(true);
  capturePointer(e);
};

const onGripPointerMove = (e: React.PointerEvent) => {
  const drag = barDragRef.current;
  if (!drag) return;
  drag.moved = true;
  setPos(
    clampToCanvasArea(
      { x: e.clientX - drag.dx, y: e.clientY - drag.dy },
      barRef.current?.offsetWidth ?? BAR_THICKNESS,
      barRef.current?.offsetHeight ?? 200
    )
  );
};

```

```

const endBarDrag = useCallback(() => {
  const drag = barDragRef.current;
  barDragRef.current = null;
  if (!drag) return;
  setMovingBar(false);
  // A tap on the grip never moved: persisting would pointlessly rewrite
  // storage (and re-trigger the adopt-position effect) on every tap.
  if (drag.moved) setPinnedBarPos(posRef.current);
}, [setPinnedBarPos]);

// ---- reordering pins -----
const onPinPointerDown = (e: React.PointerEvent, pin: PinnedPen, index: number) => {
  // Only the primary button starts a drag; right-click opens the browser menu.
  if (e.button !== 0) return;
  pinDragRef.current = {
    id: pin.id,
    fromIndex: index,
    startX: e.clientX,
    startY: e.clientY,
    dragging: false,
  };
  capturePointer(e);
};

/**
 * Which slot a drag is over. Measured along the bar's own axis – using Y
 * while the bar is horizontal would make every drop land on the same slot.
 */
const indexFromPointer = (clientX: number, clientY: number): number => {
  const bar = barRef.current;
  if (!bar) return 0;
  const buttons = [...bar.querySelectorAll("[data-pin-index]")] as HTMLInputElement[];
  for (let i = 0; i < buttons.length; i++) {
    const r = buttons[i].getBoundingClientRect();
    const midpoint = vertical ? r.top + r.height / 2 : r.left + r.width / 2;
    const along = vertical ? clientY : clientX;
    if (along < midpoint) return i;
  }
  return buttons.length - 1;
};

const onPinPointerMove = (e: React.PointerEvent) => {
  const drag = pinDragRef.current;
  if (!drag) return;
  if (!drag.dragging) {
    const moved = Math.hypot(e.clientX - drag.startX, e.clientY - drag.startY);
    if (moved < DRAG_THRESHOLD_PX) return; // still a click, not a drag
    drag.dragging = true;
    // A drag is not a tap: forget tap history so a tap, drag, tap sequence
    // cannot combine into a phantom double-tap that opens the editor.
    lastTapRef.current = {};
    setDragPinId(drag.id);
  }
  setDropIndex(indexFromPointer(e.clientX, e.clientY));
};

const onPinPointerUp = (e: React.PointerEvent, pin: PinnedPen, index: number) => {
  const drag = pinDragRef.current;
  pinDragRef.current = null;

  if (drag?.dragging) {
    const target = indexFromPointer(e.clientX, e.clientY);
    if (target !== index) reorderPinnedPens(index, target);
  } else {
    // A press that never became a drag is a plain click: use this pen –
    // unless it is the second tap of a double-tap, which opens the editor.
    const now = Date.now();
    const last = lastTapRef.current[pin.id] ?? 0;
    if (now - last < DOUBLE_TAP_MS) {
      delete lastTapRef.current[pin.id];
      setEditingPin(pin.id);
    } else {
      lastTapRef.current[pin.id] = now;
      applyPinnedPen(pin.id);
    }
  }
  setDragPinId(null);
};

```

```

    setDropIndex(null);
  };

const cancelPinDrag = () => {
  // A cancelled press is not a tap: forget it, or a tap, cancel, tap
  // sequence would combine into a phantom double-tap that opens the editor.
  if (pinDragRef.current) delete lastTapRef.current[pinDragRef.current.id];
  pinDragRef.current = null;
  setDragPinId(null);
  setDropIndex(null);
};

if (!pinnedBarOpen) return null;

// Clamp at RENDER time, not only in a resize handler. `pos` is the position
// the user chose; this is where it is actually allowed to sit right now. A
// listener alone is not enough – a window can be resized while the app is
// backgrounded, and Electron/Android restore a size without a resize event –
// and a stale position leaves the bar stranded off screen where it cannot be
// dragged back.
const renderPos = clampToCanvasArea(
  pos,
  barRef.current?.offsetWidth ?? BAR_THICKNESS,
  barRef.current?.offsetHeight ?? 200
);

// Read through the accessors. This used to index `style["gelColor"]` by
// string, which stopped existing when per-tool ink moved into a map – so the
// pin you were actually drawing with never showed as selected.
const isPinActive = (pin: PinnedPen) =>
  tool === pin.tool &&
  getToolColor(style, pin.tool as ToolType).toLowerCase() === pin.color.toLowerCase() &&
  getToolSize(style, pin.tool as ToolType) === pin.size;

return (
  <div
    ref={barRef}
    className={cn(
      // touch-none + select-none: without them a finger drag on the grip or
      // a pin is hijacked by the browser for scroll/pan, the gesture dies
      // with a pointercancel, and the bar looks stuck where it is. Every
      // other touch-drag surface (canvas, minimap, page grip, steppers)
      // already has it; this one was missed. Taps still click, and the pin
      // editor lives in a portal outside this tree so its slider is unaffected.
      "fixed z-40 flex touch-none select-none items-center gap-1.5 rounded-2xl border border-border bg-backg
        round/95 p-1.5 shadow-lg backdrop-blur",
      vertical ? "flex-col" : "flex-row",
      movingBar && "cursor-grabbing select-none"
    )}
    style={
      vertical
        ? { left: renderPos.x, top: renderPos.y, width: BAR_THICKNESS }
        : { left: renderPos.x, top: renderPos.y, height: BAR_THICKNESS }
    }
    // The canvas binds window-level shortcuts; keep bar interaction local.
    onPointerDown={(e) => e.stopPropagation()}
  >
    {/* Grip – drag to move the whole bar */}
    <div
      className={cn(
        "flex cursor-grab items-center justify-center rounded-lg text-muted-foreground hover:bg-muted active
          :cursor-grabbing",
        vertical ? "w-full py-0.5" : "h-full px-0.5"
      )}
      onPointerDown={onGripPointerDown}
      onPointerMove={onGripPointerMove}
      onPointerUp={endBarDrag}
      onPointerCancel={endBarDrag}
      onLostPointerCapture={endBarDrag}
      title="Drag to move the favorites bar"
    >
      <GripVertical size={14} className={vertical ? "" : "rotate-90"} />
    </div>

    {pinnedPens.length === 0 && (
      <div className="px-1 pb-1 text-center text-[9px] leading-tight text-muted-foreground">
        Pin your pens
      </div>
    )}
  </div>

```

```

  })

  {pinnedPens.map((pin, index) => {
    const Icon = PEN_ICONS[pin.tool] ?? Pen;
    const active = isPinActive(pin);
    const isDragging = dragPinId === pin.id;
    const showDropLine = dropIndex === index && dragPinId !== null && !isDragging;

    return (
      <div key={pin.id} className={cn("relative", vertical ? "w-full" : "h-full")}>
        {showDropLine && (
          <div
            className={cn(
              "absolute rounded bg-primary",
              vertical ? "-top-1 left-1 right-1 h-0.5" : "-left-1 top-1 bottom-1 w-0.5"
            )}
          />
        )}
        <Popover
          open={editingPin === pin.id}
          onOpenChange={(o) => setEditingPin(o ? pin.id : null)}
        >
          <PopoverTrigger asChild>
            <button
              data-pin-index={index}
              onPointerDown={(e) => onPinPointerDown(e, pin, index)}
              onPointerMove={onPinPointerMove}
              onPointerUp={(e) => onPinPointerUp(e, pin, index)}
              onPointerCancel={cancelPinDrag}
              onLostPointerCapture={cancelPinDrag}
              onContextMenu={(e) => {
                e.preventDefault();
                setEditingPin(pin.id);
              }}
              // The popover must open from the edit affordance, never from
              // a plain click – a plain click selects the pen.
              onClick={(e) => e.preventDefault()}
              title={` ${PEN_TYPE_PRESETS[pin.tool].label} · ${pin.size}px – click to use, double-click for
options, drag to reorder, right-click or long-press to edit`}
              className={cn(
                "relative flex w-full items-center justify-center rounded-xl border transition-all",
                active
                  ? "border-primary bg-primary/10 ring-2 ring-primary"
                  : "border-transparent hover:bg-muted",
                isDragging && "opacity-40"
              )}
              style={{ height: PIN_SIZE }}
            >
              <Icon size={18} className="drop-shadow-sm" />
              {/* Colour + thickness read-out: the swatch height tracks the
pen's actual size so two pins of one type are tellable. */}
              <span
                className="absolute bottom-1 left-1/2 -translate-x-1/2 rounded-full border border-backgrou
nd"
                style={{
                  backgroundColor: pin.color,
                  width: 18,
                  height: Math.max(2, Math.min(7, pin.size / 4)),
                }}
              />
            </button>
          </PopoverTrigger>
          <PopoverContent side={pos.x < 260 ? "right" : "left"} align="start" className="w-60 space-y-3 p-
3">
            <PinEditor
              pin={pin}
              onChange={(patch) => updatePinnedPen(pin.id, patch)}
              onMoveEarlier={
                index > 0 ? () => reorderPinnedPens(index, index - 1) : undefined
              }
              onMoveLater={
                index < pinnedPens.length - 1
                  ? () => reorderPinnedPens(index, index + 1)
                  : undefined
              }
              onDelete={() => {
                removePinnedPen(pin.id);
                setEditingPin(null);
              }}
            />
          </PopoverContent>
        </div>
      )
    )
  })}

```

```

        }}
      />
    </PopoverContent>
  </Popover>
</div>
);
}}

{/* Add – pins whatever pen is currently selected, then opens its editor */}
<button
  onClick={() => {
    const id = pinCurrentPen();
    if (id) setEditingPin(id);
  }}
  title="Pin the current pen to favorites"
  className={cn(
    "flex items-center justify-center rounded-xl border border-dashed border-border text-muted-foreground
    d transition-colors hover:border-primary hover:text-primary",
    vertical ? "w-full py-2" : "h-full px-2"
  )}
>
  <Plus size={16} />
</button>

<button
  onClick={toggleOrientation}
  title={vertical ? "Rotate: lay the bar flat" : "Rotate: stand the bar upright"}
  aria-label="Rotate the favorites bar"
  className={cn(
    "flex items-center justify-center rounded-lg text-muted-foreground transition-colors hover:bg-muted
    hover:text-foreground",
    vertical ? "w-full py-1" : "h-full px-1"
  )}
>
  <RotateCw size={13} />
</button>

<button
  onClick={togglePinnedBar}
  title="Hide favorites bar"
  className={cn(
    "flex items-center justify-center rounded-lg text-muted-foreground hover:text-foreground",
    vertical ? "w-full py-0.5" : "h-full px-0.5"
  )}
>
  <X size={12} />
</button>
</div>
);
}

/** Type / colour / size editor for one pinned pen. */
function PinEditor({
  pin,
  onChange,
  onMoveEarlier,
  onMoveLater,
  onDelete,
}): {
  pin: PinnedPen;
  onChange: (patch: Partial<Omit<PinnedPen, "id">>) => void;
  /** Absent at the ends of the list, where there is nowhere to move. */
  onMoveEarlier?: () => void;
  onMoveLater?: () => void;
  onDelete: () => void;
}) {
  const [customColor, setCustomColor] = useState(pin.color);

  return (
    <>
      <div className="flex items-center gap-1.5 text-xs font-semibold uppercase tracking-wider text-muted-foreground">
        <Star size={12} /> Favorite pen
      </div>

      <div>
        <div className="mb-1 text-[11px] text-muted-foreground">Pen type</div>
        <div className="grid grid-cols-3 gap-1">

```

```

    {(Object.keys(PEN_TYPE_PRESETS) as StrokeStyle[]).map((st) => {
      const Icon = PEN_ICONS[st];
      return (
        <button
          key={st}
          onClick={() => onChange({ tool: st })}
          className={cn(
            "flex flex-col items-center gap-0.5 rounded-lg border p-1.5 text-[10px] font-medium transiti
            on-colors",
            pin.tool === st
              ? "border-primary bg-primary/10 text-primary"
              : "border-border hover:bg-muted"
          )}
        >
          <Icon size={14} />
          {PEN_TYPE_PRESETS[st].label}
        </button>
      );
    })}
  </div>
</div>

<div>
  <div className="mb-1 text-[11px] text-muted-foreground">Color</div>
  <div className="flex flex-wrap gap-1">
    {PEN_COLORS.map((c) => (
      <button
        key={c}
        onClick={() => onChange({ color: c })}
        className={cn(
          "h-5 w-5 rounded-full border transition-transform",
          pin.color === c ? "scale-110 border-foreground ring-2 ring-primary" : "border-border"
        )}
        style={{ backgroundColor: c }}
        title={c}
      />
    ))}
    { /* Any colour, not just the presets */ }
    <label
      className="flex h-5 w-5 cursor-pointer items-center justify-center rounded-full border border-dash
      ed border-border text-[9px]"
      title="Custom color"
    >
      <input
        type="color"
        value={customColor}
        onChange={(e) => {
          setCustomColor(e.target.value);
          onChange({ color: e.target.value });
        }}
        className="h-0 w-0 opacity-0"
      />
    </label>
  </div>
</div>

<div>
  <div className="mb-1 flex items-center justify-between text-[11px] text-muted-foreground">
    <span>Size</span>
    <span className="tabular-nums">{pin.size}px</span>
  </div>
  <div className="flex items-center gap-2">
    <StepButton
      direction={-1}
      value={pin.size}
      min={1}
      max={64}
      onChange={(size) => onChange({ size })}
    />
    <Slider
      value={[pin.size]}
      min={1}
      max={64}
      step={1}
      onValueChange={([v]) => onChange({ size: v })}
    />
    <StepButton

```

```

        direction={1}
        value={pin.size}
        min={1}
        max={64}
        onChange={(size) => onChange({ size })}}
      />
    </div>
  </div>

  <div>
    <div className="mb-1 text-[11px] text-muted-foreground">Position in the bar</div>
    <div className="flex gap-1">
      <button
        onClick={onMoveEarlier}
        disabled={!onMoveEarlier}
        className="flex flex-1 items-center justify-center gap-1 rounded-lg border border-border px-2 py-1
          .5 text-[11px] font-medium transition-colors hover:bg-muted disabled:pointer-events-none disab
            led:opacity-40"
      >
        <ChevronUp size={13} /> Earlier
      </button>
      <button
        onClick={onMoveLater}
        disabled={!onMoveLater}
        className="flex flex-1 items-center justify-center gap-1 rounded-lg border border-border px-2 py-1
          .5 text-[11px] font-medium transition-colors hover:bg-muted disabled:pointer-events-none disab
            led:opacity-40"
      >
        Later <ChevronDown size={13} />
      </button>
    </div>
  </div>

  <Button variant="ghost" size="sm" className="w-full text-destructive" onClick={onDelete}>
    <Trash2 size={14} className="mr-1" /> Remove pin
  </Button>
</>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/FavoritePensBar.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/FileDropZone.tsx ===
// File 52 of 194 | Lines: 305 | Size: 12358 bytes
=====

```

```
"use client";
```

```

/**
 * Drop files onto the app
 * =====
 * Drop a PDF and it opens as a notebook you can write on, pages running down
 * the canvas. Drop images and they become pages in the notebook you have open.
 *
 * The listeners are on `window`, not on a drop target, for a reason that bites
 * hard in a desktop build: Chromium's DEFAULT action for a dropped file is to
 * navigate to it. Inside Electron that replaces the whole app with a file
 * viewer and there is no back button – the user's session is simply gone. So
 * dragover and drop are cancelled globally, and only then routed.
 */

```

```

import React from "react";
import { FileText, ImagePlus, Loader2 } from "lucide-react";
import { useWhiteboard, useActiveBoard } from "@store/whiteboard";
import { isNotebook } from "@lib/whiteboard/notebook";
import { storeImage } from "@lib/whiteboard/imagestore";
import { officeKindOf } from "@lib/whiteboard/officeImport";
import { cn } from "@lib/utils";

```

```

function isPdf(file: File): boolean {
  return file.type === "application/pdf" || /\.pdf$/i.test(file.name);
}

```

```

/** Word / PowerPoint (and the old .doc/.ppt, which get a clear message). */

```

```

function isOffice(file: File): boolean {
  return officeKindOf(file) !== null;
}

function isImage(file: File): boolean {
  return file.type.startsWith("image/") || /\. (png|jpe?g|gif|webp|bmp|svg)$/i.test(file.name);
}

/**
 * Reads an image, measures it, and puts the pixels in IndexedDB.
 *
 * Measuring keeps pages fitted rather than stretched. Storing keeps the bytes
 * out of the workspace: a single image was measured at 908 KB against a ~5 MB
 * localStorage budget, so dropping a handful of photos would fill it, and the
 * over-quota fallback discards image data permanently.
 */
/**
 * Imports picture files into a notebook living in `folderId`, reusing the
 * drop path: pictures become pages of a fresh notebook that opens at once.
 * Shared by the drop zone and the sidebar's "Import here" menu – one path,
 * not two.
 */
export async function importPicturesAsNotebook(files: File[], folderId: string | null): Promise<void> {
  const images = files.filter(isImage);
  if (images.length === 0) {
    throw new Error(
      files.length === 1 ? `${files[0].name} is not a picture.` : "Those files are not pictures."
    );
  }
  const api = useWhiteboard.getState();
  const measured = await Promise.all(images.map(readImage));
  const id = api.createNotebook("Imported pictures", folderId);
  api.setActiveBoard(id);
  useWhiteboard.getState().addImagePages(measured);
}

function readImage(file: File): Promise<{ src: string; width: number; height: number }> {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.onload = () => {
      const dataUrl = String(reader.result);
      const img = new Image();
      img.onload = () => {
        storeImage(dataUrl).then((src) =>
          resolve({ src, width: img.naturalWidth, height: img.naturalHeight })
        );
      };
      img.onerror = () => reject(new Error(`Could not read the image "${file.name}".`));
      img.src = dataUrl;
    };
    reader.onerror = () => reject(new Error(`Could not read "${file.name}".`));
    reader.readAsDataURL(file);
  });
}

export function FileDropZone() {
  const importPdfAsNotebook = useWhiteboard((s) => s.importPdfAsNotebook);
  const addImagePages = useWhiteboard((s) => s.addImagePages);
  const createNotebook = useWhiteboard((s) => s.createNotebook);
  const activeBoard = useActiveBoard();

  const [dragging, setDragging] = React.useState(false);
  const [busy, setBusy] = React.useState<string | null>(null);
  const [error, setError] = React.useState<string | null>(null);

  // dragenter/dragleave fire for every element the cursor crosses, so a naive
  // boolean flickers the overlay while you move across the page. Counting
  // enters against leaves is the standard fix.
  const depth = React.useRef(0);

  // Kept in a ref so the window listeners, registered once, never act on a
  // stale board.
  const latest = React.useRef({ activeBoard, importPdfAsNotebook, addImagePages, createNotebook });
  React.useEffect(() => {
    latest.current = { activeBoard, importPdfAsNotebook, addImagePages, createNotebook };
  });
}

```

```

* Opens files the way a drop does: each PDF becomes a notebook, pictures
* become pages. Used by drag-and-drop AND by files other apps send us
* ("Open with" / "Share" on Android - see MainActivity.handleIncoming).
*/
const openFilesRef = React.useRef<(files: File[]) => Promise<void>>(async () => {});
openFilesRef.current = async (files: File[]) => {
  if (files.length === 0) return;
  setError(null);
  // PDFs, Word and PowerPoint files each become their own notebook -
  // merging two documents into one would be impossible to undo without
  // deleting pages by hand.
  const pdfs = files.filter((f) => isPdf(f) || isOffice(f));
  const images = files.filter((f) => !isPdf(f) && !isOffice(f) && isImage(f));
  const ignored = files.filter((f) => !isPdf(f) && !isOffice(f) && !isImage(f));
  const problems: string[] = [];
  try {
    for (const file of pdfs) {
      setBusy(`Opening ${file.name}...`);
      try {
        await useWhiteboard.getState().importDocumentAsNotebook(file, null, (p) =>
          setBusy(`${file.name}: ${p.label}`)
        );
      } catch (err) {
        // One bad file must not stop the others that came with it.
        problems.push((err as Error).message || `Could not open "${file.name}".`);
      }
    }
    if (images.length > 0) {
      setBusy(images.length === 1 ? "Adding image..." : `Adding ${images.length} images...`);
      const measured = await Promise.all(images.map(readImage));
      // Images need somewhere paged to go. If the open document is a
      // canvas (or a PDF notebook was just made), start a notebook for
      // them rather than silently doing nothing.
      const board = useWhiteboard.getState().boards.find(
        (b) => b.id === useWhiteboard.getState().activeBoardId
      );
      if (!board || !isNotebook(board)) {
        useWhiteboard.getState().createNotebook("Dropped images", null);
      }
      useWhiteboard.getState().addImagePages(measured);
    }
    if (ignored.length > 0 && pdfs.length === 0 && images.length === 0) {
      setError(
        ignored.length === 1
          ? `${ignored[0].name}" can't be opened: ProNotes opens PDF, Word, PowerPoint and pictures.`
          : "Those files can't be opened: ProNotes opens PDF, Word, PowerPoint and pictures."
      );
    } else if (problems.length) {
      setError(problems[0]);
    }
  } catch (err) {
    // Import fails for real reasons – an encrypted PDF, a corrupt file, no
    // storage left. Silence would be indistinguishable from a slow import.
    setError((err as Error).message || "Could not open that file.");
  } finally {
    setBusy(null);
  }
};

// Files from other apps. The Android side copies each into the app's cache
// and calls this with its path (retrying until it exists); the bytes come
// back through Capacitor's local file server. Registered only once the
// workspace has loaded, so an import can't be overwritten by the load.
const isReady = useWhiteboard((s) => s.isReady);
React.useEffect(() => {
  if (!isReady || typeof window === "undefined") return;
  type Incoming = { path?: string; name?: string; type?: string; error?: string };
  const w = window as unknown as { __pronotesReceiveFiles?: (items: Incoming[]) => void };
  w.__pronotesReceiveFiles = (items: Incoming[]) => {
    void (async () => {
      const files: File[] = [];
      const problems: string[] = [];
      for (const it of items) {
        if (it.error || !it.path) {
          problems.push(it.error || "A file could not be read.");
          continue;
        }
        try {

```

```

        setBusy(`Opening ${it.name ?? "file"}...`);
        const { Capacitor } = await import("@capacitor/core");
        const res = await fetch(Capacitor.convertFileSrc(it.path));
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const blob = await res.blob();
        const name = it.name || "Shared file";
        const type = it.type || blob.type || (/\.pdf$/i.test(name) ? "application/pdf" : "");
        files.push(new File([blob], name, { type }));
      } catch (err) {
        problems.push(`Could not open "${it.name ?? "the file"}": ${(err as Error).message}`);
      }
    }
    setBusy(null);
    await openFilesRef.current(files);
    if (problems.length) setError(problems[0]);
  })();
};
return () => {
  delete w.__pronotesReceiveFiles;
};
}, [isReady]);

React.useEffect(() => {
  const hasFiles = (e: DragEvent) =>
    Array.from(e.dataTransfer?.types ?? []).includes("Files");

  const onDragEnter = (e: DragEvent) => {
    if (!hasFiles(e)) return;
    e.preventDefault();
    depth.current += 1;
    setDragging(true);
  };

  const onDragOver = (e: DragEvent) => {
    if (!hasFiles(e)) return;
    // Without this the drop never fires and Chromium navigates to the file,
    // which in the desktop build throws the whole app away.
    e.preventDefault();
    if (e.dataTransfer) e.dataTransfer.dropEffect = "copy";
  };

  const onDragLeave = (e: DragEvent) => {
    if (!hasFiles(e)) return;
    depth.current = Math.max(0, depth.current - 1);
    if (depth.current === 0) setDragging(false);
  };

  const onDrop = async (e: DragEvent) => {
    if (!hasFiles(e)) return;
    e.preventDefault();
    depth.current = 0;
    setDragging(false);
    setError(null);

    const files = Array.from(e.dataTransfer?.files ?? []);
    await openFilesRef.current(files);
  };

  window.addEventListener("dragenter", onDragEnter);
  window.addEventListener("dragover", onDragOver);
  window.addEventListener("dragleave", onDragLeave);
  window.addEventListener("drop", onDrop);
  return () => {
    window.removeEventListener("dragenter", onDragEnter);
    window.removeEventListener("dragover", onDragOver);
    window.removeEventListener("dragleave", onDragLeave);
    window.removeEventListener("drop", onDrop);
  };
}, []);

// Dismiss an error on the next interaction rather than making the user aim
// at a close button.
React.useEffect(() => {
  if (!error) return;
  const clear = () => setError(null);
  const t = setTimeout(clear, 8000);
  window.addEventListener("pointerdown", clear, { once: true });
  return () => {

```

```

    clearTimeout(t);
    window.removeEventListener("pointerdown", clear);
  };
}, [error]);

const notebookOpen = isNotebook(activeBoard);

return (
  <>
    {dragging && (
      <div className="pointer-events-none fixed inset-0 z-[95] flex items-center justify-center bg-primary/10 backdrop-blur-[2px]">
        <div className="flex flex-col items-center gap-3 rounded-2xl border-2 border-dashed border-primary bg-white/95 px-8 py-6 shadow-2xl dark:bg-zinc-900/95">
          <div className="flex gap-3 text-primary">
            <FileText size={28} />
            <ImagePlus size={28} />
          </div>
          <p className="text-sm font-semibold">Drop to open</p>
          <p className="max-w-[260px] text-center text-[11px] leading-snug text-muted-foreground">
            A <strong>PDF</strong> becomes a notebook you can write on.{ " "}
            <strong>Images</strong> become pages
            {notebookOpen ? " in this notebook" : " in a new notebook"}.
          </p>
        </div>
      </div>
    )}

    {(busy || error) && (
      <div
        className={cn(
          "fixed bottom-6 left-1/2 z-[96] flex -translate-x-1/2 items-center gap-2 rounded-xl border px-3 py-2 text-[12px] shadow-lg",
          error
            ? "border-destructive/40 bg-destructive/10 text-destructive"
            : "border-border bg-white/95 text-foreground dark:bg-zinc-900/95"
        )}
        role="status"
      >
        {busy && <Loader2 size={14} className="animate-spin" />}
        <span>{busy ?? error}</span>
      </div>
    )}
  </>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/FileDropZone.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/GraphView.tsx ===
// File 53 of 194 | Lines: 1519 | Size: 60179 bytes
=====

```

```
"use client";
```

```

/**
 * Interactive Knowledge Graph View (Obsidian Canvas & Network Grade v4)
 * =====
 * Comprehensive interactive knowledge network connecting boards, folders,
 * notes/cards, and #tags with:
 * - Full tablet multi-touch pinch-to-zoom & dual-pointer pan
 * - S Pen stylus hover peek cards & pen gesture isolation
 * - Drag vs tap disambiguation with Euclidean deadband
 * - Global Universe vs Local Neighborhood subgraphs
 * - Node Inspector Drawer with backlink & outbound connection analysis
 * - Tag clustering (#tags) with semantic relationship edges
 * - Interactive Minimap Radar with camera viewport tracking
 * - True bounding-box Auto-Fit (Zoom to Extents)
 * - Vector SVG & 2x Retina PNG export
 * - Organic force-directed physics with settle/reheat
 */

```

```

import React, { useEffect, useMemo, useRef, useState, useCallback } from "react";
import { useWhiteboard } from "@store/whiteboard";

```

```

import {
  buildGraph,
  parseElementNodeId,
  filterGraphNeighborhood,
  type GraphData,
  type GraphNode,
} from "@lib/whiteboard/links";
import {
  X,
  Star,
  Search,
  Network,
  ZoomIn,
  ZoomOut,
  RotateCcw,
  Maximize2,
  Download,
  Filter,
  Compass,
  Layers,
  Hash,
  FileText,
  Folder as FolderIcon,
  Tag as TagIcon,
  Sparkles,
  Pin,
  ExternalLink,
  Play,
  Pause,
  ArrowUpRight,
  ArrowDownLeft,
} from "lucide-react";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";

interface GraphViewProps {
  open: boolean;
  onClose: () => void;
}

interface SimNode extends GraphNode {
  x: number;
  y: number;
  vx: number;
  vy: number;
  degree: number;
  isPinned?: boolean;
}

interface SimEdge {
  id: string;
  source: string;
  target: string;
  kind?: "link" | "parent" | "tag";
  label?: string;
  color?: string;
  linkType?: string;
}

/**
 * SVG node silhouettes corresponding to whiteboard element types.
 */
function elementNodeShape(elementType: string | undefined, r: number): React.ReactNode {
  switch (elementType) {
    case "text":
      return <rect x={-r * 1.35} y={-r * 0.72} width={r * 2.7} height={r * 1.44} rx={3} />;
    case "sticky":
      return (
        <path d={`M ${-r} ${-r} H ${r} V ${r * 0.35} L ${r * 0.35} ${r} H ${-r} Z`} />
      );
    case "image":
      return <rect x={-r * 1.2} y={-r * 0.9} width={r * 2.4} height={r * 1.8} rx={2} />;
    case "diamond":
      return <path d={`M 0 ${-r} L ${r} 0 L 0 ${r} L ${-r} 0 Z`} />;
    case "triangle":
      return <path d={`M 0 ${-r} L ${r} ${r * 0.8} L ${-r} ${r * 0.8} Z`} />;
    case "star":
      return (

```

```

    <path
      d={Array.from({ length: 10 }, (_, i) => {
        const rad = i % 2 === 0 ? r : r * 0.45;
        const a = (Math.PI / 5) * i - Math.PI / 2;
        return `${i === 0 ? "M" : "L"} ${Math.cos(a) * rad.toFixed(2)} ${Math.sin(a) * rad.toFixed(2)}
      ;
      }).join(" ") + " Z"}
    />
  );
  case "ellipse":
    return <ellipse rx={r * 1.15} ry={r * 0.85} />;
  case "line":
  case "arrow":
    return <rect x={-r * 1.3} y={-r * 0.3} width={r * 2.6} height={r * 0.6} rx={r * 0.3} />;
  case "stroke":
    return <path d={`M ${-r} ${r * 0.5} Q 0 ${-r * 1.3} ${r} ${r * 0.5}`} fill="none" strokeLinecap="round"
    />;
  case "tag":
    return (
      <path
        d={`M ${-r * 1.2} ${-r * 0.6} L ${r * 0.5} ${-r * 0.6} L ${r * 1.2} 0 L ${r * 0.5} ${r * 0.6} L ${-r
          * 1.2} ${r * 0.6} Z`}
      />
    );
  default:
    return <rect x={-r} y={-r} width={r * 2} height={r * 2} rx={4} />;
}
}

export function GraphView({ open, onClose }: GraphViewProps) {
  const boards = useWhiteboard((s) => s.boards);
  const folders = useWhiteboard((s) => s.folders);
  const activeBoardId = useWhiteboard((s) => s.activeBoardId);
  const setActiveBoard = useWhiteboard((s) => s.setActiveBoard);
  const revealFolder = useWhiteboard((s) => s.revealFolder);
  const focusElement = useWhiteboard((s) => s.focusElement);
  const customLinkTypes = useWhiteboard((s) => s.customLinkTypes);

  const containerRef = useRef<HTMLDivElement>(null);
  const svgRef = useRef<SVGSVGElement>(null);

  // Layout Viewport
  const [dims, setDims] = useState({ width: 1000, height: 700 });
  const [zoom, setZoom] = useState(1);
  const [pan, setPan] = useState({ x: 0, y: 0 });

  // Mode & Filtering
  const [graphMode, setGraphMode] = useState<"global" | "local">("global");
  const [localFocusId, setLocalFocusId] = useState<string | null>(null);
  const [localDepth, setLocalDepth] = useState<number>(1);

  const [showFolders, setShowFolders] = useState(true);
  const [showElements, setShowElements] = useState(true);
  const [showTags, setShowTags] = useState(true);
  const [showOrphans, setShowOrphans] = useState(true);
  const [physicsEnabled, setPhysicsEnabled] = useState(true);
  const [filterMenuOpen, setFilterMenuOpen] = useState(false);

  // Search & Inspection
  const [searchQuery, setSearchQuery] = useState("");
  const [selectedNodeId, setSelectedNodeId] = useState<string | null>(null);
  const [hoveredNode, setHoveredNode] = useState<string | null>(null);
  const [hoverAnchor, setHoverAnchor] = useState<{ x: number; y: number } | null>(null);

  // Multi-Touch & Gesture Refs
  const activePointersRef = useRef<Map<number, { x: number; y: number; pointerType: string }>>>(new Map());
  const pinchStartRef = useRef<{ dist: number; zoom: number; mid: { x: number; y: number }; pan: { x: number;
    y: number } } | null>(null);
  const canvasPanStartRef = useRef<{ x: number; y: number } | null>(null);
  const isPanningRef = useRef(false);

  // Node Drag & Tap Disambiguation Refs
  const [dragNodeId, setDragNodeId] = useState<string | null>(null);
  const [nodePositions, setNodePositions] = useState<Record<string, { x: number; y: number }>>>({});
  const [pinnedNodeIds, setPinnedNodeIds] = useState<Set<string>>>(new Set());
  const nodeDragStartRef = useRef<{ x: number; y: number } | null>(null);
  const nodeHasDraggedRef = useRef<boolean>(false);
  const lastTapRef = useRef<{ id: string; time: number } | null>(null);

```

```

// Cached positions across renders to avoid layout scrambling
const cachedPositionsRef = useRef<Map<string, { x: number; y: number }>>(new Map());

// Set local focus to active board on open
useEffect(() => {
  if (open && activeBoardId) {
    setLocalFocusId(activeBoardId);
  }
}, [open, activeBoardId]);

// Track container size
useEffect(() => {
  if (!open) return;
  const update = () => {
    if (containerRef.current) {
      const rect = containerRef.current.getBoundingClientRect();
      if (rect.width > 100 && rect.height > 100) {
        setDims({ width: rect.width, height: rect.height });
      }
    }
  };
  update();
  const timer = setTimeout(update, 50);
  window.addEventListener("resize", update);
  return () => {
    clearTimeout(timer);
    window.removeEventListener("resize", update);
  };
}, [open]);

// Keyboard shortcut: Escape to close
useEffect(() => {
  if (!open) return;
  const onKeyDown = (e: KeyboardEvent) => {
    if (e.key === "Escape") {
      if (selectedNodeId) {
        setSelectedNodeId(null);
      } else {
        onClose();
      }
    }
  };
  window.addEventListener("keydown", onKeyDown);
  return () => window.removeEventListener("keydown", onKeyDown);
}, [open, selectedNodeId, onClose]);

// 1. Build Global Graph Data
const rawGlobalGraphData = useMemo<GraphData>(() => {
  return buildGraph(boards, customLinkTypes, folders, {
    includeTags: true,
    includeElements: true,
  });
}, [boards, customLinkTypes, folders]);

// 2. Select between Global and Local Subgraph
const baseGraphData = useMemo<GraphData>(() => {
  if (graphMode === "local") {
    const focusTarget = localFocusId || activeBoardId || boards[0]?.id;
    if (focusTarget) {
      return filterGraphNeighborhood(rawGlobalGraphData, focusTarget, localDepth);
    }
  }
  return rawGlobalGraphData;
}, [graphMode, localFocusId, activeBoardId, boards, localDepth, rawGlobalGraphData]);

// 3. Assemble and Filter Edges
const allEdges = useMemo<SimEdge[]>(() => {
  const wikilinkEdges: SimEdge[] = baseGraphData.edges.map((e) => ({
    id: e.id,
    source: e.source,
    target: e.target,
    kind: (e.linkType === "tag" ? "tag" : "link") as "link" | "tag",
    label: e.label,
    color: e.color || "#6366f1",
    linkType: e.linkType,
  }));
});

```

```

// Folder parent-child edges (only in global mode or when folder is included)
const folderParentEdges: SimEdge[] = [];
if (graphMode === "global") {
  for (const b of boards) {
    if (b.folderId) {
      folderParentEdges.push({
        id: `parent-${b.folderId}-${b.id}`,
        source: b.folderId,
        target: b.id,
        kind: "parent",
        color: "#9ca3af",
      });
    }
  }
}

return [...wikilinkEdges, ...folderParentEdges];
}, [baseGraphData, boards, graphMode]);

// 4. Force-Directed Layout Simulation (Cohesive & Organic)
const simulatedNodes = useMemo<SimNode[]>(() => {
  if (!open || dims.width === 0) return [];
  if (baseGraphData.nodes.length === 0 && folders.length === 0) return [];

  const cx = dims.width / 2;
  const cy = dims.height / 2;
  const initialRadius = Math.min(dims.width, dims.height) / 2.8;

  // Combine base nodes with folder vertices
  const allItems: GraphNode[] = [
    ...baseGraphData.nodes,
    ...(graphMode === "global"
      ? folders.map((f) => ({
          id: f.id,
          name: f.name,
          inDegree: 0,
          outDegree: 0,
          orphan: true,
          starred: false,
          tagCount: 0,
          folderId: null,
          kind: "folder" as const,
          color: f.color || "#f59e0b",
        })))
    : []],
  ];

  // Deduplicate by ID
  const uniqueItemsMap = new Map<string, GraphNode>();
  for (const item of allItems) {
    if (!uniqueItemsMap.has(item.id)) {
      uniqueItemsMap.set(item.id, item);
    }
  }
  const items = Array.from(uniqueItemsMap.values());

  const simNodes: SimNode[] = items.map((n, i) => {
    const cached = cachedPositionsRef.current.get(n.id);
    const angle = (i / Math.max(1, items.length)) * Math.PI * 2;
    const board = boards.find((b) => b.id === n.id);
    const folder = folders.find((f) => f.id === n.id);
    const degree = allEdges.filter((e) => e.source === n.id || e.target === n.id).length;

    const initX = cached ? cached.x : cx + Math.cos(angle) * (initialRadius * 0.85);
    const initY = cached ? cached.y : cy + Math.sin(angle) * (initialRadius * 0.85);

    return {
      ...n,
      x: initX,
      y: initY,
      vx: 0,
      vy: 0,
      degree,
      starred: !!board?.starred,
      orphan: degree === 0,
      color: n.color || board?.color || folder?.color || (n.kind === "tag" ? "#10b981" : "#3b82f6"),
      isPinned: pinnedNodeIds.has(n.id),
    };
  });
};

```

```

});

// Run force layout simulation if physics enabled
if (physicsEnabled) {
  const nodeMap = new Map(simNodes.map((n) => [n.id, n]));
  const iterations = 85;
  const k = 120; // Ideal spring rest length
  const repulsion = 12000;
  const attraction = 0.045;
  const damping = 0.86;

  for (let iter = 0; iter < iterations; iter++) {
    const cooling = Math.pow(0.96, iter);

    // Repulsion between all node pairs
    for (let i = 0; i < simNodes.length; i++) {
      const a = simNodes[i];
      for (let j = i + 1; j < simNodes.length; j++) {
        const b = simNodes[j];
        const dx = b.x - a.x;
        const dy = b.y - a.y;
        const dist = Math.max(16, Math.hypot(dx, dy));
        const force = (repulsion / (dist * dist)) * cooling;
        const fx = (dx / dist) * force;
        const fy = (dy / dist) * force;
        if (!a.isPinned) {
          a.vx -= fx;
          a.vy -= fy;
        }
        if (!b.isPinned) {
          b.vx += fx;
          b.vy += fy;
        }
      }
    }

    // Attraction along edges
    for (const edge of allEdges) {
      const a = nodeMap.get(edge.source);
      const b = nodeMap.get(edge.target);
      if (!a || !b) continue;
      const dx = b.x - a.x;
      const dy = b.y - a.y;
      const dist = Math.max(1, Math.hypot(dx, dy));
      const force = (dist - k) * attraction * cooling;
      const fx = (dx / dist) * force;
      const fy = (dy / dist) * force;
      if (!a.isPinned) {
        a.vx += fx;
        a.vy += fy;
      }
      if (!b.isPinned) {
        b.vx -= fx;
        b.vy -= fy;
      }
    }

    // Integration with radial soft gravity (organic cohesion in open space)
    for (const n of simNodes) {
      if (n.isPinned) continue;
      n.vx += (cx - n.x) * 0.0035;
      n.vy += (cy - n.y) * 0.0035;
      n.vx *= damping;
      n.vy *= damping;
      n.x += n.vx;
      n.y += n.vy;
    }
  }

  // Cache updated simulated coordinates
  for (const n of simNodes) {
    cachedPositionsRef.current.set(n.id, { x: n.x, y: n.y });
  }

  return simNodes;
}, [open, dims, baseGraphData, folders, graphMode, boards, allEdges, physicsEnabled, pinnedNodeIds]);

```

```
// Combine simulated positions with user drag overrides
const nodes = useMemo<SimNode[]>(() => {
  return simulatedNodes.map((n) => {
    const override = nodePositions[n.id];
    if (override) {
      return { ...n, x: override.x, y: override.y };
    }
    return n;
  });
}, [simulatedNodes, nodePositions]);

// 5. Apply User Filtering (Folders, Elements, Tags, Orphans)
const filteredNodes = useMemo<SimNode[]>(() => {
  return nodes.filter((n) => {
    if (!showFolders && n.kind === "folder") return false;
    if (!showElements && n.kind === "element") return false;
    if (!showTags && n.kind === "tag") return false;
    if (!showOrphans && n.orphan) return false;
    return true;
  });
}, [nodes, showFolders, showElements, showTags, showOrphans]);

const visibleNodeIds = useMemo(() => new Set(filteredNodes.map((n) => n.id)), [filteredNodes]);

const visibleEdges = useMemo<SimEdge[]>(() => {
  return allEdges.filter((e) => visibleNodeIds.has(e.source) && visibleNodeIds.has(e.target));
}, [allEdges, visibleNodeIds]);

// 6. Search Highlighting & Neighborhood Illumination
const searchQueryNormalized = searchQuery.trim().toLowerCase();

const matchingNodeIds = useMemo<Set<string> | null>(() => {
  if (!searchQueryNormalized) return null;
  const matches = new Set<string>();
  for (const n of filteredNodes) {
    if (
      n.name.toLowerCase().includes(searchQueryNormalized) ||
      n.tags?.some((t) => t.toLowerCase().includes(searchQueryNormalized)) ||
      (n.snippet && n.snippet.toLowerCase().includes(searchQueryNormalized))
    ) {
      matches.add(n.id);
    }
  }
  return matches;
}, [filteredNodes, searchQueryNormalized]);

const highlightedNodeIds = useMemo<Set<string> | null>(() => {
  if (!matchingNodeIds) return null;
  const connected = new Set<string>(matchingNodeIds);
  for (const edge of visibleEdges) {
    if (matchingNodeIds.has(edge.source)) connected.add(edge.target);
    if (matchingNodeIds.has(edge.target)) connected.add(edge.source);
  }
  return connected;
}, [matchingNodeIds, visibleEdges]);

// 7. Auto-Fit Viewport Algorithm (Zoom to Extents)
const fitToView = useCallback(() => {
  if (filteredNodes.length === 0 || !containerRef.current) return;
  const padding = 70;
  let minX = Infinity;
  let maxX = -Infinity;
  let minY = Infinity;
  let maxY = -Infinity;

  for (const n of filteredNodes) {
    if (n.x < minX) minX = n.x;
    if (n.x > maxX) maxX = n.x;
    if (n.y < minY) minY = n.y;
    if (n.y > maxY) maxY = n.y;
  }

  const graphWidth = Math.max(120, maxX - minX);
  const graphHeight = Math.max(120, maxY - minY);
  const availableWidth = Math.max(200, dims.width - padding * 2);
  const availableHeight = Math.max(200, dims.height - padding * 2);

  const targetZoom = Math.min(
```

```

    Math.max(Math.min(availableWidth / graphWidth, availableHeight / graphHeight), 0.35),
    2.2
  );

  const centerX = (minX + maxX) / 2;
  const centerY = (minY + maxY) / 2;
  const targetPanX = dims.width / 2 - centerX * targetZoom;
  const targetPanY = dims.height / 2 - centerY * targetZoom;

  setZoom(targetZoom);
  setPan({ x: targetPanX, y: targetPanY });
}, [filteredNodes, dims]);

// Auto-fit on initial modal open when nodes become available
const hasAutoFittedRef = useRef(false);
useEffect(() => {
  if (open && filteredNodes.length > 0 && !hasAutoFittedRef.current) {
    hasAutoFittedRef.current = true;
    const t = setTimeout(fitToView, 60);
    return () => clearTimeout(t);
  }
}, [open, filteredNodes.length, fitToView]);

// Native non-passive Wheel zoom centered around cursor
useEffect(() => {
  const el = containerRef.current;
  if (!el || !open) return;
  const onWheel = (e: WheelEvent) => {
    e.preventDefault();
    const factor = e.deltaY < 0 ? 1.15 : 0.85;
    const rect = el.getBoundingClientRect();
    const mouseX = e.clientX - rect.left;
    const mouseY = e.clientY - rect.top;

    setZoom((prevZoom) => {
      const nextZoom = Math.min(Math.max(prevZoom * factor, 0.25), 3.5);
      setPan((prevPan) => ({
        x: mouseX - ((mouseX - prevPan.x) / prevZoom) * nextZoom,
        y: mouseY - ((mouseY - prevPan.y) / prevZoom) * nextZoom,
      }));
      return nextZoom;
    });
  };
  el.addEventListener("wheel", onWheel, { passive: false });
  return () => el.removeEventListener("wheel", onWheel);
}, [open]);

// 8. Multi-Touch Gesture Handling (Pinch-to-Zoom & Dual-Pointer Pan)
const handleCanvasPointerDown = (e: React.PointerEvent) => {
  const target = e.target as HTMLElement;
  if (target.tagName === "svg" || target.id === "graph-canvas" || target.id === "graph-background") {
    activePointersRef.current.set(e.pointerId, {
      x: e.clientX,
      y: e.clientY,
      pointerType: e.pointerType,
    });

    if (activePointersRef.current.size === 1) {
      isPanningRef.current = true;
      canvasPanStartRef.current = { x: e.clientX - pan.x, y: e.clientY - pan.y };
      target.setPointerCapture?.(e.pointerId);
    } else if (activePointersRef.current.size === 2) {
      // Multi-touch pinch started!
      const pts = Array.from(activePointersRef.current.values());
      const dist = Math.hypot(pts[0].x - pts[1].x, pts[0].y - pts[1].y);
      const mid = { x: (pts[0].x + pts[1].x) / 2, y: (pts[0].y + pts[1].y) / 2 };
      pinchStartRef.current = { dist, zoom, mid, pan: { ...pan } };
    }
  }
};

const handleCanvasPointerMove = (e: React.PointerEvent) => {
  if (activePointersRef.current.has(e.pointerId)) {
    activePointersRef.current.set(e.pointerId, {
      x: e.clientX,
      y: e.clientY,
      pointerType: e.pointerType,
    });
  }
};

```

```

    }

    // 2-Finger Pinch to Zoom & Pan
    if (activePointersRef.current.size === 2 && pinchStartRef.current) {
      const pts = Array.from(activePointersRef.current.values());
      const currentDist = Math.hypot(pts[0].x - pts[1].x, pts[0].y - pts[1].y);
      const currentMid = { x: (pts[0].x + pts[1].x) / 2, y: (pts[0].y + pts[1].y) / 2 };

      const scale = currentDist / Math.max(1, pinchStartRef.current.dist);
      const newZoom = Math.min(Math.max(pinchStartRef.current.zoom * scale, 0.25), 3.5);

      const containerRect = containerRef.current?.getBoundingClientRect();
      const midRelX = currentMid.x - (containerRect?.left || 0);
      const midRelY = currentMid.y - (containerRect?.top || 0);

      const newPanX = midRelX - ((midRelX - pinchStartRef.current.pan.x) / pinchStartRef.current.zoom) * newZoom;
      const newPanY = midRelY - ((midRelY - pinchStartRef.current.pan.y) / pinchStartRef.current.zoom) * newZoom;

      setZoom(newZoom);
      setPan({ x: newPanX, y: newPanY });
      return;
    }

    // 1-Finger / Pen Canvas Pan
    if (isPanningRef.current && canvasPanStartRef.current && activePointersRef.current.size === 1) {
      setPan({
        x: e.clientX - canvasPanStartRef.current.x,
        y: e.clientY - canvasPanStartRef.current.y,
      });
      return;
    }

    // Node Drag
    if (dragNodeId && nodeDragStartRef.current && containerRef.current) {
      const travel = Math.hypot(
        e.clientX - nodeDragStartRef.current.x,
        e.clientY - nodeDragStartRef.current.y
      );
      if (travel > 6) {
        nodeHasDraggedRef.current = true;
      }
      const rect = containerRef.current.getBoundingClientRect();
      const svgX = (e.clientX - rect.left - pan.x) / zoom;
      const svgY = (e.clientY - rect.top - pan.y) / zoom;

      setNodePositions((prev) => ({
        ...prev,
        [dragNodeId]: { x: svgX, y: svgY },
      }));
    }
  };

  const handleCanvasPointerUp = (e: React.PointerEvent) => {
    activePointersRef.current.delete(e.pointerId);
    if (activePointersRef.current.size < 2) {
      pinchStartRef.current = null;
    }
    if (activePointersRef.current.size === 0) {
      isPanningRef.current = false;
      canvasPanStartRef.current = null;
    }
  };

  // 9. Node Interaction & Tap/Drag Disambiguation
  const handleNodePointerDown = (nodeId: string, e: React.PointerEvent) => {
    e.stopPropagation();
    nodeHasDraggedRef.current = false;
    nodeDragStartRef.current = { x: e.clientX, y: e.clientY };
    setDragNodeId(nodeId);
    e.currentTarget.setPointerCapture?.(e.pointerId);
  };

  const handleNodePointerUp = (nodeId: string) => {
    if (nodeHasDraggedRef.current) {
      // It was a drag: stop here, do not trigger tap or open
      setDragNodeId(null);
    }
  };

```

```

        nodeDragStartRef.current = null;
        return;
    }

    setDragNodeId(null);
    nodeDragStartRef.current = null;

    // It was a clean tap!
    const now = Date.now();
    if (lastTapRef.current && lastTapRef.current.id === nodeId && now - lastTapRef.current.time < 350) {
        // Double tap: immediately open board/note
        handleNodeOpen(nodeId);
    } else {
        // Single tap: select node to open Inspector Drawer
        setSelectedNodeId(nodeId);
        lastTapRef.current = { id: nodeId, time: now };
    }
};

const handleNodeOpen = (nodeId: string) => {
    const el = parseElementNodeId(nodeId);
    if (el) {
        focusElement(el.boardId, el.elementId);
        onClose();
        return;
    }
    if (boards.some((b) => b.id === nodeId)) {
        setActiveBoard(nodeId);
        onClose();
        return;
    }
    if (folders.some((f) => f.id === nodeId)) {
        revealFolder(nodeId);
        const firstBoard = boards.find((b) => b.folderId === nodeId);
        if (firstBoard) setActiveBoard(firstBoard.id);
        onClose();
        return;
    }
};

// Reheat / Jiggle layout to untangle clusters
const reheatLayout = () => {
    setNodePositions({});
    cachedPositionsRef.current.clear();
    setPhysicsEnabled(true);
};

// High-Resolution Export as SVG
const handleExportSVG = () => {
    if (!svgRef.current) return;
    const svgClone = svgRef.current.cloneNode(true) as SVGSVGElement;
    svgClone.setAttribute("xmlns", "http://www.w3.org/2000/svg");
    const svgData = new XMLSerializer().serializeToString(svgClone);
    const blob = new Blob([svgData], { type: "image/svg+xml;charset=utf-8" });
    const url = URL.createObjectURL(blob);
    const link = document.createElement("a");
    link.href = url;
    link.download = `pronotes-knowledge-graph-${Date.now()}.svg`;
    link.click();
    URL.revokeObjectURL(url);
};

// High-Resolution Export as 2x PNG
const handleExportPNG = () => {
    if (!svgRef.current || !containerRef.current) return;
    const svgClone = svgRef.current.cloneNode(true) as SVGSVGElement;
    const rect = containerRef.current.getBoundingClientRect();
    const scale = 2;
    const width = rect.width * scale;
    const height = rect.height * scale;

    const canvas = document.createElement("canvas");
    canvas.width = width;
    canvas.height = height;
    const ctx = canvas.getContext("2d");
    if (!ctx) return;

    ctx.fillStyle = document.documentElement.classList.contains("dark") ? "#09090b" : "#f8fafc";

```

```

    ctx.fillRect(0, 0, width, height);

    const svgData = new XMLSerializer().serializeToString(svgClone);
    const img = new Image();
    const blob = new Blob([svgData], { type: "image/svg+xml;charset=utf-8" });
    const url = URL.createObjectURL(blob);

    img.onload = () => {
      ctx.drawImage(img, 0, 0, width, height);
      URL.revokeObjectURL(url);
      const pngUrl = canvas.toDataURL("image/png");
      const link = document.createElement("a");
      link.href = pngUrl;
      link.download = `pronotes-knowledge-graph-${Date.now()}.png`;
      link.click();
    };
    img.src = url;
  };

  // Node Inspector Data
  const inspectedNode = useMemo(() => {
    if (!selectedNodeId) return null;
    return nodes.find((n) => n.id === selectedNodeId) || null;
  }, [selectedNodeId, nodes]);

  const inspectedInboundEdges = useMemo(() => {
    if (!selectedNodeId) return [];
    return visibleEdges.filter((e) => e.target === selectedNodeId);
  }, [selectedNodeId, visibleEdges]);

  const inspectedOutboundEdges = useMemo(() => {
    if (!selectedNodeId) return [];
    return visibleEdges.filter((e) => e.source === selectedNodeId);
  }, [selectedNodeId, visibleEdges]);

  if (!open) return null;

  return (
    <div
      data-wb-modal
      className="fixed inset-0 z-50 flex items-center justify-center bg-black/60 backdrop-blur-md p-3 sm:p-5
        elect-none animate-in fade-in"
      onClick={(e) => {
        if (e.target === e.currentTarget) {
          onClose();
        }
      }}
    >
      <div
        ref={containerRef}
        style={{ touchAction: "none" }}
        className="relative w-full max-w-7xl h-[92vh] bg-white dark:bg-zinc-900 rounded-2xl shadow-2xl border
          border-border overflow-hidden flex flex-col"
      >
        {/* Top Header & Toolbar */}
        <div className="flex flex-wrap items-center justify-between gap-3 px-5 py-3.5 border-b border-border b
          g-white/95 dark:bg-zinc-900/95 backdrop-blur-md z-30">
          <div className="flex items-center gap-3">
            <div className="p-2 rounded-xl bg-primary/10 text-primary">
              <Network size={22} />
            </div>
            <div>
              <div className="flex items-center gap-2.5">
                <h2 className="text-base sm:text-lg font-bold text-foreground">
                  Interactive Knowledge Graph
                </h2>
                <span className="text-xs px-2.5 py-0.5 rounded-full bg-primary/10 text-primary font-semibold t
                  abular-nums">
                  {filteredNodes.length} nodes · {visibleEdges.length} connections
                </span>
              </div>
            </div>
            <p className="text-[11px] text-muted-foreground hidden sm:block">
              Tap node to inspect • Double-tap to open • Pinch to zoom • Drag to organize
            </p>
          </div>
        </div>

        {/* Center Mode Controls: Global vs Local Neighborhood */}

```

```

<div className="flex items-center gap-1.5 p-1 rounded-xl bg-muted/60 border border-border/60">
  <button
    onClick={() => setGraphMode("global")}
    className={`px-3 py-1 text-xs font-medium rounded-lg transition-all ${
      graphMode === "global"
        ? "bg-white dark:bg-zinc-800 text-foreground shadow-xs font-semibold"
        : "text-muted-foreground hover:text-foreground"
    }`}
  >
    Universe
  </button>
  <button
    onClick={() => {
      setGraphMode("local");
      if (!localFocusId && activeBoardId) setLocalFocusId(activeBoardId);
    }}
    className={`px-3 py-1 text-xs font-medium rounded-lg transition-all ${
      graphMode === "local"
        ? "bg-white dark:bg-zinc-800 text-foreground shadow-xs font-semibold"
        : "text-muted-foreground hover:text-foreground"
    }`}
  >
    Local Focus
  </button>
  {graphMode === "local" && (
    <div className="flex items-center gap-1 pl-1 border-l border-border">
      {[1, 2, 3].map((d) => (
        <button
          key={d}
          onClick={() => setLocalDepth(d)}
          className={`px-1.5 py-0.5 text-[11px] rounded ${
            localDepth === d ? "bg-primary text-primary-foreground font-bold" : "text-muted-foreground"
          }`}
        >
          {d}H
        </button>
      ))}
    </div>
  )}
</div>

{/* Right Action Tools: Search, Filter Popover, Export, Close */}
<div className="flex items-center gap-2">
  {/* Search Input */}
  <div className="relative">
    <Search size={14} className="absolute left-3 top-1/2 -translate-y-1/2 text-muted-foreground" />
    <Input
      value={searchQuery}
      onChange={(e) => setSearchQuery(e.target.value)}
      placeholder="Search graph..."
      className="h-8 w-44 sm:w-56 pl-8 text-xs rounded-xl"
    />
    {matchingNodeIds && (
      <span className="absolute right-2.5 top-1/2 -translate-y-1/2 text-[10px] font-bold px-1.5 py-0.2 rounded bg-primary/20 text-primary">
        {matchingNodeIds.size}
      </span>
    )}
  </div>

  {/* Filter Toggle Menu Button */}
  <div className="relative">
    <Button
      variant={filterMenuOpen ? "secondary" : "outline"}
      size="sm"
      onClick={() => setFilterMenuOpen((v) => !v)}
      className="h-8 gap-1.5 rounded-xl text-xs px-2.5"
      title="Filter elements and nodes"
    >
      <Filter size={14} />
      <span className="hidden md:inline">Filters</span>
    </Button>

    {filterMenuOpen && (
      <div className="absolute right-0 top-10 w-52 p-2.5 bg-white dark:bg-zinc-900 border border-border shadow-2xl rounded-xl z-50 text-xs space-y-2 animate-in fade-in zoom-in-95">
        <div className="font-semibold text-foreground px-1 pb-1 border-b border-border text-[11px] u

```

```

    ppercase tracking-wider">
      Node Filters
    </div>
    <label className="flex items-center justify-between px-1.5 py-1 rounded-md hover:bg-muted cursor-pointer">
      <span className="flex items-center gap-2">
        <FolderIcon size={14} className="text-amber-500" /> Folders
      </span>
      <input
        type="checkbox"
        checked={showFolders}
        onChange={(e) => setShowFolders(e.target.checked)}
        className="rounded accent-primary"
      />
    </label>
    <label className="flex items-center justify-between px-1.5 py-1 rounded-md hover:bg-muted cursor-pointer">
      <span className="flex items-center gap-2">
        <FileText size={14} className="text-teal-500" /> Note Cards
      </span>
      <input
        type="checkbox"
        checked={showElements}
        onChange={(e) => setShowElements(e.target.checked)}
        className="rounded accent-primary"
      />
    </label>
    <label className="flex items-center justify-between px-1.5 py-1 rounded-md hover:bg-muted cursor-pointer">
      <span className="flex items-center gap-2">
        <TagIcon size={14} className="text-emerald-500" /> Tags (#tag)
      </span>
      <input
        type="checkbox"
        checked={showTags}
        onChange={(e) => setShowTags(e.target.checked)}
        className="rounded accent-primary"
      />
    </label>
    <label className="flex items-center justify-between px-1.5 py-1 rounded-md hover:bg-muted cursor-pointer">
      <span className="flex items-center gap-2">
        <Layers size={14} className="text-zinc-400" /> Orphan Notes
      </span>
      <input
        type="checkbox"
        checked={showOrphans}
        onChange={(e) => setShowOrphans(e.target.checked)}
        className="rounded accent-primary"
      />
    </label>
  </div>
)}
</div>

{/* Export Dropdown */}
<Button
  variant="outline"
  size="sm"
  onClick={handleExportPNG}
  className="h-8 gap-1 rounded-xl text-xs px-2.5"
  title="Export high-resolution graph image"
>
  <Download size={14} />
  <span className="hidden sm:inline">PNG</span>
</Button>
<Button
  variant="outline"
  size="sm"
  onClick={handleExportSVG}
  className="h-8 gap-1 rounded-xl text-xs px-2.5 hidden md:flex"
  title="Export scalable vector graphics"
>
  <Download size={14} />
  <span>SVG</span>
</Button>

{/* Close Button */}

```

```

    <Button variant="ghost" size="icon" onClick={onClose} className="h-8 w-8 rounded-xl">
      <X size={18} />
    </Button>
  </div>
</div>

{/* SVG Canvas Area */}
<div
  id="graph-canvas"
  style={{ touchAction: "none" }}
  onPointerDown={handleCanvasPointerDown}
  onPointerMove={handleCanvasPointerMove}
  onPointerUp={handleCanvasPointerUp}
  onPointerCancel={handleCanvasPointerUp}
  className="flex-1 w-full h-full relative overflow-hidden bg-slate-50/70 dark:bg-zinc-950/70 cursor-grab active:cursor-grabbing"
>
  <svg
    ref={svgRef}
    style={{ touchAction: "none" }}
    className="w-full h-full"
  >
    <rect id="graph-background" width="100%" height="100%" fill="transparent" />

    <g transform={`translate(${pan.x}, ${pan.y}) scale(${zoom})`}>
      {/* Edges */}
      {visibleEdges.map((edge) => {
        const a = filteredNodes.find((n) => n.id === edge.source);
        const b = filteredNodes.find((n) => n.id === edge.target);
        if (!a || !b) return null;

        const isParent = edge.kind === "parent";
        const isTagEdge = edge.kind === "tag";
        const isHighlighted =
          hoveredNode === a.id ||
          hoveredNode === b.id ||
          selectedNodeId === a.id ||
          selectedNodeId === b.id;

        const isSearchDimmed =
          matchingNodeIds !== null &&
          !matchingNodeIds.has(a.id) &&
          !matchingNodeIds.has(b.id);

        const edgeColor = edge.color || (isTagEdge ? "#10b981" : isParent ? "#9ca3af" : "#6366f1");

        const radiusA = (a.kind === "element" ? 7 : a.kind === "tag" ? 9 : 9) + Math.min(16, a.degree * 2.5);
        const radiusB = (b.kind === "element" ? 7 : b.kind === "tag" ? 9 : 9) + Math.min(16, b.degree * 2.5);
        const angle = Math.atan2(b.y - a.y, b.x - a.x);

        const startX = a.x + Math.cos(angle) * radiusA;
        const startY = a.y + Math.sin(angle) * radiusA;
        const endX = b.x - Math.cos(angle) * radiusB;
        const endY = b.y - Math.sin(angle) * radiusB;

        const midX = (startX + endX) / 2;
        const midY = (startY + endY) / 2;

        const arrowSize = isParent ? 6 : isTagEdge ? 7 : 9;
        const back1X = endX - Math.cos(angle - 0.38) * arrowSize;
        const back1Y = endY - Math.sin(angle - 0.38) * arrowSize;
        const back2X = endX - Math.cos(angle + 0.38) * arrowSize;
        const back2Y = endY - Math.sin(angle + 0.38) * arrowSize;

        return (
          <g key={edge.id} opacity={isSearchDimmed ? 0.12 : 1}>
            <line
              x1={startX}
              y1={startY}
              x2={endX}
              y2={endY}
              stroke={isHighlighted ? "#6366f1" : edgeColor}
              strokeWidth={isHighlighted ? 3 : isParent ? 1.4 : isTagEdge ? 1.6 : 2}
              strokeDasharray={isParent ? "5,4" : isTagEdge ? "3,3" : undefined}
              opacity={isHighlighted ? 1 : 0.75}
            />
          </g>
        );
      })}
    </g>
  </svg>

```

```

    <polygon
      points={`${endX},${endY} ${back1X},${back1Y} ${back2X},${back2Y}`}
      fill={isHighlighted ? "#6366f1" : edgeColor}
      opacity={isHighlighted ? 1 : 0.85}
    />
    {edge.label && !isParent && zoom > 0.65 && (
      <g transform={`translate(${midX}, ${midY})`}>
        <rect
          x={`-(edge.label.length * 3.4 + 7)`}
          y={`-8.5`}
          width={edge.label.length * 6.8 + 14}
          height={17}
          rx={8.5}
          fill="white"
          className="dark:fill-zinc-900 shadow-xs"
          stroke={edgeColor}
          strokeWidth={1}
        />
        <text
          textAnchor="middle"
          y={3.5}
          fontSize={9}
          fontWeight={600}
          fill={edgeColor}
          style={{ pointerEvents: "none", userSelect: "none" }}
        >
          {edge.label}
        </text>
      </g>
    )}
  </g>
);
}}}

{/* Nodes */}
{filteredNodes.map((node) => {
  const isActive = node.id === activeBoardId;
  const isSelected = selectedNodeId === node.id;
  const isHovered = hoveredNode === node.id;
  const isFolder = node.kind === "folder";
  const isTag = node.kind === "tag";
  const isElement = node.kind === "element";

  const isDirectMatch = matchingNodeIds?.has(node.id);
  const isDimmed = matchingNodeIds !== null && !highlightedNodeIds?.has(node.id);

  const radius = (isElement ? 7 : isTag ? 8 : 10) + Math.min(isElement ? 8 : 18, node.degree * 2
    .5);

  const fill = isTag
    ? "#10b981"
    : isElement
    ? "#14b8a6"
    : isFolder
    ? node.color || "#f59e0b"
    : isActive
    ? "#6366f1"
    : node.starred
    ? "#f59e0b"
    : node.orphan
    ? "#94a3b8"
    : node.color || "#3b82f6";

  return (
    <g
      key={node.id}
      transform={`translate(${node.x}, ${node.y})`}
      opacity={isDimmed ? 0.2 : 1}
      style={{ cursor: "pointer", touchAction: "none" }}
      onPointerDown={(e) => handleNodePointerDown(node.id, e)}
      onPointerUp={() => handleNodePointerUp(node.id)}
      onPointerEnter={(e) => {
        if (e.pointerType === "pen" || e.pointerType === "mouse") {
          setHoveredNode(node.id);
          setHoverAnchor({ x: node.x, y: node.y });
        }
      }}
      onPointerLeave={(e) => {

```

```

        if (e.pointerType === "pen" || e.pointerType === "mouse") {
            if (hoveredNode === node.id) setHoveredNode(null);
        }
    }
}
>
{ /* Search match illuminated halo */
{isDirectMatch && (
    <circle
        r={radius + 8}
        fill="none"
        stroke="#ec4899"
        strokeWidth={3}
        className="animate-pulse"
    />
)}

{ /* Node Geometry */
{isTag ? (
    <g
        fill={fill}
        stroke={isSelected ? "#ec4899" : isActive ? "#4338ca" : "white"}
        strokeWidth={isSelected ? 3.5 : 2}
        style={{
            filter: isHovered || isSelected
                ? "drop-shadow(0 0 12px rgba(16,185,129,0.75))"
                : "drop-shadow(0 2px 4px rgba(0,0,0,0.15))",
            transition: "filter 0.15s",
        }}
    >
        {elementNodeShape("tag", radius * 0.9)}
    </g>
) : isElement ? (
    <g
        fill={fill}
        stroke={isSelected ? "#ec4899" : isActive ? "#4338ca" : "white"}
        strokeWidth={node.elementType === "stroke" ? 2.5 : isSelected ? 3.5 : 2}
        style={{
            filter: isHovered || isSelected
                ? "drop-shadow(0 0 12px rgba(99,102,241,0.7))"
                : "drop-shadow(0 2px 4px rgba(0,0,0,0.15))",
            transition: "filter 0.15s",
        }}
    >
        {elementNodeShape(node.elementType, radius * 0.82)}
    </g>
) : isFolder ? (
    <rect
        x={-radius}
        y={-radius}
        width={radius * 2}
        height={radius * 2}
        rx={6}
        fill={fill}
        stroke={isSelected ? "#ec4899" : isActive ? "#4338ca" : "white"}
        strokeWidth={isSelected || isActive ? 3.5 : 2}
        style={{
            filter: isHovered || isSelected
                ? "drop-shadow(0 0 12px rgba(245,158,11,0.7))"
                : "drop-shadow(0 2px 4px rgba(0,0,0,0.15))",
            transition: "filter 0.15s",
        }}
    />
) : (
    <circle
        r={radius}
        fill={fill}
        stroke={isSelected ? "#ec4899" : isActive ? "#4338ca" : "white"}
        strokeWidth={isSelected || isActive ? 3.5 : 2}
        style={{
            filter: isHovered || isSelected
                ? "drop-shadow(0 0 12px rgba(99,102,241,0.7))"
                : "drop-shadow(0 2px 4px rgba(0,0,0,0.15))",
            transition: "filter 0.15s",
        }}
    />
)}

{ /* Starred badge */

```

```

        {node.starred && (
          <Star
            size={11}
            className="text-white fill-white absolute"
            style={{ transform: "translate(-50%, -50%)" }}
          />
        )}

        {/* Pinned badge */}
        {node.isPinned && (
          <Pin
            size={10}
            className="text-white fill-white absolute"
            style={{ transform: "translate(-50%, -50%)" }}
          />
        )}

        {/* Node Text Label */}
        <text
          y={radius + 14}
          textAnchor="middle"
          fontSize={11.5}
          className="fill-zinc-900 dark:fill-zinc-100"
          fontWeight={isActive || isHovered || isSelected ? 700 : 500}
          style={{ pointerEvents: "none", userSelect: "none" }}
        >
          {node.name.length > 24 ? node.name.slice(0, 22) + "..." : node.name}
        </text>
      </g>
    );
  }
}
</g>
</svg>

{/* S Pen Stylus Hover Peek Card */}
{hoveredNode && hoverAnchor && (
  <div
    className="absolute pointer-events-none z-40 bg-white/95 dark:bg-zinc-900/95 backdrop-blur-md rounded-xl border border-border p-3 shadow-2xl text-xs max-w-xs animate-in fade-in"
    style={{
      left: Math.min(dims.width - 240, Math.max(20, hoverAnchor.x * zoom + pan.x + 20)),
      top: Math.min(dims.height - 140, Math.max(20, hoverAnchor.y * zoom + pan.y - 40)),
    }}
  >
    {(() => {
      const n = filteredNodes.find((x) => x.id === hoveredNode);
      if (!n) return null;
      return (
        <div className="space-y-1.5">
          <div className="flex items-center gap-1.5">
            <span className="text-[10px] uppercase font-bold px-1.5 py-0.5 rounded bg-primary/10 text-primary">
              {n.kind}
            </span>
            <span className="font-bold text-foreground truncate">{n.name}</span>
          </div>
          {n.snippet && (
            <div className="text-[11px] text-muted-foreground line-clamp-2">
              {n.snippet}
            </div>
          )}
          <div className="text-[10px] text-muted-foreground pt-1 border-t border-border flex items-center justify-between">
            <span>{n.degree} connections</span>
            {n.folderName && <span>■ {n.folderName}</span>}
          </div>
        </div>
      );
    })()}
  </div>
)}

{/* Floating Radar Minimap (Bottom-Left) */}
<div className="absolute bottom-5 left-5 hidden sm:block p-1.5 bg-white/90 dark:bg-zinc-900/90 backdrop-blur-md shadow-xl border border-border rounded-xl z-20">
  <div className="text-[10px] font-bold text-muted-foreground px-1 pb-1 flex items-center justify-between">
    <span>Radar</span>
  </div>
</div>

```

```

    <span className="text-[9px] font-mono">{Math.round(zoom * 100)}%</span>
  </div>
  <svg
    width={120}
    height={80}
    className="rounded-lg bg-slate-100 dark:bg-zinc-950 border border-border/50 cursor-pointer"
    onClick={(e) => {
      const rect = e.currentTarget.getBoundingClientRect();
      const clickNormX = (e.clientX - rect.left) / rect.width;
      const clickNormY = (e.clientY - rect.top) / rect.height;
      setPan({
        x: dims.width / 2 - clickNormX * dims.width * zoom,
        y: dims.height / 2 - clickNormY * dims.height * zoom,
      });
    }}
  >
    {filteredNodes.map((n) => {
      const mx = (n.x / Math.max(1, dims.width)) * 120;
      const my = (n.y / Math.max(1, dims.height)) * 80;
      return (
        <circle
          key={n.id}
          cx={Math.max(3, Math.min(117, mx))}
          cy={Math.max(3, Math.min(77, my))}
          r={2}
          fill={n.id === selectedNodeId ? "#ec4899" : n.color || "#6366f1"}
        />
      );
    })}
    { /* Viewport indicator box */
      (() => {
        const vx = (-pan.x / (dims.width * zoom)) * 120;
        const vy = (-pan.y / (dims.height * zoom)) * 80;
        const vw = (1 / zoom) * 120;
        const vh = (1 / zoom) * 80;
        return (
          <rect
            x={Math.max(0, vx)}
            y={Math.max(0, vy)}
            width={Math.min(120, vw)}
            height={Math.min(80, vh)}
            fill="rgba(99, 102, 241, 0.15)"
            stroke="#6366f1"
            strokeWidth={1}
            rx={2}
          />
        );
      })()
    }
  </svg>
</div>

{ /* Floating Navigation & Physics Controls (Bottom-Right) */ }
<div className="absolute bottom-5 right-5 flex items-center gap-1.5 p-1.5 bg-white/95 dark:bg-zinc-900/95 backdrop-blur-md shadow-xl border border-border rounded-xl z-20">
  <button
    onClick={() => setPhysicsEnabled((v) => !v)}
    className={`p-1.5 rounded-lg transition-colors ${
      physicsEnabled
        ? "bg-primary/10 text-primary font-bold"
        : "text-muted-foreground hover:bg-muted"
    }`}
    title={physicsEnabled ? "Pause physics" : "Enable physics"}
  >
    {physicsEnabled ? <Pause size={15} /> : <Play size={15} />}
  </button>
  <button
    onClick={reheatLayout}
    className="p-1.5 rounded-lg hover:bg-muted text-muted-foreground hover:text-foreground"
    title="Reheat and jiggle layout"
  >
    <Sparkles size={15} />
  </button>
  <div className="w-px h-4 bg-border mx-0.5" />
  <button
    onClick={() => setZoom((z) => Math.max(0.25, z * 0.85))}
    className="p-1.5 rounded-lg hover:bg-muted text-muted-foreground hover:text-foreground"
    title="Zoom out"
  >

```

```

        <ZoomOut size={15} />
    </button>
    <span className="text-xs font-semibold px-1.5 tabular-nums min-w-[42px] text-center">
        {Math.round(zoom * 100)}%
    </span>
    <button
        onClick={() => setZoom((z) => Math.min(3.5, z * 1.15))}
        className="p-1.5 rounded-lg hover:bg-muted text-muted-foreground hover:text-foreground"
        title="Zoom in"
    >
        <ZoomIn size={15} />
    </button>
    <div className="w-px h-4 bg-border mx-0.5" />
    <button
        onClick={fitToView}
        className="p-1.5 rounded-lg hover:bg-muted text-muted-foreground hover:text-foreground"
        title="Fit to view (Zoom to extents)"
    >
        <Maximize2 size={15} />
    </button>
    <button
        onClick={() => {
            setZoom(1);
            setPan({ x: 0, y: 0 });
            setNodePositions({});
        }}
        className="p-1.5 rounded-lg hover:bg-muted text-muted-foreground hover:text-foreground"
        title="Reset camera"
    >
        <RotateCcw size={15} />
    </button>
</div>
</div>

{/* Node Inspector Drawer (Slide-out Right Panel) */}
{inspectedNode && (
    <div className="absolute top-14 right-4 bottom-4 w-80 bg-white/95 dark:bg-zinc-900/95 backdrop-blur-
md rounded-2xl shadow-2xl border border-border p-4 flex flex-col z-40 animate-in slide-in-from-r
ight-4 duration-200">
        <div className="flex items-center justify-between pb-3 border-b border-border">
            <div className="flex items-center gap-2">
                <span className="text-[10px] font-bold uppercase px-2 py-0.5 rounded-full bg-primary/10 text-p
rimary">
                    {inspectedNode.kind}
                </span>
                {inspectedNode.folderName && (
                    <span className="text-xs text-muted-foreground truncate max-w-[140px]">
                        ■ {inspectedNode.folderName}
                    </span>
                )}
            </div>
            <Button
                variant="ghost"
                size="icon"
                onClick={() => setSelectedNodeId(null)}
                className="h-7 w-7 rounded-lg"
            >
                <X size={15} />
            </Button>
        </div>

        <div className="flex-1 overflow-y-auto py-3 space-y-3.5 pr-1 text-xs">
            <div>
                <h3 className="text-base font-bold text-foreground leading-snug">
                    {inspectedNode.name}
                </h3>
                <div className="text-[11px] text-muted-foreground mt-0.5">
                    {inspectedNode.degree} active connections
                </div>
            </div>
        </div>

        {/* Snippet Preview */}
        {inspectedNode.snippet && (
            <div className="p-3 rounded-xl bg-muted/50 border border-border/80 font-mono text-[11px] text-
foreground/90 leading-relaxed max-h-28 overflow-y-auto">
                {inspectedNode.snippet}
            </div>
        )}
    </div>
)}

```

```

    { /* Tags Badges */
    {inspectedNode.tags && inspectedNode.tags.length > 0 && (
      <div>
        <div className="text-[11px] font-semibold text-muted-foreground mb-1.5 flex items-center gap
        -1">
          <Hash size={12} /> Tags
        </div>
        <div className="flex flex-wrap gap-1">
          {inspectedNode.tags.map((tag) => (
            <span
              key={tag}
              onClick={() => setSearchQuery(tag)}
              className="px-2 py-0.5 rounded-md bg-emerald-500/10 text-emerald-600 dark:text-emerald
              -400 font-semibold text-[10px] cursor-pointer hover:bg-emerald-500/20"
            >
              #{tag}
            </span>
          ))}
        </div>
      </div>
    )}

    { /* Inbound Backlinks */
    <div>
      <div className="text-[11px] font-semibold text-muted-foreground mb-1 flex items-center gap-1">
        <ArrowDownLeft size={13} className="text-indigo-500" /> Incoming Backlinks ({inspectedInbound
        dEdges.length})
      </div>
      {inspectedInboundEdges.length === 0 ? (
        <div className="text-[11px] text-muted-foreground italic pl-1">No incoming links</div>
      ) : (
        <div className="space-y-1">
          {inspectedInboundEdges.map((e) => {
            const src = nodes.find((n) => n.id === e.source);
            if (!src) return null;
            return (
              <button
                key={e.id}
                onClick={() => setSelectedNodeId(src.id)}
                className="w-full text-left p-1.5 rounded-lg hover:bg-muted flex items-center justif
                y-between text-[11px] transition-colors"
              >
                <span className="font-medium text-foreground truncate">{src.name}</span>
                <span className="text-[10px] text-muted-foreground">{e.label || "link"}</span>
              </button>
            );
          })}
        </div>
      )}
    </div>

    { /* Outbound Connections */
    <div>
      <div className="text-[11px] font-semibold text-muted-foreground mb-1 flex items-center gap-1">
        <ArrowUpRight size={13} className="text-teal-500" /> Outgoing Links ({inspectedOutboundEdges
        .length})
      </div>
      {inspectedOutboundEdges.length === 0 ? (
        <div className="text-[11px] text-muted-foreground italic pl-1">No outgoing links</div>
      ) : (
        <div className="space-y-1">
          {inspectedOutboundEdges.map((e) => {
            const tgt = nodes.find((n) => n.id === e.target);
            if (!tgt) return null;
            return (
              <button
                key={e.id}
                onClick={() => setSelectedNodeId(tgt.id)}
                className="w-full text-left p-1.5 rounded-lg hover:bg-muted flex items-center justif
                y-between text-[11px] transition-colors"
              >
                <span className="font-medium text-foreground truncate">{tgt.name}</span>
                <span className="text-[10px] px-1.5 py-0.2 rounded bg-primary/10 text-primary font-s
                emibold">
                  {e.label || e.linkType || "leads to"}
                </span>
              </button>
            );
          })}
        </div>
      )}
    </div>
  }

```

```

    });
  }}
</div>
  })
</div>
</div>

{/* Bottom Actions */}
<div className="pt-3 border-t border-border space-y-2">
  <Button
    size="sm"
    className="w-full gap-2 rounded-xl text-xs font-semibold"
    onClick={() => handleNodeOpen(inspectedNode.id)}
  >
    <ExternalLink size={14} /> Open Note / Board
  </Button>
  <div className="flex gap-2">
    <Button
      variant="outline"
      size="sm"
      className="flex-1 gap-1.5 rounded-xl text-xs"
      onClick={() => {
        setLocalFocusId(inspectedNode.id);
        setGraphMode("local");
      }}
    >
      <Compass size={13} /> Focus Here
    </Button>
    <Button
      variant={inspectedNode.isPinned ? "secondary" : "outline"}
      size="sm"
      className="gap-1.5 rounded-xl text-xs px-3"
      onClick={() => {
        setPinnedNodeIds((prev) => {
          const next = new Set(prev);
          if (next.has(inspectedNode.id)) {
            next.delete(inspectedNode.id);
          } else {
            next.add(inspectedNode.id);
          }
          return next;
        });
      }}
      title={inspectedNode.isPinned ? "Unpin position" : "Pin position"}
    >
      <Pin size={13} className={inspectedNode.isPinned ? "fill-primary text-primary" : ""} />
    </Button>
  </div>
</div>
</div>
  })
</div>
</div>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/GraphView.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/ImageActions.tsx ===
// File 54 of 194 | Lines: 146 | Size: 5960 bytes
=====

```

```
"use client";
```

```

/**
 * The bar that floats over ONE selected picture.
 *
 * The quick things happen right here, one tap each: turn a quarter, mirror,
 * swap the photo for another, lock it in place. Anything that needs a closer
 * look – cropping, a look, a frame – opens the editor (ImageStudio).
 * Resizing is the selection box's own corner handles, as for everything else.
 */

```

```
import React from "react";
```

```

import { Crop, FlipHorizontal2, Lock, Replace, RotateCw, Undo2 } from "lucide-react";
import { useWhiteboard, useActiveBoard } from "@store/whiteboard";
import { getBoardElements } from "@lib/whiteboard/notebook";
import { getElementBounds, type Viewport } from "@lib/whiteboard/geometry";
import type { ImageElement } from "@lib/whiteboard/types";
import { FULL_CROP, boxForCrop, flipCrop, isUnedited, normalizeCrop, quarterTurn } from "@lib/whiteboard/imageEdit";
import { prepareImage } from "@lib/whiteboard/imageImport";
import { storeImage } from "@lib/whiteboard/imagestore";
import { cn } from "@lib/utils";

export function ImageActions({ viewport }: { viewport: Viewport }) {
  const selectedIds = useWhiteboard((s) => s.selectedIds);
  const camera = useWhiteboard((s) => s.camera);
  const lockedMode = useWhiteboard((s) => s.lockedMode);
  const studioOpen = useWhiteboard((s) => s.imageStudioId !== null);
  const board = useActiveBoard();
  const replaceRef = React.useRef<HTMLInputElement>(null);

  if (selectedIds.length !== 1 || !board || lockedMode || studioOpen) return null;
  const found = getBoardElements(board).find((e) => e.id === selectedIds[0]);
  if (!found || found.type !== "image" || found.locked) return null;
  const el = found as ImageElement;

  const store = () => useWhiteboard.getState();
  const patch = (p: Partial<ImageElement>) => {
    store().commitHistory();
    store().updateElement(el.id, p, false);
  };

  const onReplace = async (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0];
    e.target.value = "";
    if (!file) return;
    const dataUrl = await prepareImage(file);
    const img = new Image();
    img.onload = async () => {
      const src = await storeImage(dataUrl);
      const nw = img.naturalWidth || el.naturalWidth;
      const nh = img.naturalHeight || el.naturalHeight;
      // Same width and place; the height follows the new photo's shape. The
      // old edits described the OLD photo, so they go with it.
      const height = (el.width * nh) / nw;
      patch({
        src,
        naturalWidth: nw,
        naturalHeight: nh,
        height,
        y: el.y + (el.height - height) / 2,
        crop: undefined,
        flipX: undefined,
        flipY: undefined,
      });
    };
    img.src = dataUrl;
  };

  const b = getElementBounds(el);
  const sx = (b.x - camera.x) * camera.zoom + viewport.width / 2;
  const sy = (b.y - camera.y) * camera.zoom + viewport.height / 2;
  const sw = b.width * camera.zoom;
  const sh = b.height * camera.zoom;
  const left = Math.min(Math.max(sx + sw / 2, 150), viewport.width - 150);
  // BELOW the picture: the rotate handle sits above it, and a bar there
  // covered it. With no room below (a tall picture), above the handle; failing
  // both, inside the bottom edge.
  const below = sy + sh + 16;
  const above = sy - 96;
  const top = below + 48 < viewport.height - 80 ? below : above > 8 ? above : Math.min(sy + sh - 56, viewport.height - 140);

  const btn =
    "flex h-9 items-center gap-1.5 rounded-full px-2.5 text-[12px] font-medium text-zinc-200 transition hover:bg-white/10 active:scale-95";

  return (
    <div
      className="absolute z-40 flex -translate-x-1/2 items-center gap-0.5 rounded-full border border-white/10

```

```

    bg-zinc-900/95 p-1 shadow-2xl backdrop-blur-md animate-in fade-in zoom-in-95"
    style={{ left, top }}
    onPointerDown={(e) => e.stopPropagation()}
    onMouseDown={(e) => e.stopPropagation()}
  >
  <input ref={replaceRef} type="file" accept="image/*" className="hidden" onChange={onReplace} />
  <button
    onClick={() => store().setImageStudioId(el.id)}
    className="flex h-9 items-center gap-1.5 rounded-full bg-indigo-500 px-3.5 text-[12px] font-semibold text-white shadow-md shadow-indigo-500/30 transition hover:bg-indigo-400 active:scale-95"
  >
    <Crop size={15} /> Edit
  </button>
  <button className={btn} title="Turn a quarter" aria-label="Turn a quarter" onClick={() => patch({ rotation: quarterTurn(el.rotation, 1) })}>
    <RotateCw size={16} />
  </button>
  <button
    className={cn(btn, el.flipX && "text-indigo-300")}
    title="Mirror"
    aria-label="Mirror"
    onClick={() => patch({ flipX: !el.flipX || undefined, crop: el.crop ? flipCrop(normalizeCrop(el.crop), "x") : undefined })}
  >
    <FlipHorizontal2 size={16} />
  </button>
  <button className={btn} title="Replace with another picture" aria-label="Replace" onClick={() => replaceRef.current?.click()}>
    <Replace size={16} />
  </button>
  {!isUnedited(el) && (
    <button
      className={btn}
      title="Back to the original photo"
      onClick={() =>
        patch({
          crop: undefined,
          flipX: undefined,
          flipY: undefined,
          filter: undefined,
          frame: undefined,
          borderWidth: undefined,
          // Uncropped again, at the same scale: the whole photo comes back.
          ...boxForCrop(el, FULL_CROP),
        })
      }
    >
      <Undo2 size={16} />
    </button>
  )}
  <span className="mx-0.5 h-5 w-px bg-white/15" />
  <button
    className={btn}
    title="Lock in place (unlock from Select → Locked items)"
    onClick={() => {
      store().toggleLock([el.id]);
      store().clearSelection();
    }}
  >
    <Lock size={15} /> Lock
  </button>
</div>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/ImageActions.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/ImageInsertSheet.tsx ===
// File 55 of 194 | Lines: 472 | Size: 19106 bytes
=====

```

```

"use client";

```

```

/**

```

```

* "Add a picture" — the toolbar's image button.
*
* One tap used to open the system file picker directly, with no say in where
* the picture came from and no word on what happens to it. It now opens this
* sheet: pick the SOURCE (take a photo, pick from photos, browse files, paste),
* choose whether the picture goes ON the page or becomes a page of its own,
* or reuse a picture already in your notes.
*
* Privacy: before ProNotes opens the camera or your photos for the first time
* it says, in plain words, what it will and won't do, and asks. The answer is
* remembered on this device and can be withdrawn from the same sheet. (The
* camera itself is the system camera app, which needs no extra Android
* permission — so the only "ask" is this honest one.)
*/

import React from "react";
import {
  Camera,
  ClipboardPaste,
  FileImage,
  FilePlus2,
  Images,
  Image as ImageIcon,
  Loader2,
  ShieldCheck,
  StickyNote,
  X,
} from "lucide-react";
import { useWhiteboard } from "@store/whiteboard";
import { isNotebook } from "@lib/whiteboard/notebook";
import { listRecentImages, storeImage, ASSET_PREFIX } from "@lib/whiteboard/imagestore";
import { isImageFile, prepareImage } from "@lib/whiteboard/imageImport";
import { appClipForPaste, getAppClipImage } from "@lib/whiteboard/appClipboard";
import { Popover, PopoverContent, PopoverTrigger } from "@components/ui/popover";
import { Tooltip, TooltipContent, TooltipTrigger } from "@components/ui/tooltip";
import { cn } from "@lib/utils";

type Source = "camera" | "photos" | "files";
type Access = "camera" | "media";
type PlaceAs = "picture" | "page";

const CONSENT_KEY = (a: Access) => `pronotes:consent:${a}`;
const PLACE_KEY = "pronotes:imagePlaceAs";

function readConsent(a: Access): boolean {
  try {
    return localStorage.getItem(CONSENT_KEY(a)) === "yes";
  } catch {
    return false;
  }
}

function writeConsent(a: Access, yes: boolean) {
  try {
    if (yes) localStorage.setItem(CONSENT_KEY(a), "yes");
    else localStorage.removeItem(CONSENT_KEY(a));
  } catch {
    /* a remembered answer is a convenience: without storage we simply ask again */
  }
}

const accessOf = (s: Source): Access => (s === "camera" ? "camera" : "media");

const CONSENT_COPY: Record<Access, { title: string; points: string[]; allow: string }> = {
  camera: {
    title: "Use your camera?",
    points: [
      "Your camera opens only when you tap Camera.",
      "The photo is kept in this notebook, on this device.",
      "Nothing is uploaded or shared unless you export or share it.",
    ],
    allow: "Allow camera",
  },
  media: {
    title: "Open your photos and files?",
    points: [
      "You choose each picture; ProNotes sees only what you pick.",
      "Your other photos and files stay private.",
      "Pictures are kept on this device, never uploaded.",
    ],
  },
};

```

```

    },
    allow: "Allow access",
  },
};

const SOURCES: Array<{ id: Source; label: string; hint: string; icon: typeof Camera; tint: string }> = [
  { id: "camera", label: "Camera", hint: "Take a photo", icon: Camera, tint: "from-rose-500 to-orange-400" },
  { id: "photos", label: "Photos", hint: "From your gallery", icon: Images, tint: "from-indigo-500 to-violet-500" },
  { id: "files", label: "Files", hint: "Browse storage", icon: FileImage, tint: "from-sky-500 to-cyan-400" },
];

interface Recent {
  id: string;
  url: string;
}

export function ImageInsertButton({ flyout }: { flyout: "top" | "bottom" | "left" | "right" }) {
  const [open, setOpen] = React.useState(false);
  const [asking, setAsking] = React.useState<Source | null>(null);
  const [consent, setConsent] = React.useState<Record<Access, boolean>>({ camera: false, media: false });
  const [placeAs, setPlaceAs] = React.useState<PlaceAs>("picture");
  const [busy, setBusy] = React.useState<string | null>(null);
  const [error, setError] = React.useState<string | null>(null);
  const [recent, setRecent] = React.useState<Recent[]>([]);
  const inputs = {
    camera: React.useRef<HTMLInputElement>(null),
    photos: React.useRef<HTMLInputElement>(null),
    files: React.useRef<HTMLInputElement>(null),
  };
  const inNotebook = useWhiteboard((s) => {
    const b = s.boards.find((x) => x.id === s.activeBoardId);
    return !!b && isNotebook(b);
  });

  // Read on open: localStorage is absent during the static export, and the
  // answers may have changed since the sheet was last shown.
  React.useEffect(() => {
    if (!open) return;
    setConsent({ camera: readConsent("camera"), media: readConsent("media") });
    try {
      setPlaceAs(localStorage.getItem(PLACE_KEY) === "page" ? "page" : "picture");
    } catch {
      /* default placement */
    }
    setAsking(null);
    setError(null);
    let alive = true;
    listRecentImages(10).then((recs) => {
      if (!alive) return;
      // Only pictures still used somewhere are worth offering again.
      const used = new Set<string>();
      for (const b of useWhiteboard.getState().boards) {
        const pages = (b as { pages?: Array<{ type: string; src?: string }> })?.pages;
        const lists = [(b as { elements?: Array<{ type: string; src?: string }> })?.elements ?? []];
        if (pages) for (const p of pages) lists.push(p.elements ?? []);
        for (const els of lists) for (const el of els) if (el.type === "image" && el.src) used.add(el.src);
      }
      setRecent(
        recs
          .filter((r) => used.has(`${ASSET_PREFIX}${r.id}`))
          .map((r) => ({ id: r.id, url: r.dataUrl }));
      );
    });
    return () => {
      alive = false;
    };
  }, [open]);

  const choosePlace = (p: PlaceAs) => {
    setPlaceAs(p);
    try {
      localStorage.setItem(PLACE_KEY, p);
    } catch {
      /* remembered placement is a convenience */
    }
  };
};

```

```

/** Launch a source. Must run inside the tap, or Android blocks the picker. */
const launch = (src: Source) => {
  if (!consent[accessOf(src)]) {
    setAsking(src);
    return;
  }
  setError(null);
  inputs[src].current?.click();
};

const allow = () => {
  if (!asking) return;
  const a = accessOf(asking);
  writeConsent(a, true);
  setConsent((c) => ({ ...c, [a]: true }));
  const src = asking;
  setAsking(null);
  inputs[src].current?.click(); // still inside the Allow tap
};

const revoke = (a: Access) => {
  writeConsent(a, false);
  setConsent((c) => ({ ...c, [a]: false }));
};

const handleFiles = async (files: File[], source: Source | "paste") => {
  const pictures = files.filter(isImageFile);
  if (pictures.length === 0) {
    setError(files.length ? "That file is not a picture." : null);
    return;
  }
  setBusy(pictures.length === 1 ? "Adding picture..." : `Adding ${pictures.length} pictures...`);
  setError(null);
  try {
    const prepared = await Promise.all(pictures.map((f) => prepareImage(f)));
    if (placeAs === "page" && inNotebook) {
      const measured = await Promise.all(
        prepared.map(
          (dataUrl) =>
            new Promise<{ src: string; width: number; height: number }>((resolve, reject) => {
              const img = new Image();
              img.onload = () =>
                storeImage(dataUrl).then((src) =>
                  resolve({ src, width: img.naturalWidth, height: img.naturalHeight })
                );
              img.onerror = () => reject(new Error("A picture could not be read."));
              img.src = dataUrl;
            })
        )
      );
      useWhiteboard.getState().addImagePages(measured);
    } else {
      prepared.forEach((dataUrl, index) =>
        window.dispatchEvent(
          new CustomEvent("pronotes:insertimage", {
            // A fresh photo nearly always wants cropping: open the editor.
            detail: { dataUrl, index, edit: source === "camera" && prepared.length === 1 },
          })
        )
      );
    }
    setOpen(false);
  } catch (err) {
    setError((err as Error).message || "The picture could not be added.");
  } finally {
    setBusy(null);
  }
};

const onPicked = (source: Source) => (e: React.ChangeEvent<HTMLInputElement>) => {
  const files = Array.from(e.target.files ?? []);
  e.target.value = "";
  if (files.length) void handleFiles(files, source);
};

const paste = async () => {
  setError(null);
  // A capture copied inside ProNotes that the system clipboard refused (the

```

```

// Android WebView will not take pictures): it lives on the app's own
// clipboard, and pasting it here must work all the same.
const own = appClipForPaste();
if (own) {
  if (placeAs === "page" && inNotebook) {
    const src = await storeImage(own.dataUrl);
    useWhiteboard.getState().addImagePages([
      { src, width: own.width, height: own.height }
    ]);
  } else {
    window.dispatchEvent(
      new CustomEvent("pronotes:insertimage", {
        detail: { dataUrl: own.dataUrl, size: own.worldSize }
      })
    );
  }
  setOpen(false);
  return;
}
try {
  const items = await navigator.clipboard.read();
  const blobs: File[] = [];
  for (const item of items) {
    const type = item.types.find((t) => t.startsWith("image/"));
    if (type) {
      const blob = await item.getType(type);
      blobs.push(new File([blob], "pasted", { type }));
    }
  }
  if (blobs.length === 0) {
    setError("There is no picture on the clipboard. Copy one first.");
    return;
  }
  await handleFiles(blobs, "paste");
} catch {
  // Copying may have worked while reading back is refused (Android): a
  // capture copied here is still on ProNotes' own clipboard.
  const mine = getAppClipImage();
  if (mine) {
    window.dispatchEvent(
      new CustomEvent("pronotes:insertimage", {
        detail: { dataUrl: mine.dataUrl, size: mine.worldSize }
      })
    );
    setOpen(false);
    return;
  }
  setError("ProNotes can't read the clipboard here. Close this and press Ctrl+V on the page.");
}
};

const reuse = (r: Recent) => {
  if (placeAs === "page" && inNotebook) {
    const img = new Image();
    img.onload = () => {
      useWhiteboard
        .getState()
        .addImagePages([
          {
            src: `${ASSET_PREFIX}${r.id}`,
            width: img.naturalWidth,
            height: img.naturalHeight
          }
        ]);
      setOpen(false);
    };
    img.src = r.url;
    return;
  }
  window.dispatchEvent(
    new CustomEvent("pronotes:insertimage", {
      detail: { dataUrl: `${ASSET_PREFIX}${r.id}` }
    })
  );
  setOpen(false);
};

const ask = asking ? CONSENT_COPY[accessOf(asking)] : null;

return (
  <>
    <input
      ref={inputs.camera}
      type="file"
      accept="image/*"
      capture="environment"
      className="hidden"
      onChange={onPicked("camera")}
    />
    <input
      ref={inputs.photos}
      type="file"
      accept="image/*"
      multiple
      className="hidden"
      onChange={onPicked("photos")}
    />
    </* Several types make Android open its file browser instead of the photo picker. */>
    <input
      ref={inputs.files}
      type="file"
      accept="image/png,image/jpeg,image/webp,image/gif,image/bmp,image/svg+xml,image/heic"
      multiple
    />
  </>

```

```

        className="hidden"
        onChange={onPicked("files")}
      />
    <Popover open={open} onOpenChange={setOpen}>
      <Tooltip>
        <TooltipTrigger asChild>
          <PopoverTrigger asChild>
            <button
              aria-label="Add a picture"
              className={cn(
                "p-2.5 rounded-xl transition-all",
                open ? "bg-indigo-500/15 text-indigo-600 dark:text-indigo-300" : "hover:bg-muted text-muted-foreground hover:text-foreground"
              )}
            >
              <ImageIcon size={18} />
            </button>
          </PopoverTrigger>
        </TooltipTrigger>
        <TooltipContent side={flyout}>Add a picture (or paste)</TooltipContent>
      </Tooltip>
    <PopoverContent
      side={flyout}
      align="center"
      className="w-[320px] max-w-[calc(100vw-24px)] overflow-hidden rounded-3xl border-border/70 p-0 shadow-w-2xl"
    >
      {ask && asking ? (
        <div className="p-5">
          <div className="mb-3 flex h-11 w-11 items-center justify-center rounded-2xl bg-emerald-500/12 text-emerald-600 dark:text-emerald-400">
            <ShieldCheck size={22} />
          </div>
          <h3 className="text-base font-semibold">{ask.title}</h3>
          <ul className="mt-2 space-y-1.5">
            {ask.points.map((p) => (
              <li key={p} className="flex gap-2 text-[13px] leading-snug text-muted-foreground">
                <span className="mt-1.5 h-1.5 w-1.5 shrink-0 rounded-full bg-emerald-500" />
                {p}
              </li>
            ))}
          </ul>
          <p className="mt-3 text-[11px] text-muted-foreground">You can turn this off any time at the bottom of this panel.</p>
          <div className="mt-4 flex gap-2">
            <button
              onClick={() => setAsking(null)}
              className="flex-1 rounded-xl border border-border py-2.5 text-sm font-medium hover:bg-muted"
            >
              Not now
            </button>
            <button
              onClick={allow}
              className="flex-1 rounded-xl bg-indigo-600 py-2.5 text-sm font-semibold text-white shadow-sm hover:bg-indigo-500"
            >
              {ask.allow}
            </button>
          </div>
        </div>
      ) : (
        <div className="relative">
          <div className="flex items-center justify-between px-4 pb-1 pt-3.5">
            <div>
              <h3 className="text-sm font-semibold">Add a picture</h3>
              <p className="text-[11px] text-muted-foreground">Crop, flip and restyle it after it lands.</p>
            </div>
            <button
              onClick={() => setOpen(false)}
              aria-label="Close"
              className="rounded-full p-1.5 text-muted-foreground hover:bg-muted"
            >
              <X size={15} />
            </button>
          </div>
          <div className="grid grid-cols-3 gap-2 px-3 pt-2">
            {SOURCES.map(({ id, label, hint, icon: Icon, tint }) => (
              <button
                key={id}

```

```

      onClick={() => launch(id)}
      disabled={!busy}
      className="group flex flex-col items-center gap-1.5 rounded-2xl border border-border/60 bg
-muted/30 px-1 pb-2.5 pt-3 transition hover:border-indigo-400/60 hover:bg-indigo-500/5 active:
scale-[0.97] disabled:opacity-50"
    >
      <span className={cn("flex h-11 w-11 items-center justify-center rounded-2xl bg-gradient-to
-br text-white shadow-md transition group-hover:scale-105", tint)}>
        <Icon size={21} />
      </span>
      <span className="text-[13px] font-semibold leading-none">{label}</span>
      <span className="text-[10px] leading-none text-muted-foreground">{hint}</span>
    </button>
  )}}
</div>

<button
  onClick={paste}
  disabled={!busy}
  className="mx-3 mt-2 flex w-[calc(100%-24px)] items-center gap-2 rounded-xl px-3 py-2 text-[13
px] text-muted-foreground transition hover:bg-muted hover:text-foreground disabled:opacity-50"
  >
    <ClipboardPaste size={15} /> Paste from clipboard
  </button>

{inNotebook && (
  <div className="mx-3 mt-1 grid grid-cols-2 gap-1 rounded-xl bg-muted/60 p-1" role="radiogroup"
    aria-label="Place as">
    {
      [
        { id: "picture" as const, label: "On this page", icon: StickyNote },
        { id: "page" as const, label: "As a new page", icon: FilePlus2 },
      ]
    }.map(({ id, label, icon: Icon }) => (
      <button
        key={id}
        role="radio"
        aria-checked={placeAs === id}
        onClick={() => choosePlace(id)}
        className={cn(
          "flex items-center justify-center gap-1.5 rounded-lg py-1.5 text-[12px] font-medium tr
ansition",
          placeAs === id ? "bg-background text-foreground shadow-sm" : "text-muted-foreground ho
ver:text-foreground"
        )}
      >
        <Icon size={13} /> {label}
      </button>
    )}}
  </div>
)}

{recent.length > 0 && (
  <div className="mt-3">
    <div className="px-4 text-[11px] font-medium uppercase tracking-wide text-muted-foreground">
      Use again</div>
    <div className="flex gap-2 overflow-x-auto px-3 pb-1 pt-1.5 [scrollbar-width:thin]">
      {recent.map((r) => (
        <button
          key={r.id}
          onClick={() => reuse(r)}
          className="h-16 w-16 shrink-0 overflow-hidden rounded-xl border border-border/60 bg-mu
ted transition hover:ring-2 hover:ring-indigo-400"
          aria-label="Add this picture again"
        >
          {/* eslint-disable-next-line @next/next/no-img-element */}
          <img src={r.url} alt="" className="h-full w-full object-cover" draggable={false} />
        </button>
      )}}
    </div>
  </div>
)}

{error && <p className="mx-4 mt-2 text-[12px] text-rose-600 dark:text-rose-400">{error}</p>}

<div className="mt-3 flex items-center gap-2 border-t border-border/60 bg-muted/30 px-4 py-2.5 t
ext-[11px] text-muted-foreground">
  <ShieldCheck size={13} className="shrink-0 text-emerald-600 dark:text-emerald-400" />

```

```

    <span className="flex-1">Pictures stay on this device.</span>
    {[["camera", "media"] as Access[]].map((a) =>
      consent[a] ? (
        <button key={a} onClick={() => revoke(a)} className="rounded-md px-1.5 py-0.5 underline-of
fset-2 hover:bg-muted hover:underline">
          {a === "camera" ? "Camera: on" : "Photos: on"}
        </button>
      ) : null
    )}]
  </div>

  {busy && (
    <div className="absolute inset-0 flex items-center justify-center gap-2 bg-background/80 text-
sm font-medium backdrop-blur-sm">
      <Loader2 size={16} className="animate-spin" /> {busy}
    </div>
  )}
</div>
)}
</PopoverContent>
</Popover>
</>
);
}

```

```

// === END FILE: src/components/whiteboard/ImageInsertSheet.tsx ===

```

```

// === START FILE: src/components/whiteboard/ImageStudio.tsx ===
// File 56 of 194 | Lines: 477 | Size: 18628 bytes

```

```

"use client";

/**
 * Image editor ("studio").
 *
 * Opens over the page for ONE picture: from its action bar, by double-tapping
 * it, or on its own after a fresh camera photo. Three tabs:
 *   Crop — drag the frame or its handles; fixed ratios; turn and mirror.
 *   Look — Original, Document (cleans up photos of pages), Gray, Vivid,
 *           Faded (pale, for writing on top).
 *   Shape — square, rounded or circle, with an optional border.
 *
 * Nothing is applied until Done, and Done is ONE undo step. Every edit is a
 * description on the element (lib/whiteboard/imageEdit.ts): the photo itself
 * is never re-encoded, so Reset can always bring back the whole original.
 */

import React from "react";
import { createPortal } from "react-dom";
import {
  Check,
  Crop,
  FlipHorizontal2,
  FlipVertical2,
  Palette,
  RotateCcw,
  RotateCw,
  Shapes,
  Undo2,
  X,
} from "lucide-react";
import { useWhiteboard, useActiveBoard } from "@store/whiteboard";
import { getBoardElements } from "@lib/whiteboard/notebook";
import type { ImageCrop, ImageElement, ImageFilter, ImageFrame } from "@lib/whiteboard/types";
import {
  FULL_CROP,
  IMAGE_FILTERS,
  IMAGE_FRAMES,
  boxForCrop,
  centeredAspectCrop,
  dragCrop,
  filterCss,
  fitAspect,

```